

Agent Harness: A Practitioner's Textbook

A fact-based synthesis of contemporary writing on harness engineering, drawn from the [Awesome Harness Engineering](#) reading list and the [OpenReview](#) survey [Agent Harness Engineering: A Survey](#). Every substantive claim is referenced inline.

21 sections · generated from Markdown sources

Preface

This book is about the system that surrounds a language model when it is asked to do real work. That system has a name now — *the harness* — and a small but rapidly maturing body of literature describing how to build it. The chapters below stitch that literature into a single narrative. Readers who want to follow any thread back to its source will find the original article cited at the relevant claim.

The premise of the field is simple. As Vivek Trivedy of LangChain puts it: "Agent = Model + Harness. **If you're not the model, you're the harness.**" ([LangChain — The Anatomy of an Agent Harness](#)). Everything else — system prompts, tools, sandboxes, memory, sub-agents, control flow, evaluation infrastructure — is the harness. The work of designing it well is what we will study.

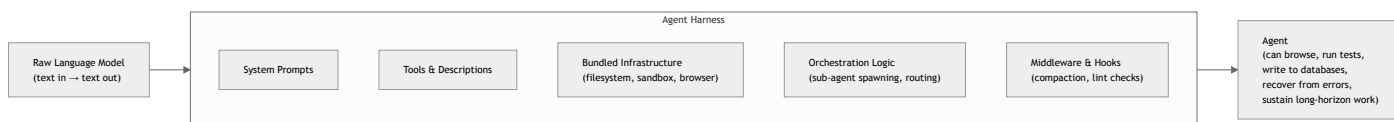
Throughout the book, a model should be understood as a component that consumes tokens and emits tokens. It may emit text for a user, or structured text that requests an action, but the action itself is always carried out by surrounding software. That distinction — model output versus real-world effect — is the reason the later chapters spend so much time on context, tools, state, tests, sandboxes, and evaluation.

The model internals this book relies on — tokens, attention and the KV-cache, the context window, sampling, retrieval, and the tool-call protocol — are covered in the companion stage-1 volume; here we name each concept in a line and build the harness-engineering layer on top of it. The genuinely new material is mostly software engineering: context, tools, state, sandboxes, and evaluation.

Before You Start

This is the second book in a two-part sequence. It assumes you have read *LLM Foundations for Harness Engineering* and are comfortable with tokens, attention and the KV-cache, the context window, sampling, post-training, retrieval, the agent loop, the tool-call protocol, prompt injection, and pass@k/pass^k. Where this book names a Foundations concept, it points back (for example, "Foundations ch 9") rather than re-deriving it. A reader arriving without the stage-1 material can still follow the narrative, but should treat the one-line recaps as pointers, not as the full teaching.

The Core Equation



Key Takeaways

- A language model alone cannot maintain state, execute code, or access real-time knowledge — these are all harness-level features.
- Harness engineering is distinct from prompt engineering: it iterates on the entire system, not just individual prompts.
- The field is young — most canonical articles were published in 2025 and 2026 — but is maturing rapidly.
- Every claim in this textbook is cited so readers can follow threads back to primary sources.

Further Reading

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>

Chapter 1: What Is an Agent Harness?

1.1 The Model + Harness Equation

A raw language model takes text in and produces text out — its native capability and limits are the subject of the companion volume (see Foundations ch 1). To turn that into an agent — something that can browse a codebase, run tests, write to a database, talk to a user, recover from errors, and sustain progress across hours of work — every additional capability must be built around the model. LangChain enumerates the harness as system prompts, tools and their descriptions, bundled infrastructure (filesystem, sandbox, browser), orchestration logic such as sub-agent spawning and model routing, and hooks or middleware for deterministic execution like compaction, continuation, and lint checks ([LangChain — The Anatomy of an Agent Harness](#)).

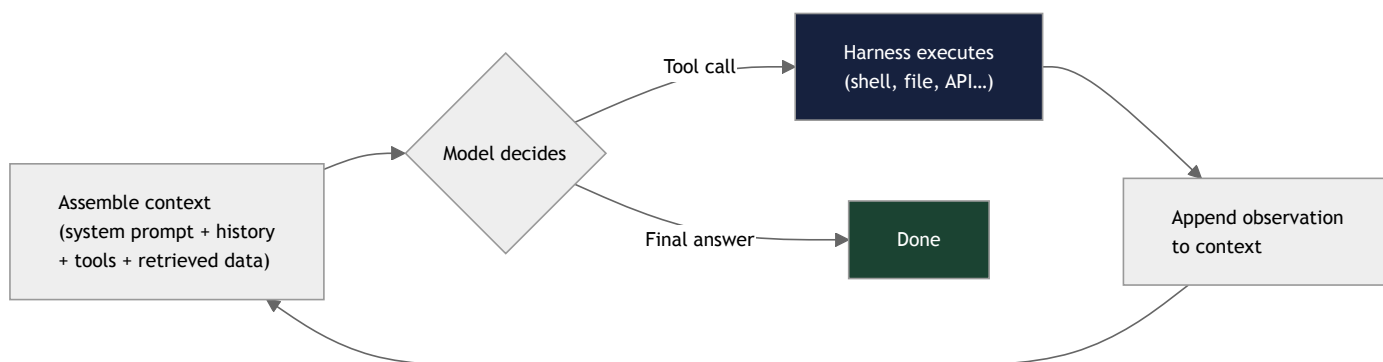
This framing matters because it forces the design question into the open. Out of the box, a model cannot maintain durable state across interactions, execute code, access real-time knowledge, or set up environments and install packages to complete work; these are all harness-level features ([LangChain — The Anatomy of an Agent Harness](#)). Even basic chat — the appearance of a model "remembering" what was just said — is a harness pattern: a while-loop that tracks previous messages and appends new ones to context.

HumanLayer's working definition is essentially the same equation, viewed from the perspective of someone configuring a coding agent: "coding agent = AI model(s) + harness," where the harness is the agent's runtime, or its peripherals — what the model uses to interact with its environment ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

1.2 The Agent Loop

The agent loop — and the principle that a tool call is structured output the harness executes, not direct agency by the model — is established in the companion volume (see Foundations ch 1 and ch 12). As a quick recap: a raw model call is one-shot (text in, text out); an *agent* wraps it in a loop that assembles context, lets the model emit either a final answer or a *tool call* (structured output, typically JSON, naming a tool and arguments), has deterministic harness code execute that call, appends the observation, and repeats on the now-longer context. HumanLayer states the same point as "tools are just structured outputs" ([HumanLayer — 12-Factor Agents](#)): file edits, shell commands, browser clicks, and database writes become real only when the harness accepts the model's request and runs it. The model at the centre of this loop is what Anthropic calls the *augmented LLM* and the research literature calls *ReAct* (reason + act) (mechanics in Foundations ch 12).

Two consequences of this loop shape the rest of the book. First, **context grows monotonically**: every turn appends a tool call and its observation, so an N-step task accumulates N rounds of history. This is why Chapter 2 treats context as a finite resource, and why compaction, sub-agents, and memory exist at all. Second, **the model never executes anything itself** — it only emits a request, and deterministic harness code decides what to honour. That gap between request and execution is the insertion point for every guardrail, sandbox, hook, and approval gate in the later chapters: the harness is what sits in the loop, between what the model asks for and what actually happens.



1.3 Bounded Contexts: Inner and Outer Harness

The word "harness" is used loosely, and writers from Thoughtworks have noted that it covers different layers depending on whom you ask. Birgitta Böckeler proposes thinking of three concentric rings: the model at the core, the coding agent's *builder harness* in the middle (the system prompt and tools shipped by Anthropic, OpenAI, etc.), and the *user harness* on the outside (the AGENTS.md, hooks, skills, and review agents the team adds to fit their codebase) ([Thoughtworks / Martin Fowler — Harness Engineering](#)). Most working engineers operate primarily in the outer ring.

The OpenReview survey *Agent Harness Engineering: A Survey* gives the same boundary a more formal name: a harness is the software and interface substrate that governs how foundation models perceive context, call tools, act over time, and remain auditable in a deployment environment ([OpenReview — Agent Harness Engineering: A Survey](#)). Its binding-constraint thesis is stronger than "harnesses are useful": for long-horizon agents, reliability often depends on the surrounding substrate as much as on raw model quality.

1.4 ETCLOVG: A Seven-Layer Map

The survey organizes the harness design space with the acronym **ETCLOVG**: Execution environment, Tool interface, Context, Lifecycle, Observability, Verification, and Governance ([OpenReview — Agent Harness Engineering: A Survey](#)). This is useful because it prevents "harness" from collapsing into only prompts or only tools.

- **Execution environment**: the sandbox, browser, operating system, code runner, or managed cloud where actions become real effects.
- **Tool interface**: protocols, schemas, registries, function calling, MCP/A2A-style boundaries, and tool-selection policies.
- **Context**: prompt assembly, retrieval, memory, compaction, state compression, and what the model is allowed to see.
- **Lifecycle**: task startup, planning, checkpoint/resume, failure recovery, handoff, session termination, and long-running state.
- **Observability**: traces, telemetry, cost attribution, latency, token accounting, and failure forensics.
- **Verification**: eval harnesses, graders, task suites, outcome checks, and readiness gates.
- **Governance**: permissions, policy languages, audit trails, human approval, constitutional or rule-based controls, and cross-layer security.

Most chapters in this book can be read as a pass over those seven layers. Context and memory dominate Chapters 2-3; tools and execution appear in Chapters 4-5; lifecycle appears in Chapters 7-9; verification, observability, and governance are developed in Chapters 10-12 and revisited in the outlook.

1.5 Why Harnesses Exist: Working Backwards from Model Deficits

LangChain offers a useful derivation: list the agent behaviors you want, then list what models cannot natively do, and the harness components fall out as necessary remediation ([LangChain — The Anatomy of an Agent Harness](#)).

Filesystems exist because models can only operate on what is in the context window, and a filesystem provides durable storage, an offload surface, and a collaboration surface for multiple agents and humans. Bash and code execution exist because pre-defining every tool the agent might need is intractable, and giving the model a general-purpose execution channel lets it design its own tools on the fly. Sandboxes exist because that execution has to happen somewhere safe. Memory and search exist because models have no knowledge beyond their weights and current context, so any new information has to be injected. Compaction, tool-result offloading, and skills exist because the context window is finite and degrades as it fills.

Each piece is a response to a specific limitation, and the harness as a whole is the sum of those responses.

1.6 The Historical Arc: From Prompt Engineering to Harness Engineering

Anthropic frames the recent shift as a natural progression. In the early days of LLM applications, the dominant work was *prompt engineering*: writing and organizing instructions for one-shot tasks. As applications grew into multi-turn agents that operate over longer time horizons, the relevant work shifted to *context engineering* — strategies for curating and maintaining the optimal set of tokens (information) during LLM inference, including everything that lands in context outside of the prompts themselves ([Anthropic — Effective Context Engineering for AI Agents](#)).

Harness engineering sits one level up from context engineering. It is the practice, as Mitchell Hashimoto has put it, of taking the time to engineer a solution every time the agent makes a mistake, so that it never makes that mistake again ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#) quoting Hashimoto). Where prompt engineering tunes a single prompt, harness engineering iterates on the entire system in which prompts run.

A further reframing arrived in 2026 under the name *loop engineering*. As agents began running unattended over long horizons, the salient unit of work shifted once more — from the prompt, to the context, to the *loop* that decides what to prompt, when, and whether the result is good enough ([Addy Osmani — Loop Engineering](#)). This is less a rival to harness engineering than its operator-facing view of the outer control loop — the trigger, the verifier, and the stop rules that surround the agent — and Chapter 8 develops it in full.

1.7 Frameworks, Runtimes, and Harnesses

These three terms are sometimes used interchangeably. LangChain's Harrison Chase distinguishes them as follows ([LangChain — Agent Frameworks, Runtimes, and Harnesses, Oh My!](#)):

A *framework* — like LangChain itself, Vercel's AI SDK, CrewAI, the OpenAI Agents SDK, or Google ADK — provides abstractions that make it easier to get started and standardize how applications are built. A *runtime* — like LangGraph, Temporal, or Inngest — provides infrastructure-level concerns: durable execution, streaming, human-in-the-loop support, thread-level and cross-thread persistence. A *harness* — like LangChain's DeepAgents or Anthropic's Claude Agent SDK — sits a level higher again: it ships with default prompts, opinionated tool handling, planning tools, filesystem access, and other "batteries included" features. Lines blur (LangGraph is reasonably described as both runtime and framework), but the distinction is useful when deciding what to adopt.

1.8 The Skeptical Counter-Position

The harness-engineering frame is not without dissent. HumanLayer's argument in "Skill Issue" is that most teams blame the model — "GPT-6 will fix it," "we just need better instruction-following" — when the actual issue is harness configuration ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)). As models improve, existing failure modes will disappear, but smarter models will be given harder problems and continue to fail in unexpected ways, because unexpected failures are a fundamental property of non-deterministic systems. The implication is that harness engineering is permanent work, not scaffolding to be discarded once models get good enough.

LangChain reaches a similar conclusion: as models become more capable, some of what lives in the harness today will be absorbed into the model, but harness engineering will remain useful — both for patching over deficiencies and for engineering systems around model intelligence to make it more effective ([LangChain — The Anatomy of an Agent Harness](#)).

Diagram: Model → Harness Layers



Key Takeaways

- **Agent = Model + Harness:** every capability beyond raw text I/O must be engineered into the surrounding system.
- **The agent loop is the foundation:** assemble context, the model emits a tool call, the harness executes it, the result is appended — and the cycle repeats, growing context every turn.
- **Tool calls are structured requests:** the model proposes actions in text; the harness decides which requests become real effects.
- **Three concentric layers:** the LLM core, the builder harness (shipped by the AI lab), and the user harness (built by the team).
- **ETCLOVG gives a systems map:** execution, tools, context, lifecycle, observability, verification, and governance are all harness layers.
- **Harness components derive from model deficits:** filesystem, sandbox, memory, compaction each address a specific limitation.
- **Harness engineering is permanent work:** as models improve, harder problems are tackled and new failure modes emerge.
- **Framework ≠ Runtime ≠ Harness:** understanding the distinction helps in making adoption decisions.

Further Reading

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>

- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses, Oh My!*, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

Chapter 2: Context as a Finite Resource

2.1 Context Rot and the Attention Budget

The single most important operating constraint for agent harnesses is that useful context is not the same thing as maximum context length. The underlying failure mode — *context rot*, the degradation of model output as the input grows — is introduced in the companion volume (see Foundations ch 9) with its two-layer model (length-induced plus irrelevant-material accumulation). Here we treat it as the chapter's governing design constraint. Anthropic links the same failure mode to needle-in-a-haystack benchmarks: as the number of tokens increases, the model's ability to accurately recall information from that context can decrease ([Anthropic — Effective Context Engineering for AI Agents](#)). The effect varies by model, task, and where the relevant information sits in the context, so the practical claim should be read probabilistically rather than as a hard law for every prompt.

Anthropic's mechanistic explanation is not that transformers literally "run out" of attention, but that long contexts create many more pairwise token relationships for the model to represent, while training data and positional mechanisms are usually stronger on shorter and more local dependencies. Position encoding interpolation and other long-context techniques let models handle longer sequences than they were originally trained on, but can still degrade positional resolution or retrieval reliability ([Anthropic — Effective Context Engineering for AI Agents](#)).

The practical conclusion: context is a finite resource with diminishing marginal returns. Anthropic phrases it as an "attention budget" that every new token spends. HumanLayer puts it more bluntly: even as models support longer context windows, production builders should usually prefer small, focused prompts and contexts; in their experience, open-ended "tool-calling loops" often become hard to recover from after roughly 10–20 turns ([HumanLayer — 12-Factor Agents](#)).

A bigger context window helps when the missing information genuinely needs to be present, but it does not automatically buy better attention or instruction-following. As HumanLayer points out, extended-context releases often use techniques like YaRN to extend sequence length rather than increasing the model's effective instruction budget; for needle-in-a-haystack-style tasks, a bigger window can simply make the haystack bigger ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

2.2 The Anatomy of Effective Context

Given the budget constraint, the goal — Anthropic's phrasing — is "the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome" ([Anthropic — Effective Context Engineering for AI Agents](#)). Concretely:

System prompts should be at the right *altitude* — neither hardcoded if-else logic for every edge case nor vague high-level guidance, but specific enough to guide behavior while leaving the model strong heuristics. Anthropic recommends organizing prompts into sections (`<background_information>`, `<instructions>`, `## Tool guidance`, etc.) using XML or Markdown delimiters, though formatting matters less as models improve.

Tools define the contract between agent and environment. They should be self-contained, clearly described, and minimally overlapping. The most common failure Anthropic sees is bloated tool sets that cover too much functionality with ambiguous decision points; "if a human engineer can't definitively say which tool should be used in a given situation, an AI agent can't be expected to do better."

Examples (few-shot) should be diverse and canonical, not a laundry list of every edge case. Anthropic's analogy: for an LLM, examples are the "pictures" worth a thousand words.

2.3 The KV-Cache: Why Stable Prefixes Pay Off

The KV-cache and the prefix-stability rule — including the "change one early token, such as a timestamp in the system prompt, and every token after it is recomputed" mechanic — are covered in the companion volume (see Foundations ch 9); here we treat the cache as a production cost lever. A practical lever that does not appear in the academic literature on context engineering but is central to production agent design is the KV-cache. Manus argues its hit rate is "the single most important metric for a production-stage AI agent" ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

Why it dominates the cost model: a typical agent receives input, picks an action from its tool space, executes it, and appends the action and observation to context for the next turn. Context grows with every step while output stays short, making the prefilling-to-decoding ratio extremely skewed — Manus reports an average input-to-output token ratio of 100:1. Identical prefixes can be served from the KV-cache, dropping time-to-first-token and inference cost.

Manus cites Claude Sonnet pricing where cached input tokens cost \$0.30 per million and uncached cost \$3 per million — a 10x difference.

Manus's three rules for keeping the cache hot: keep the prompt prefix stable (a single-token difference invalidates the cache from that point on, so common mistakes like timestamping system prompts to the second are costly); make context append-only, with deterministic JSON serialization (some libraries do not guarantee key ordering, which silently breaks the cache); and mark cache breakpoints explicitly when the inference framework requires it.

2.4 Mask, Don't Remove

Manus's second principle is about action spaces. As tool counts grow — and MCP (the Model Context Protocol — see Chapter 4) made this easy by letting users plug in hundreds of tools — the impulse is to dynamically load and unload tools mid-iteration. Manus's experiments produce a clear rule: avoid this. Tool definitions live near the front of context, so any change invalidates the cache for everything downstream; and previous turns may reference tools that no longer exist, leading to schema violations or hallucinated calls ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

The alternative is action masking: keep the stable tool surface in context, then constrain which actions can be selected at a given state. Depending on the provider and harness, this may be implemented with logit constraints, tool-choice controls, response prefill, or a runtime validator that rejects disallowed actions ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)). Manus uses consistent action-name prefixes — `browser_*` for browser tools, `shell_*` for shell tools — so an entire group can be enforced or excluded with a simple constraint.

2.5 The File System as the Ultimate Context

Even with a 128K-token window (Manus's figure at the time; current windows are larger, but the dynamic is unchanged), real agentic work overruns context regularly. Observations from web pages and PDFs are huge; performance degrades long before the technical limit; and long inputs are expensive even with caching ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

Manus's solution, and one Anthropic and LangChain converge on, is to treat the filesystem as the agent's working memory: much larger than context, persistent across turns, directly operable by the agent, and indexed by paths the agent can use as references. It is not "memory" in the human sense; without good filenames, summaries, indexes, or retrieval habits, the agent can still fail to find what it wrote. Manus's compression strategies are deliberately *restorable* — a web page can be dropped from context as long as the URL is preserved, and a document's contents can be omitted if its path remains.

LangChain calls the filesystem "arguably the most foundational harness primitive" because it provides a workspace for reading data, code, and documentation; lets agents offload work incrementally instead of holding it all in context; and acts as a natural collaboration surface for multi-agent and human-agent coordination ([LangChain — The Anatomy of an Agent Harness](#)). Adding git on top brings versioning, rollback, and branching.

For coding agents, the practical version of that idea is stronger: the repository should be the system of record. OpenAI's Codex harness guidance argues that successful agent-first repositories keep the context an agent needs close to the code: `AGENTS.md` points to the right places, short design documents explain local architecture, and structural checks encode the rules that matter ([OpenAI — Harness Engineering](#)). If a decision lives only in Slack, a ticket, or a senior engineer's memory, it is not reliable context for a fresh agent session.

This is not an argument for dumping more text into the prompt. It is an argument for making repository facts discoverable. A useful cold-start test is to open a new agent session with no oral briefing and ask it five questions: what system is this, how is it organized, how do I run it, how do I verify it, and what is the current state? If the answers are not available from repo files and standard commands, the harness has a knowledge-visibility gap. Fixing that gap usually means small documents next to the code, stable entry points, progress files, and verification commands, not a giant root instruction file.

2.6 Just-in-Time Retrieval

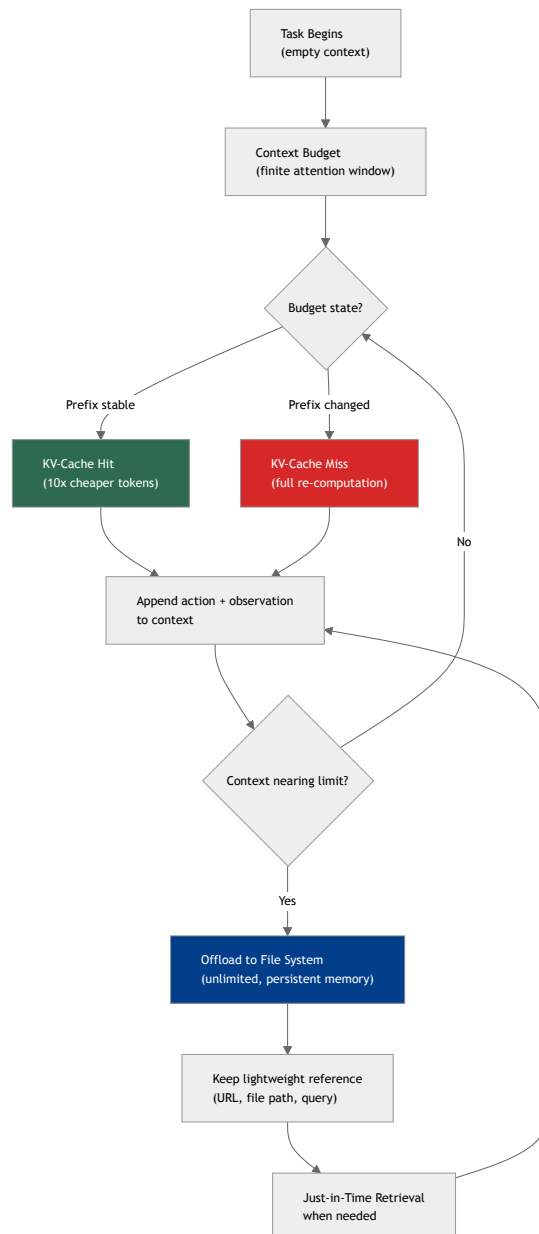
The traditional pattern — embed everything, retrieve top-k chunks, prepend to context (the RAG pipeline, see Foundations ch 11) — is being supplemented by a *just-in-time* approach. Rather than pre-processing everything up front, agents maintain lightweight identifiers (file paths, queries, links) and dynamically load data into context when needed ([Anthropic — Effective Context Engineering for AI Agents](#)).

Anthropic's Claude Code uses this pattern for large-codebase work: the model writes targeted queries, stores results, and uses tools such as `head` and `tail` to analyze large data without loading it all. The metadata of file paths itself is

informative — `test_utils.py` in a `tests/` folder implies a different role than the same name in `src/core_logic/`. Folder hierarchies, naming, and timestamps all become signals an agent can use to navigate.

The trade-off: runtime exploration is slower than retrieving pre-computed data, and an agent without proper tool guidance will waste context chasing dead ends. The hybrid pattern is now common — provide a small amount of high-value context up front, such as project instructions in `CLAUDE.md` or `AGENTS.md`, and let the agent explore the rest on demand.

Diagram: Context Budget → KV-Cache → File System as Memory



Key Takeaways

- **Context rot is real enough to design around:** larger contexts can help, but they do not remove the need for curation.
- **The KV-cache is a major production metric:** stable prefixes can matter as much as model quality for latency and cost.
- **Mask tools when possible, don't churn them casually:** dynamically adding/removing tools mid-run can break cache locality and cause schema violations.
- **The filesystem is working memory, not magic memory:** it is larger and persistent, but it still needs paths, summaries, and retrieval discipline.

- **The repository is the agent's system of record:** critical project facts need to be discoverable from repo files and commands during a cold start.
- **Just-in-time retrieval often beats pre-loading:** maintain lightweight identifiers and load data only when needed.

Further Reading

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>

Chapter 3: Compaction, Memory, and the Sub-Agent Pattern

Even with disciplined context engineering, long-horizon tasks exceed any single context window. The literature converges on three techniques.

3.1 Compaction

Compaction takes a conversation nearing the context window limit, summarizes it, and reinitiates a new context window with the summary ([Anthropic — Effective Context Engineering for AI Agents](#)). In Claude Code, the message history is passed to the model with instructions to preserve architectural decisions, unresolved bugs, and implementation details while discarding redundant tool outputs. The agent continues with the compressed context plus the most recently accessed files. The trigger — typically a token or percentage watermark on context usage — is itself a tuned parameter: fire too late and you risk overflow mid-turn, fire too early and you discard still-useful detail.

Anthropic's advice on compaction prompts: tune them on real complex traces, maximize recall first to ensure all relevant information is captured, then iterate on precision to remove the superfluous. The lightest-touch form is tool-result clearing — once a tool has been called and the result acted on, the raw result can usually be discarded.

The important caveat is that compaction is lossy. It is best for preserving decisions, goals, constraints, and pointers to artifacts, not for preserving every detail of a long debugging trace. A good harness therefore combines compaction with restorable references: commit hashes, file paths, issue IDs, URLs, and short notes that let a later agent reload primary evidence when the summary is too thin.

3.2 Structured Note-Taking

The complementary pattern is *agentic memory*: having the agent regularly write notes to disk that can be reloaded later. Anthropic offers Claude playing Pokémon as a clean example — across thousands of game steps, the agent maintains tallies ("for the last 1,234 steps I've been training my Pokémon in Route 1, Pikachu has gained 8 levels toward the target of 10"), develops maps of regions, and tracks combat strategies, allowing it to resume multi-hour training sequences after context resets ([Anthropic — Effective Context Engineering for AI Agents](#)).

Manus's `todo.md` trick is a specialized form of this — but with a twist explored in the next section.

3.3 Recitation: Manipulating Attention Through the End of Context

Manus reports that when its agent handles complex tasks, it creates a `todo.md` file and rewrites it step-by-step as the task progresses, checking off completed items. This is not just for organization. A typical Manus task takes ~50 tool calls on average; in long contexts the model is vulnerable to drifting off-topic or forgetting earlier goals. By repeatedly rewriting the todo list, the agent recites its objectives into the *end* of the context, pushing the global plan into the model's recent attention span and avoiding the well-known "lost-in-the-middle" problem (the underlying effect is covered in the companion volume, *LLM Foundations* ch9) ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

3.4 Sub-Agents and the Context Firewall

The third pattern, and the most architecturally consequential, is sub-agent decomposition. A specialized sub-agent handles a focused task with its own context window, may use tens of thousands of tokens internally, and returns only a condensed 1,000–2,000 token summary to the parent ([Anthropic — Effective Context Engineering for AI Agents](#)). HumanLayer calls this the *context firewall*: the parent thread, responsible for orchestration, never sees the intermediate noise from sub-agent work and stays out of the "dumb zone" for far longer ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

HumanLayer is emphatic about what does and does not work here. Setting up "frontend engineer" and "backend engineer" personas as sub-agents does not work; using sub-agents for context control does. Good sub-agent use cases are tasks with a simple final answer but many intermediate tool calls — locating a definition in the codebase, tracing information flow across services, running broad research.

Sub-agents also help with cost control: HumanLayer uses an expensive model (Opus) for the orchestrator and a cheaper model (Sonnet or Haiku) for sub-agents. There is no need to burn Opus tokens on a `grep`.

Anthropic's multi-agent research system is the canonical example of this pattern at scale. A lead agent analyzes the query and spawns specialized sub-agents to explore aspects in parallel; each sub-agent uses its own context window; results are compressed back to the lead, which synthesizes a final report. The lead-agent-as-Opus, sub-agents-as-Sonnet configuration outperformed single-agent Opus by 90.2% — a relative improvement — on

Anthropic's internal research evaluation ([Anthropic — How We Built Our Multi-Agent Research System](#)). The mechanism is largely token economics: in their analysis, three factors explained 95% of performance variance on the BrowseComp benchmark, with token usage alone explaining 80%.

The catch is cost. In Anthropic's data, single-agent runs use roughly 4× the tokens of a chat and multi-agent systems roughly 15× — so a multi-agent system costs on the order of 4× a single agent (a derived ratio, not a figure the source states directly). They therefore only make economic sense for high-value tasks where parallelization actually helps. They are a poor fit for tightly coupled subtasks that share mutable state — many implementation-heavy coding tasks fall into this category — but they can still help coding workflows when the delegated work is read-only, investigative, or cleanly separated by ownership boundary. Current models are also not strong at real-time coordination across agents, so the coordinator must keep task boundaries explicit.

3.5 Don't Few-Shot Yourself Into a Rut

Manus offers a counter-intuitive principle: too much consistency in the context can be harmful. Models are excellent mimics — they imitate patterns in context. If the trace is full of similar past action-observation pairs, the model will follow that pattern even when it is no longer optimal, leading to drift, overgeneralization, and hallucination. Manus's example is reviewing a batch of 20 résumés, where the agent falls into a rhythm and starts repeating actions for their own sake ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

Their fix: introduce small, structured variation — different serialization templates, alternate phrasing, minor reordering, controlled noise. Diversity in the trace keeps attention spread.

3.6 Keep the Wrong Stuff In

The complementary principle: do not erase useful errors. The natural impulse is to retry failed actions and hide the failed traces, but Manus argues this removes evidence the model needs to update its priors away from similar mistakes ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)). Leaving the latest failed action and relevant stack trace in context lets the model implicitly learn from it. This does not mean preserving unlimited logs forever; repeated identical failures should be compacted into a short diagnosis plus a retry counter. Manus calls error recovery "one of the clearest indicators of true agentic behavior" — and notes it is underrepresented in academic benchmarks, which tend to focus on success under ideal conditions.

HumanLayer formalizes this as Factor 9: compact errors into context. The agent's *self-healing* property — reading an error and adjusting its next call — is one of the genuine benefits of LLM agents, and it works only when the error is visible ([HumanLayer — 12-Factor Agents](#)). With a counter to limit consecutive identical errors, this pattern is robust.

3.7 Cost, Latency, and Model Routing

The sub-agent pattern above already used cost as a design variable — an expensive model orchestrating, cheaper models doing the legwork. It is worth making the general principle explicit: cost and latency are first-class harness concerns, not afterthoughts.

Three levers recur across the literature:

- **Model routing.** Not every step needs the strongest model. A harness can route cheap, high-volume work — a `grep`, a classification, a short summary — to a small fast model, and reserve the frontier model for reasoning-heavy steps. HumanLayer uses Opus for the orchestrator and Sonnet or Haiku for sub-agents ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)); Anthropic's research system makes the same lead-agent / sub-agent split (chapter 7).
- **The KV-cache.** Stable context prefixes are served from cache at roughly a tenth of the price, and a fraction of the latency, of uncached tokens (mechanics covered in the companion volume, *LLM Foundations* ch9; see also chapter 2). Cache discipline is often the single largest cost lever in a production agent.
- **Token accounting.** Agentic workloads are prefill-heavy — Manus reports a ~100:1 input-to-output ratio — and cost scales with context length. Multi-agent systems can burn roughly 15× the tokens of a single chat, which is why they pay off only on high-value tasks.

Latency has its own structure. Time-to-first-token is dominated by prefill, and therefore by cache hits; end-to-end latency is dominated by the number of *sequential* model round-trips. Parallel tool calls and parallel sub-agents cut wall-clock time substantially — up to 90% on Anthropic's research workloads (chapter 7) — without reducing total token cost. The general rule: treat tokens, dollars, and seconds as explicit budgets, and know which lever moves which.

3.8 Named Memory Architectures

Sections 3.1–3.3 treated compaction, note-taking, and recitation as harness *techniques*. The research literature also packages these ideas into named memory *systems* worth knowing in their own right.

- **MemGPT** draws an explicit analogy to an operating system: it treats the context window as fast "main memory" and external stores as "disk," and lets the model issue function calls to page information in and out of a fixed-size context — *virtual context management*. This is how a bounded window can present the appearance of a much larger one (Packer et al. — [MemGPT: Towards LLMs as Operating Systems](#)). The system is now productized as Letta.
- **Mem0** is a memory layer that dynamically *extracts* salient facts from a conversation, *consolidates* them against what is already stored, and *retrieves* them on later turns; a graph variant additionally captures relations between entities. Its reported win is operational — far lower token cost and latency than replaying full history across long multi-session dialogues (Chhikara et al. — [Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory](#)).
- **Sleep-time compute** puts the gaps between requests to work: instead of sitting idle, the agent processes its context offline — anticipating likely future questions and precomputing inferences — so that later queries need less test-time compute. On some benchmarks this cut the inference budget for a given accuracy by roughly 5× (Lin et al. — [Sleep-time Compute: Beyond Inference Scaling at Test-time](#)).

The harness lens is the one from §3.1: managed memory is powerful but lossy, and it adds its own failure surface — the wrong memory retrieved, a bad consolidation, a stale fact asserted with confidence. These systems earn their place when sessions are long and cross-session recall genuinely matters; on short tasks they are overhead layered on the note-taking that §3.2 already provides.

3.9 Multi-Agent Topologies and Why They Fail

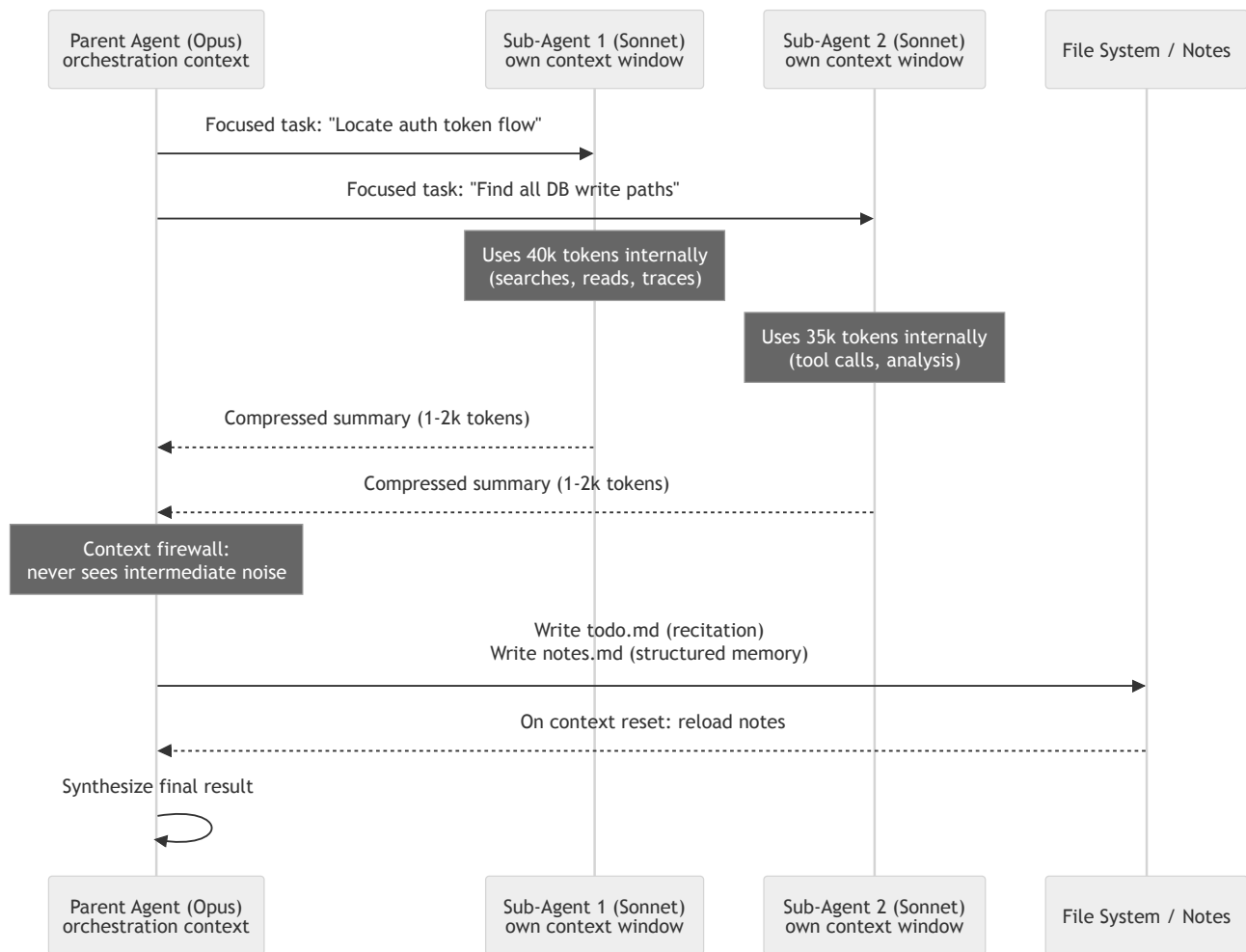
Section 3.4 introduced the sub-agent as a context-firewall device and the orchestrator-worker configuration; Chapters 6 and 7 develop orchestration further. Beyond orchestrator-worker, practitioners reach for a small vocabulary of multi-agent *topologies*:

- **Orchestrator-worker (supervisor)**: a lead agent decomposes a task, delegates to workers, and synthesizes their results (Ch 6, 7).
- **Hierarchical**: supervisors of supervisors, for tasks deep enough that one lead cannot hold the whole decomposition.
- **Blackboard / shared memory**: agents coordinate by reading and writing a common workspace rather than messaging each other directly — useful when many agents contribute to one evolving artifact.
- **Debate / voting**: several agents argue or vote to raise reliability, the multi-agent form of the parallelization-voting pattern (§6.3).

The temptation is to equate more agents with more capability. The empirical record is more sobering. The MAST study hand-annotated over 200 tasks across seven popular multi-agent frameworks and derived a taxonomy of 14 failure modes grouped into three categories: **specification issues** (underspecified roles and prompts), **inter-agent misalignment** (agents talking past each other, dropping information, or diverging from the shared goal), and **task verification** (weak or absent checking of the final result) (Cemri et al. — [Why Do Multi-Agent LLM Systems Fail?](#)). The headline is that a large share of failures are not raw model incapability but *coordination and verification* breakdowns — exactly the surfaces the harness owns.

The practical guidance falls out of the taxonomy: prefer the simplest topology that works (§6.1); keep task boundaries explicit so workers do not overlap or drop work (§3.4, Ch 7); and treat verification as a first-class agent rather than an afterthought (the generator–evaluator split of Ch 7), because MAST identifies weak verification as one of the three leading failure families.

Diagram: Parent Agent → Sub-Agent → Compressed Result (Context Firewall)



Key Takeaways

- **Compaction extends task horizons but loses detail:** preserve key decisions and restorable references, not every raw observation.
- **Structured note-taking enables multi-session continuity:** agents that write progress to disk can resume work after context resets.
- **Recitation defeats "lost-in-the-middle":** repeatedly rewriting a todo list pushes goals into the model's recent attention span.
- **The context firewall is the sub-agent pattern's key value:** the parent never sees intermediate noise; it receives only condensed results.
- **Leave useful errors in context:** self-healing only works when the relevant error trace is visible, but repeated failures should be compacted.
- **Cost and latency are design variables:** route cheap work to small models, keep the KV-cache warm, and parallelize for wall-clock speed.
- **Named memory systems package the memory patterns:** MemGPT (OS-style virtual context), Mem0 (extract-consolidate-retrieve), and sleep-time compute (offline pre-processing) are worth knowing — but each adds its own retrieval and staleness failure surface.
- **More agents multiply coordination failure, not just cost:** the MAST taxonomy finds that specification gaps, inter-agent misalignment, and weak verification — all harness-owned surfaces — dominate multi-agent failures; prefer the simplest topology and make verification first-class.

Further Reading

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
- Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
- Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>

Chapter 4: Tools and the Agent–Computer Interface

4.1 Why Tool Design Is Different

Anthropic introduces the term *agent–computer interface* (ACI) by analogy with HCI: as much engineering should go into how an agent uses tools as goes into how a human uses a screen ([Anthropic — Building Effective Agents](#)). Three concrete recommendations for tool format:

- Give the model enough tokens to "think" before it has to commit to syntax it cannot easily un-write.
- Keep formats close to what the model has seen in training data.
- Avoid formatting overhead like accurate line-counting in diff headers or excessive string-escaping in JSON-embedded code.

When Anthropic built its SWE-bench agent, it spent more time optimizing tool schemas than the prompt itself. A specific improvement: switching tools from relative to absolute filepaths fixed nearly all path-related errors after the agent moved out of the root directory ([Anthropic — Building Effective Agents](#)).

4.2 Choosing the Right Tools — and the Right Number

Anthropic's later "Writing Effective Tools for Agents" elaborates on the central trap: more tools do not lead to better outcomes ([Anthropic — Writing Effective Tools for Agents](#)). A common error is wrapping every API endpoint as a tool whether or not it suits an agent. Agents have different *affordances* than traditional software: an agent searching an address book by `list_contacts` has to read every contact token-by-token, brute-force, because its context is limited; the right tool is `search_contacts` or `message_contact`.

Tools should consolidate frequently-chained operations. Rather than `list_users`, `list_events`, and `create_event`, build `schedule_event`. Rather than `read_logs`, build `search_logs`. Rather than `get_customer_by_id` + `list_transactions` + `list_notes`, build `get_customer_context`.

4.3 Where Tools Come From: The Model Context Protocol

The sections around this one assume an agent with a set of well-designed tools. In practice, many of those tools arrive over a standard interface: the *Model Context Protocol* (MCP). As a quick recap, MCP is an open client–server standard: an *MCP server* exposes tools — and, optionally, resources and reusable prompts — over a uniform protocol, and any compatible *client* (Claude Code, an IDE, a custom agent) can discover and call them without bespoke integration. The transport mechanics and the untrusted-result caveat are covered in Foundations ch 12; this chapter treats MCP as a harness design surface ([Anthropic — Code Execution with MCP](#)).

The value is composability. A team can connect a Google Drive server, a Salesforce server, and an internal database server to the same agent — each built and maintained independently — and the agent sees one combined tool surface. This is why MCP recurs throughout this book: it is the substrate for the namespacing, masking, and code-execution patterns described in the rest of this chapter.

The cost is that MCP made tool over-supply trivial. As Chapter 2 noted, MCP let users plug in hundreds of tools, and a context loaded with hundreds of tool definitions is exactly the bloat that Chapter 2's *mask, don't remove* rule and the consolidation advice above push back on. MCP is plumbing, not a substitute for tool design: a poorly designed MCP server simply delivers poorly designed tools at scale. The discipline of this chapter — consolidate frequently-chained operations, namespace, cap responses, write descriptions like onboarding docs — applies whether a tool is hand-written or arrives over MCP.

4.4 Four Integration Boundaries

The OpenReview survey argues that tool and protocol standards are easier to compare by the boundary they cross, rather than by vendor or release date ([OpenReview — Agent Harness Engineering: A Survey](#)):

- **Model -> Function:** structured invocation such as function calling, where the model emits a machine-readable request for deterministic code.
- **Agent -> External capability:** MCP-style decoupling, where an agent runtime discovers tools, resources, and prompts exposed by external servers.
- **Agent -> Agent:** A2A-style delegation, where one agentic application hands work to another opaque agent with its own state and tools.
- **Agent -> Repo/environment:** version-controlled policy and affordances, such as AGENTS.md, local skills, repo commands, and environment-specific tool rules.

This boundary view clarifies why MCP, A2A, OpenAPI, function calling, and AGENTS.md should not be treated as direct substitutes. They solve different integration problems. The harness designer's job is to decide which boundary is actually in play, then choose a protocol and governance model that preserves provenance, permissions, costs, and failure evidence across that boundary.

4.5 Namespacing

When agents have access to dozens of MCP servers and hundreds of tools, name collisions and ambiguous purpose become critical failure modes. Anthropic recommends grouping related tools under common prefixes — service prefixes (`asana_*`, `jira_*`) and resource prefixes within those (`asana_projects_*`, `asana_users_*`). They report that prefix vs. suffix namespacing schemes have non-trivial effects on tool-use evaluations and that the right scheme is workload-specific ([Anthropic — Writing Effective Tools for Agents](#)).

Manus uses the same pattern for action-space control: by giving all browser tools a `browser_` prefix and all shell tools a `shell_` prefix, they can mask large groups of tools at once with simple logit constraints ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)).

4.6 Returning Meaningful Context

Tool responses should prioritize relevance over flexibility, and natural-language identifiers over technical ones. Anthropic finds that resolving alphanumeric UUIDs to semantically meaningful labels (or even 0-indexed IDs) significantly improves Claude's precision and reduces hallucinations ([Anthropic — Writing Effective Tools for Agents](#)). Where both are needed — natural names for the agent, technical IDs for downstream calls — a `response_format` enum with `concise` and `detailed` modes works well: concise responses can be a third the size of detailed ones in their Slack examples.

4.7 Token-Efficient Responses

Tool responses are a major source of context bloat. Anthropic restricts Claude Code's tool responses to 25,000 tokens by default, and recommends a combination of pagination, range selection, filtering, and truncation with sensible defaults ([Anthropic — Writing Effective Tools for Agents](#)). Truncated responses should include guidance steering the agent toward more efficient strategies (small targeted searches over one broad search, for instance), and error responses should be helpful, not opaque tracebacks.

HumanLayer's "back-pressure" practice in their own codebase is a direct application of this: their build and test hooks swallow output on success, surfacing only errors. Early on they had the agent run the full test suite after every change, and 4,000 lines of passing tests would flood the context window, causing the agent to lose track of the actual task and start hallucinating about test files ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

4.8 Prompt-Engineering Tool Descriptions

Anthropic positions this as one of the most effective levers, and reports that tool-description refinements were a key lever in reaching state-of-the-art on SWE-bench Verified with Claude Sonnet 3.5 ([Anthropic — Writing Effective Tools for Agents](#)). The advice: write the tool description as you would for a new junior engineer joining the team. Make implicit context explicit (specialized query formats, niche terminology, relationships between resources). Use unambiguous parameter names — `user_id` rather than `user`. Run many examples in a workbench, look at the mistakes, and iterate.

A concrete debugging example: when Anthropic launched Claude's web search tool, traces revealed that Claude was needlessly appending `2025` to the `query` parameter, biasing results. Fixing it required no model retraining — only a clearer tool description.

4.9 Code Execution as a Meta-Tool

A more recent shift: instead of presenting MCP tools as direct calls, present them as a code API that the agent invokes by writing code. Anthropic's "Code Execution with MCP" makes the case that for agents with hundreds of tools across dozens of MCP servers, the standard pattern of loading every tool definition into context up front and passing every intermediate result through the model is wasteful ([Anthropic — Code Execution with MCP](#)).

The alternative: expose MCP servers as a filesystem of TypeScript files, one per tool, each file containing a typed wrapper around `callMCPTool`. The agent discovers tools by listing directories and reading the specific tool files it needs. In Anthropic's Google Drive → Salesforce example, this drops token usage from 150,000 to 2,000 — a 98.7% saving.

The benefits compound:

- **Progressive disclosure:** tools are loaded only when needed, addressing the up-front context cost.
- **Context-efficient results:** the agent can filter a 10,000-row spreadsheet to five matching rows in the execution environment before any data crosses into the model's context.
- **Better control flow:** loops, conditionals, and error handling use familiar code patterns, saving on time-to-first-token because the runtime evaluates the conditions, not the model.
- **Privacy-preserving operations:** intermediate results stay in the execution environment by default; only what the agent explicitly logs reaches the model. With a correctly designed proxy, PII can be tokenized at the MCP-client boundary so raw values need not reach the model.
- **State persistence and skills:** agents can save working code as reusable functions backed by `SKILL.md` files, building up a toolbox over time.

Cloudflare reported similar findings under the name "Code Mode," reinforcing the conclusion: LLMs are good at writing code, and developers should let them ([Anthropic — Code Execution with MCP](#)).

The catch: code execution requires sandboxing infrastructure, which has its own operational and security cost.

4.10 Iterative Tool Refinement With Evals

Anthropic's recommended workflow for tool development has four stages ([Anthropic — Writing Effective Tools for Agents](#)):

1. **Prototype** the tools in a local MCP server, test by hand, collect intuition.
2. **Build an evaluation** with realistic tasks (multiple tool calls, real data, no toy sandboxes), each paired with verifiable success criteria.
3. **Run the evaluation** programmatically, capturing traces that include planning summaries, tool calls, tool results, runtime, token counts, and tool errors. Where a model exposes a visible thinking mode, that can help debug behavior, but the eval should not depend on access to hidden chain-of-thought.
4. **Analyze results** by reading transcripts, paying attention to what agents *don't* say (LLMs do not always say what they mean), and refactor tools accordingly.

Anthropic ran this loop on their own internal Slack and Asana tools and found that a Claude-optimized version of human-written tools outperformed expert manual implementations on held-out test sets — a result that validates the loop and is an early instance of agents improving their own tools.

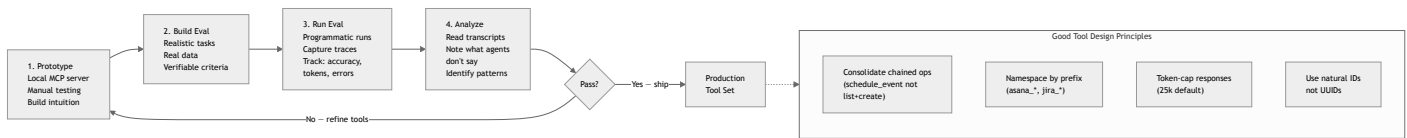
4.11 The Agent-to-Agent Boundary: A2A

Of the four integration boundaries in §4.4, three — model-to-function, agent-to-external-capability (MCP), and agent-to-repo/environment — have been developed above. The fourth, *agent-to-agent*, has its own emerging standard. The **Agent2Agent (A2A) protocol**, introduced by Google in April 2025 and donated to the Linux Foundation in June 2025, is an open standard for one agentic application to delegate work to another *opaque* agent — a peer with its own model, tools, memory, and internal state that it does not expose ([Agent2Agent \(A2A\) Protocol](#); [Linux Foundation — Agent2Agent Protocol Project](#)).

The mechanics are deliberately conventional so that existing web infrastructure applies: communication is JSON-RPC 2.0 over HTTP(S); each agent publishes an *Agent Card* describing its capabilities so others can discover it; and a task lifecycle covers submission, negotiation of interaction modality (text, files, structured data), and streaming of results back to the caller.

The contrast with MCP is the useful part. MCP connects an agent *downward* to tools and resources it controls and can inspect; A2A connects an agent *across* to a peer it does not control and cannot see inside. That flips the primary engineering concern from tool-schema design to *trust and provenance across an organizational boundary*. Because you cannot inspect the other agent's context or sandbox, a response from an A2A peer is untrusted content in exactly the sense of Chapter 5 — the lethal-trifecta discipline applies to peer agents as much as to web pages — and governance (scoped identity, delegated auth, audit) must span the A2A call rather than stopping at your own process (Ch 5, Ch 17). A2A does not remove the trust problem; it standardizes where it has to be solved.

Diagram: Tool Design Pipeline — Prototype → Eval → Iterate



Key Takeaways

- **Tool design deserves as much care as prompt design:** the ACI (agent-computer interface) analogy with HCI is apt.
- **More tools hurt, not help:** consolidate frequently-chained operations into single, purpose-built tools.
- **MCP standardizes where tools come from:** it makes tools composable across independent servers, but also makes over-supply easy — tool-design discipline applies regardless.
- **Tool protocols cross different boundaries:** function calling, MCP, A2A, and repo-local policy are complementary, not interchangeable.
- **Namespacing is not cosmetic:** it enables logit-level masking of tool groups and prevents collision in large MCP environments.
- **Tool responses are a major source of context bloat:** cap, paginate, filter, and truncate by default.
- **Code execution as meta-tool is a step-change:** exposing MCP tools as a typed code API cut token usage by 98.7% in Anthropic's example.
- **The four-stage eval loop is the recommended workflow:** prototype → build eval → run eval → analyze transcripts → iterate.
- **A2A standardizes the agent-to-agent boundary:** it lets one agent delegate to an opaque peer over JSON-RPC + Agent Cards — shifting the concern from tool-schema design to trust and provenance across an organizational boundary, where the lethal-trifecta and governance rules of Ch 5 apply.

Further Reading

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
- Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>

Chapter 5: Sandboxing, Guardrails, and Safe Autonomy

5.1 The Agent Security Threat Model

Most of this chapter is about *mitigations* — sandboxes, hooks, approval gates. It is worth first stating plainly what they mitigate. An agent that reads untrusted content and can act on the world has a specific risk profile ([Anthropic — Beyond Permission Prompts](#)):

- **Prompt injection** (see Foundations ch 8, "Prompt Injection as Context Confusion," and ch 12) — the model cannot reliably separate data from instructions, so untrusted content the agent reads (a web page, an issue comment, a source file, a tool result) can steer it as if it were a command.
- **Data exfiltration** — a steered agent with network access can send secrets — SSH keys, API tokens, proprietary source — to an attacker-controlled destination.
- **Destructive action** — a steered agent with filesystem or shell access can delete or corrupt files, or commit and push bad code.
- **Tool and supply-chain risk** — a malicious or compromised MCP server, package, or dependency can introduce hostile tools or instructions that the agent then trusts.

The combination practitioners worry about most is sometimes called the *lethal trifecta*: access to private data, exposure to untrusted content, and the ability to communicate externally ([Simon Willison — The lethal trifecta for AI agents](#)). Any one alone is survivable; all three in a single agent mean one injected instruction can read a secret and send it out. Most controls in this chapter work by breaking one leg of the trifecta — network isolation removes external communication, filesystem isolation removes private-data access, and approval gates put a human in the path of consequential actions.

The framing to carry into the rest of the chapter: the model is not a trusted component. It is a capable but steerable core, and the harness is what stands between a hostile instruction and a real-world consequence.

5.2 The Permission Fatigue Problem

Coding agents that run with no oversight are dangerous; coding agents that ask permission for every action are unusable. Anthropic frames this as approval fatigue: "Constantly clicking 'approve' slows down development cycles and can lead to 'approval fatigue,' where users might not pay close attention to what they're approving, and in turn making development less safe" ([Anthropic — Beyond Permission Prompts: Making Claude Code More Secure and Autonomous](#)). The solution is structural: define boundaries within which the agent can act freely, and only ask for permission when those boundaries are crossed.

In their internal usage, sandboxing safely reduces permission prompts by 84% ([Anthropic — Beyond Permission Prompts](#)).

5.3 Sandbox as Cage, Reset Button, and License

The OpenReview survey makes the sandbox's role broader than security. In agent systems, a sandbox has three simultaneous purposes ([OpenReview — Agent Harness Engineering: A Survey](#)):

- **Security**: it bounds the blast radius of unpredictable model-generated actions and prompt-injection-driven behavior.
- **Reproducibility**: it gives evals, training trajectories, and long-running sessions a resettable baseline. A container or microVM can be destroyed and rebuilt; a developer workstation cannot.
- **Liveness**: it defines a region where the agent is allowed to act without asking a human on every file write, package install, or network call.

That third purpose is specific to the agent era. A sandbox is not just a cage; it is also a license. By moving permission from a per-action question to a session configuration, it makes long-horizon autonomy usable without collapsing into approval fatigue.

5.4 Filesystem and Network Isolation Must Be Paired

Claude Code's sandbox enforces two boundaries simultaneously, and Anthropic argues both are required. Filesystem isolation prevents a prompt-injected agent from modifying sensitive files; network isolation prevents it from leaking data or downloading malware. Without network isolation, a compromised agent could exfiltrate SSH keys; without filesystem isolation, a compromised agent could escape the sandbox and reach the network ([Anthropic — Beyond Permission Prompts](#)).

The implementation builds on OS-level primitives — Linux bubblewrap and macOS seatbelt — and covers not just direct Claude Code interactions but any subprocess. Network access is funneled through a Unix domain socket to a proxy that enforces domain restrictions and handles user confirmation for newly requested domains. The runtime is open-sourced.

Claude Code on the web extends this to a cloud sandbox where sensitive credentials (git credentials, signing keys) are never inside the sandbox with the agent at all. A custom proxy handles git interactions, attaching scoped credentials only after validating that the operation is permitted (e.g., pushing only to the configured branch).

5.5 Governance: Identity, Policy, and Audit

Sandbox boundaries are necessary but not sufficient. The Governance layer asks who the agent is acting for, what authority it has, how authority changes with task context, and what evidence remains after the action ([OpenReview — Agent Harness Engineering: A Survey](#)).

Three design moves matter in production:

- **Identity and delegated auth:** the agent should act under scoped credentials or delegated identity, not under a user's full ambient authority. Credential vaults and proxies should attach secrets only at the boundary where the operation is authorized.
- **Context-dependent permission policy:** static allow/deny lists are inspectable but blunt. Task-aware policies can evaluate tool name, arguments, session state, target repo, network domain, and user role before each invocation, while a deterministic checker enforces the result.
- **Audit-grade traces:** logs must capture not only the tool call, but the identity, permission decision, policy version, arguments, output summary, and whether a human approved escalation.

This is also where supply-chain attacks become harness concerns. MCP tool poisoning, tool squatting, rug-pull updates, hallucinated packages, and retrieval-source poisoning all cross the line between "tool interface" and "governance." A safe harness needs provenance and integrity checks for tools, packages, datasets, and retrieval sources, not only prompts that say "be careful."

5.6 Hooks and Middleware as Programmatic Enforcement

The sandbox is one form of programmatic guardrail; hooks and middleware are another, finer-grained one. Claude Code supports user-defined commands or scripts that run automatically on lifecycle events — at agent start, after a tool call, on stop, and so on ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)). LangChain's middleware concept is structurally similar. Some hooks are fully deterministic scripts; others are procedural checkpoints that inject context back into the model. The reliability comes from the harness executing them automatically, not from the model remembering a rule.

Common uses are notifications (sounds when an agent finishes), automated approvals or denials (deny migration commands; ask the user to run them instead), integrations (post a Slack message, open a PR), and verification (run typecheck and build on stop, surface errors to the agent so it has to fix them before finishing). HumanLayer's example hook runs Biome and TypeScript in parallel on every Claude stop, exits silently on success, and on failure surfaces only the errors with exit code 2, telling the harness to re-engage the agent.

LangChain reports this kind of middleware was central to lifting their deepagents-cli from Top 30 to Top 5 on Terminal-Bench 2.0. Their `PreCompletionChecklistMiddleware` intercepts the agent before exit and reminds it to run a verification pass against the task spec; a `LocalContextMiddleware` runs at start to map the working directory and discover available tools; a `LoopDetectionMiddleware` tracks per-file edit counts and prompts the agent to reconsider after N edits to the same file, breaking "doom loops" of small variations on a broken approach ([LangChain — Improving Deep Agents with Harness Engineering](#)).

The survey's governance taxonomy makes those hooks part of a wider enforcement pipeline: pre-invocation checks can deny unsafe tool calls, post-invocation hooks can taint or redact untrusted outputs before they enter context, stop hooks can require verification or audit updates, and escalation hooks can route ambiguous cases to humans. The more consequential the action, the less it should depend on the model remembering an instruction.

5.7 Feedforward and Feedback: A Cybernetic View

Thoughtworks' Birgitta Böckeler offers a higher-level taxonomy ([Thoughtworks — Harness Engineering](#)). Outer-harness controls fall into two directions:

- **Guides (feedforward)** anticipate the agent's behavior and steer it before it acts. They increase the probability of good output on the first attempt — instructions in AGENTS.md, skills, reference documentation, language-server hints.
- **Sensors (feedback)** observe after the agent acts and help it self-correct. Tests, linters, type checkers, AI code review.

A harness with only feedforward guides keeps issuing rules but never learns whether they work; a harness with only feedback sensors keeps catching the same mistake without preventing recurrence. Both are needed.

Within each direction there is a second axis:

- **Computational** controls — linters, type checkers, structural tests — are deterministic, run in milliseconds to seconds, and produce reliable results.
- **Inferential** controls — semantic analysis, AI code review, LLM-as-judge — handle nuance but are slower, more expensive, and non-deterministic.

The two axes are independent. Coding conventions in AGENTS.md are inferential feedforward. ArchUnit tests checking module boundaries on commit are computational feedback. A `/code-review` skill is inferential feedback. A pre-bootstrap script that sets up the project structure is computational feedforward. A well-engineered harness mixes all four.

5.8 Three Regulation Categories

Böckeler further distinguishes harnesses by what they regulate ([Thoughtworks — Harness Engineering](#)):

- **Maintainability harness** — internal code quality, duplication, complexity, coverage, style. The easiest category, with a long history of pre-existing tooling.
- **Architecture fitness harness** — performance, observability, debuggability. Captures cross-cutting "fitness functions" of the application.
- **Behavior harness** — does the application functionally behave the way it should? This is the unsolved category. Today, most teams rely on functional specs as feedforward and AI-generated test suites as feedback, occasionally augmented with mutation testing — and Böckeler is candid that trusting AI-generated tests "is not good enough yet."

The point of these categories is to make it possible to assess harness coverage. A harness that is strong on maintainability but weak on behavior gives a false sense of safety.

5.9 Timing: Keep Quality Left

Continuous integration teaches that the earlier you find issues the cheaper they are to fix, and the same holds for harness design. Fast computational sensors (linters, fast tests) should run before commit; expensive computational and inferential sensors (mutation testing, broader code review) run post-integration in the pipeline; continuous-drift sensors (dead-code detection, dependency scanners, log-anomaly judges) run outside the change lifecycle altogether ([Thoughtworks — Harness Engineering](#)).

The OpenAI Codex team's harness, as Böckeler notes, follows the same shape: layered architecture enforced by custom linters and structural tests, plus recurring "garbage collection" passes that scan for drift and have agents suggest fixes.

5.10 Harnessability and Ambient Affordances

Not every codebase is equally amenable to harnessing. A strongly-typed language brings type-checking sensors for free; clear module boundaries afford architectural constraint rules; opinionated frameworks like Spring abstract away details the agent does not have to worry about ([Thoughtworks — Harness Engineering](#)).

A term Böckeler credits to Ned Letcher, *ambient affordances*, captures this: properties of the environment itself that make it legible, navigable, and tractable to agents ([Thoughtworks — Harness Engineering](#)). Greenfield teams can engineer affordances in from day one; legacy teams face the inverse — the harness is most needed where it is hardest to build.

Anticipating the future, Böckeler suggests *harness templates* — bundled guides and sensors per service topology (CRUD service in JVM, event processor in Go, dashboard in Node) — that ride along with existing service templates. Böckeler invokes Ashby's Law of Requisite Variety to make the case formally — a regulator must have at least as much variety as the system it governs — so committing to a constrained topology is itself a variety-reduction move that makes a comprehensive harness more achievable ([Thoughtworks — Harness Engineering](#)).

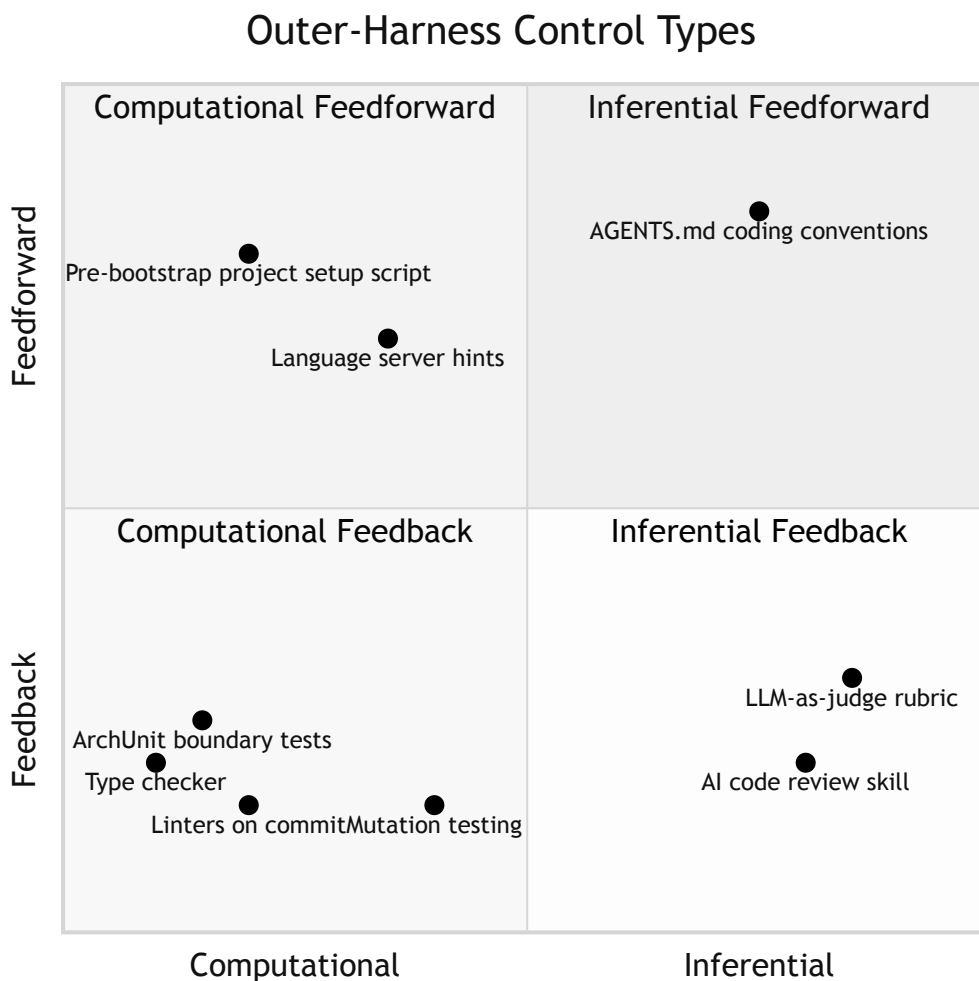
5.11 Operational Safety: Circuit Breakers, Kill Switches, Budgets, and Canaries

Sandboxes, governance, and hooks (§5.3–5.6) bound what an agent *may* do. A second family of controls bounds what happens when the agent or its tools *misbehave at runtime*. These are borrowed almost unchanged from distributed-systems reliability and security operations, and they belong in the harness because a steered or looping agent cannot be trusted to apply them to itself.

- **Circuit breakers.** Wrap a flaky or expensive dependency — a tool, a downstream service, a sub-agent — so that after a threshold of failures the breaker trips and further calls fail fast instead of hanging or retrying into a storm (Fowler — [CircuitBreaker](#)). For agents this bounds the blast radius of a tool that starts erroring or an agent stuck retrying the same failing action — the counterpart to "keep the wrong stuff in" (Ch 3): that pattern keeps one failure visible, while the breaker keeps failures from recurring without limit.
- **Kill switches.** A human- or policy-triggered stop that halts an agent, or an entire fleet, immediately and independently of the agent's own control flow. Because a prompt-injected agent may be actively working against its instructions, the switch has to live in the harness — a supervisor process, a revocable credential, a sandbox teardown — not in a prompt that says "stop if asked."
- **Action budgets, iteration caps, and cost governors.** Hard limits on tool calls, tokens, wall-clock time, or spend, after which the loop stops and escalates rather than running away. This is the operational form of the loop stop rules of Chapter 8 and the per-task budgets of Chapter 17: an unbounded loop is both a runaway bill and a runaway blast radius.
- **Canary tokens.** Fake secrets — an unused API key, a decoy file, a tripwire URL — planted where a prompt-injected agent would try to read or exfiltrate them. A callback on a canary is a high-signal alarm that the agent has been steered into touching data it should not ([Thinkst — Canarytokens](#)). Unlike the sandbox, a canary does not *prevent* the exfiltration leg of the lethal trifecta (§5.1); it *detects* it, which is what makes it a useful last line when prevention is imperfect.

The framing matches the rest of the chapter: the more consequential the failure, the less it should depend on the model choosing to avoid it. Prevention (sandboxes, policy) and detection (canaries, drift alerts in Ch 17) compose; neither alone is sufficient.

Diagram: Feedforward/Feedback × Computational/Inferential Quadrant



Key Takeaways

- **The threat model comes first:** prompt injection, data exfiltration, destructive action, and supply-chain risk — the *lethal trifecta* of private data + untrusted content + external communication is the core danger every control targets.
- **Sandboxing reduces permission prompts by 84%** while maintaining safety — structural boundaries beat approval dialogs.
- **Sandboxing has three jobs:** security, reproducibility, and liveness.
- **Filesystem and network isolation must be paired:** each addresses a different attack vector, and either alone is insufficient.
- **Governance is more than approvals:** identity, scoped credentials, policy checks, provenance, and audit trails must compose across tools and sessions.
- **Hooks and middleware are programmatic enforcement:** they run regardless of model memory, making them more reliable than prompt-only constraints.
- **Feedforward and feedback are both required:** guides without sensors have no learning loop; sensors without guides react but don't prevent.
- **Three categories of harness coverage:** maintainability (well-tooled), architecture fitness (achievable), and behavior (the unsolved problem).
- **Ambient affordances matter:** strongly-typed languages and opinionated frameworks make harnessing easier from day one.
- **Runtime safety needs operational controls too:** circuit breakers, kill switches, action/cost budgets, and canary tokens bound misbehavior when it happens — they live in the harness because a steered agent cannot be trusted to stop itself.

Further Reading

- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
- Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>

Chapter 6: Agentic Workflow Patterns

6.1 Workflows vs. Agents

Anthropic distinguishes two architectures within the broader category of "agentic systems" ([Anthropic — Building Effective Agents](#)):

- **Workflows** orchestrate LLMs and tools through predefined code paths.
- **Agents** dynamically direct their own processes and tool usage.

The first principle they push is to find the simplest solution and only add complexity when needed. Many use cases do not need agents at all — single LLM calls with retrieval and in-context examples are usually enough. Workflows give predictability and consistency for well-defined tasks; agents are right when flexibility and model-driven decision-making are needed at scale.

6.2 The Augmented LLM

The basic building block is the *augmented LLM*: a model with retrieval, tools, and memory. Modern models can actively use these — generating their own queries, selecting tools, deciding what to retain ([Anthropic — Building Effective Agents](#)). MCP (see ch4) is one increasingly common way to expose these augmentations.

6.3 Compositional Workflow Patterns

From simplest to most flexible:

Prompt chaining decomposes a task into sequential steps, each LLM call processing the previous output, with optional programmatic gates. Use it when a task can be cleanly decomposed and you want to trade latency for accuracy by making each call simpler. Example: write a marketing copy, then translate it. Failure mode: latency compounds across steps, and an error in an early call propagates downstream — gate aggressively.

Routing classifies an input and dispatches it to a specialized follow-up. Use when distinct categories benefit from separate handling and classification can be done reliably. Example: route customer service queries to refund-handling, technical-support, or general-question pipelines. Failure mode: it breaks silently on misclassification and ambiguous inputs — instrument the classifier and watch its error rate.

Parallelization runs LLM calls simultaneously and aggregates outputs. Two variants — *sectioning* breaks into independent subtasks, *voting* runs the same task multiple times. Use for speed or when multiple perspectives improve confidence (vulnerability review across multiple prompts, content-moderation with multiple votes). Failure mode: voting multiplies token cost N-fold, and correlated errors can make N agreeing votes falsely reassuring.

Orchestrator-workers has a central LLM that dynamically breaks down tasks, delegates to worker LLMs, and synthesizes results. Differs from parallelization because subtasks are not pre-defined. Use for complex tasks where the subtask shape depends on input — coding agents touching many files, research over many sources. Failure mode: the orchestrator can fan out unboundedly — cap the worker count and the total token budget.

Evaluator-optimizer has one LLM generating, another critiquing, in a loop. Use when there are clear evaluation criteria and iterative refinement provides measurable value. The two indicators: human feedback measurably improves output, and an LLM can plausibly produce that feedback. Examples: literary translation with critic, multi-round research with relevance evaluator. Failure mode: the loop may not converge — bound the iteration count, since each round adds latency and cost.

6.4 Three Principles for Agent Implementation

Anthropic ends with three rules ([Anthropic — Building Effective Agents](#)):

1. **Maintain simplicity** in agent design.
2. **Prioritize transparency** by explicitly showing the agent's planning steps.
3. **Carefully craft the agent-computer interface** through tool documentation and testing.

Frameworks help you start fast but can introduce abstraction layers that obscure the underlying prompts and tool calls. Anthropic recommends starting with direct API calls when you are still learning the shape of the problem, then reaching for a framework when the repeated patterns and operational needs justify it.

6.5 The Micro-Agent Pattern

HumanLayer's pragmatic version of the same insight ([HumanLayer — 12-Factor Agents](#)): the "loop until done" pattern hits a wall around 10–20 turns, after which agents lose coherence as accumulated context and per-turn error compound (see ch2, Context as a Finite Resource). What works is sprinkling small, focused agents into a broader deterministic DAG (directed acyclic graph of code-defined steps). Embedding LLM micro-agents in deterministic code this way is the harness shape LangChain describes too ([LangChain — The Anatomy of an Agent Harness](#)). Their deploybot example has deterministic code handling staging deployment, e2e tests, and the actual prod deploy commands; the LLM only intervenes to interpret human plaintext feedback ("can you deploy the backend first?") and propose updated steps. By keeping the agent's domain to 5–10 steps, error spin-outs become rare.

The principle generalizes: when the model gets smarter, agents may grow to handle more steps, but the small-focused-agent approach lets you ship results today and expand scope incrementally as model capabilities allow.

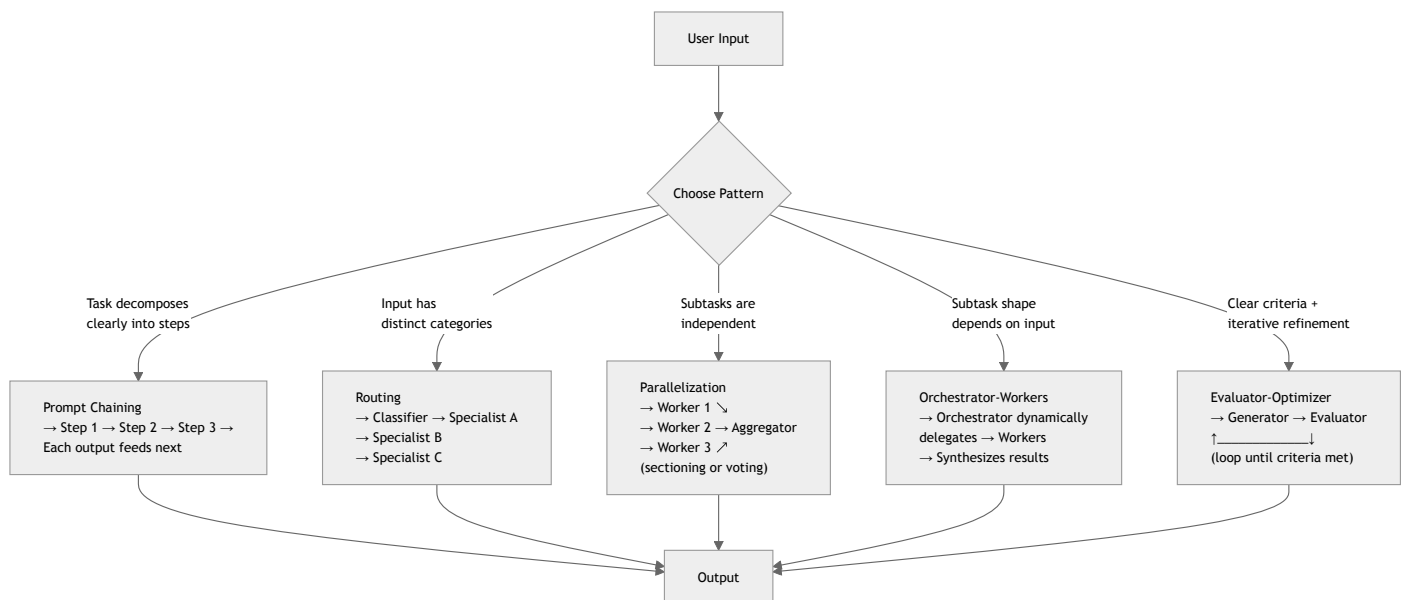
6.6 Reasoning and Self-Correction Patterns

The five patterns of §6.3 are compositional *control-flow* patterns — they orchestrate calls. A parallel line of research contributes *reasoning* patterns: ways a single agent, or a tight loop, structures its own deliberation and self-correction. They compose with the workflow patterns rather than replacing them, and most are named research artifacts worth recognizing.

- **ReAct** interleaves reasoning and acting — the model thinks, calls a tool, observes, and repeats. It is the base pattern under most agent loops (Ch 1; *LLM Foundations* ch 12).
- **Reflexion** adds a memory of self-critique: after a failed attempt, the agent writes a natural-language reflection on *why* it failed and retries with that reflection in context — "verbal reinforcement learning" without any weight update ([Shinn et al. — Reflexion](#)).
- **Self-Refine** collapses the evaluator-optimizer pattern (§6.3) into one model: it generates, critiques its own output, and revises, iterating until satisfied ([Madaan et al. — Self-Refine](#)).
- **CRITIC** grounds that critique in *external tools* — search, code execution, calculators — rather than introspection alone, so the correction is checked against the world instead of the model's own confidence ([Gou et al. — CRITIC](#)).
- **Tree of Thoughts (ToT)** replaces the single reasoning chain with a search over branches, using lookahead and backtracking to explore multiple partial solutions before committing ([Yao et al. — Tree of Thoughts](#)).
- **LATS** unifies reasoning, acting, and planning by running Monte Carlo Tree Search over agent trajectories, with an LM value function and reflection guiding the search ([Zhou et al. — Language Agent Tree Search](#)).
- **ReWOO** decouples planning from execution: a *Planner* writes the full plan up front, *Workers* execute the tool calls, and a *Solver* composes the answer — cutting tokens by not re-reasoning after every observation ([Xu et al. — ReWOO](#)).

The harness lens ties them together. Each mostly trades tokens and latency for reliability, and — as §6.1 and Chapter 14 warn — that trade pays off on hard, checkable tasks and is pure overhead on easy ones. The more trustworthy patterns are the ones whose self-correction is *grounded* in tools or tests (CRITIC here; the verifier-gated cascades of Ch 14; the generator–evaluator split of Ch 7) rather than resting on the model critiquing itself, matching the book's recurring theme that verification beats introspection (Ch 7, Ch 10).

Diagram: The Five Workflow Patterns



Key Takeaways

- **Start with the simplest pattern:** many tasks need only a single LLM call; adding agent loops is often premature.
- **Workflows give predictability; agents give flexibility:** choose based on whether the subtask structure is known ahead of time.
- **Five patterns cover most cases:** chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer.
- **The micro-agent approach scales well today:** 5–10 step focused agents embedded in a deterministic DAG outperform "loop until done" for most tasks.
- **Preserve visibility before abstraction:** direct API calls make early behavior easier to inspect; frameworks pay off once patterns stabilize.
- **Reasoning patterns compose with workflow patterns:** Reflexion, Self-Refine, CRITIC, Tree of Thoughts, LATS, and ReWOO structure a model's own deliberation — most valuable on hard, checkable tasks, and most trustworthy when the self-correction is grounded in tools or tests rather than introspection.

Further Reading

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
- Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
- Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
- Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
- Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
- Bin Feng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>

Chapter 7: Long-Running Agents and Multi-Context-Window Tasks

7.1 The Shift-Change Problem

Anthropic's "Effective Harnesses for Long-Running Agents" frames the core challenge with a metaphor: imagine a software project staffed by engineers working in shifts, each one arriving with no memory of what happened before ([Anthropic — Effective Harnesses for Long-Running Agents](#)). Because context windows are limited and most projects exceed one window, agents need a way to bridge sessions.

Compaction alone is not always enough. In Anthropic's long-running app experiments, even with the Claude Agent SDK's automatic compaction, a simple Opus 4.5 loop was not reliable for building a production-quality app from a high-level prompt like "build a clone of claude.ai." The failures clustered in two patterns: the agent tried to one-shot the app and ran out of context mid-implementation, leaving the next session to clean up; or, after some features were built, a later agent declared the job done.

7.2 The Initializer + Coding Agent Pattern

Anthropic's solution decomposes the work into two roles ([Anthropic — Effective Harnesses for Long-Running Agents](#)):

The **initializer agent** runs once with a specialized prompt. It produces:

- An `init.sh` script that runs the development server.
- A `claude-progress.txt` log to be updated each session.
- An initial git commit.
- A comprehensive feature-list file (in JSON; Markdown was tried first but the model was more likely to inappropriately edit it). For the claude.ai clone the list ran to over 200 features, each marked `passes: false` initially.

Each entry in the feature list is a JSON object with category, description, steps to verify, and a `passes` boolean. The coding agent is allowed to flip `passes` but is told strongly that removing or editing features is unacceptable.

The **coding agent** runs every subsequent session with a different prompt that asks it to make incremental progress. Each session begins with a structured warm-up:

1. Run `pwd` to confirm the directory.
2. Read git logs and the progress file to see what was last worked on.
3. Read the feature-list file and pick the highest-priority unfinished feature.
4. Run `init.sh` to start the dev server, then run a basic end-to-end test before implementing anything new.
5. Implement one feature.
6. Verify end-to-end (Anthropic uses the Puppeteer MCP for browser-driven verification, since the agent is otherwise prone to declaring features done after passing unit tests but failing in practice).
7. Commit with a descriptive message and update the progress file.

The pattern's benefit is that the agent is forced into a clean state at session boundaries — the kind of state appropriate for merging to a main branch — which means the next agent can start work without first cleaning up the previous one's mess.

7.3 Session Lifecycle and Clean Exit

Long-running harnesses need an explicit session lifecycle: start, warm up, choose bounded work, verify, record evidence, and exit cleanly. The last step is load-bearing. If a session ends with failing tests, stale temporary files, an unupdated feature list, or a vague progress note, the next agent spends its first turns reconstructing the previous state instead of advancing the task.

A clean exit should therefore be part of the definition of done:

- The standard startup path still works.
- The relevant build, lint, test, or end-to-end checks have been run, and failures are either fixed or recorded as blockers.
- The progress artifact says what changed, what was verified, what remains uncertain, and the best next action.
- The feature list or task list reflects reality: no item is marked passing without evidence.
- Temporary debugging artifacts, commented-out experiments, and obsolete notes are removed or explicitly quarantined.

OpenAI describes a related maintenance loop in its Codex harness work: agent-generated systems tend to copy whatever patterns already exist in the repository, so architectural rules and "golden principles" need to be written into docs, linters, and recurring cleanup passes rather than left as occasional human taste ([OpenAI — Harness Engineering](#)). Anthropic's initializer/coding-agent pattern reaches the same operational conclusion from the session side: the next session should be able to resume from repository artifacts, not from the previous agent's private memory.

For larger projects, a lightweight quality document is useful alongside the progress log. The progress log answers "what happened in the last session?" A quality document answers "which modules are healthy, risky, hard for agents to understand, or missing verification?" That distinction matters because the next agent should not only know the next feature; it should also know where the codebase is deteriorating.

7.4 Generator-Evaluator (GAN-Inspired)

A follow-up by Prithvi Rajasekaran extends the pattern to a harder problem: building production-quality apps from short prompts ([Anthropic — Harness Design for Long-Running Application Development](#)). The motivating observation: when asked to evaluate their own work, agents reliably skew positive even when output is mediocre. Separating the agent doing the work from the agent judging it is a strong lever, because tuning a separate skeptical evaluator is more tractable than making a generator critical of its own output.

Inspired by Generative Adversarial Networks, the architecture is three agents:

- **Planner** expands a 1–4 sentence prompt into a full product spec, deliberately staying at the product/architecture level rather than detailed technical design (so errors do not cascade), and is encouraged to weave AI features into the spec.
- **Generator** implements the spec one feature at a time using a React + Vite + FastAPI + SQLite stack, with git for version control.
- **Evaluator** uses the Playwright MCP to click through the running application as a user would, testing UI, API endpoints, and database state, then grades against a rubric covering product depth, functionality, visual design, and code quality. Each criterion has a hard threshold, and a single failure causes the sprint to fail with detailed feedback.

The two coordinate via *sprint contracts*: before each sprint, the generator proposes what it will build and how success will be verified; the evaluator reviews until they agree; the generator then builds against the agreed contract.

Communication is file-based — one agent writes a file, another reads and responds.

The cost is significant. For a "create a 2D retro game maker" prompt, the solo run took 20 minutes and cost \$9, producing an app that looked plausible but where the game itself did not work — entities appeared on screen but nothing responded to input. The full harness took 6 hours and cost \$200, producing a working game maker with sprite editor, level editor, AI-assisted level generation, and playable mode. The more-than-20× cost premium bought a working application instead of broken stubs.

7.5 Self-Verification Is the Headline Lever

LangChain's Top-30-to-Top-5 case study reaches the same conclusion via a different route ([LangChain — Improving Deep Agents with Harness Engineering](#)). With trace analysis, they identified the most common failure pattern: the agent wrote a solution, re-read its own code, decided it looked fine, and stopped. They added structured guidance to the system prompt — Plan, Build with verification in mind, Verify by running tests and comparing output to spec, Fix — and a `PreCompletionChecklistMiddleware` that intercepts the agent before exit and forces a verification pass.

This pattern echoes the "Ralph Wiggum loop" that has spread through the developer community: a hook that intercepts the agent's exit attempt and reinjects the original prompt in a clean context window, forcing the agent to continue against its goal ([LangChain — The Anatomy of an Agent Harness](#)).

LangChain's combined changes — context middleware mapping the cwd and tooling, build-verify guidance, loop detection, and a "reasoning sandwich" of high-low-high reasoning compute — improved the score by 13.7 points (from 52.8% to 66.5%) with no model change.

7.6 Context Resets vs. Compaction

Anthropic's harness-design follow-up makes an explicit distinction ([Anthropic — Harness Design for Long-Running Application Development](#)). Compaction summarizes earlier parts of a conversation in place; the same agent continues with a shortened history. A *context reset* clears the context entirely and starts a fresh agent, with a structured handoff carrying the previous agent's state and next steps.

The two address different problems. Compaction preserves continuity. Resets cure "context anxiety" — a tendency Anthropic observed in Sonnet 4.5 where the agent began wrapping up work prematurely as it neared what it believed to be its context limit. Resets give the agent a clean slate; the cost is that the handoff artifact must carry enough state for the next agent to resume cleanly.

When Opus 4.5 largely fixed the context-anxiety behavior on its own, Anthropic was able to drop context resets from the harness entirely. This is an explicit example of the model-harness coupling discussed in chapter 12.

7.7 Managed Agents: Decoupling Brain, Hands, and Session State

The OpenReview survey highlights a platform direction that Anthropic later calls managed agents: separate the model-side **brain**, the execution-side **hands**, and the durable **session/event log** ([OpenReview — Agent Harness Engineering: A Survey](#)). The brain decides what should happen; the hands run shell commands, edit files, browse, and call external services inside a replaceable environment; the session log records enough state to rebuild either side.

This split matters for long-running work:

- If the model context is exhausted, a new brain can resume from the event log and repository artifacts.
- If the sandbox is corrupted, timed out, or compromised, the hands can be rebuilt from a clean image.
- If credentials are required, proxies and vaults can attach them at the boundary instead of placing secrets inside the sandbox.
- If a deployment changes while agents are running, the platform can keep old and new worker versions alive during a gradual handoff.

Seen this way, context reset is only one recovery mechanism. A production long-running harness also needs environment reset, credential isolation, resumable event logs, and migration rules for in-flight sessions.

7.8 Multi-Agent Research Systems

For tasks with parallel structure — research with many independent threads to explore — the orchestrator-worker pattern from chapter 6 applies. Anthropic's research feature uses Claude Opus 4 as the lead agent and Claude Sonnet 4 as sub-agents ([Anthropic — How We Built Our Multi-Agent Research System](#)). The lead analyzes the query, develops a strategy, and spawns parallel sub-agents that each search and return condensed findings; the lead synthesizes; a citation agent then attributes claims to sources.

Eight prompt-engineering principles surface from their experience:

1. **Think like the agent:** simulate prompts in the Console with the exact tools to see step-by-step behavior.
2. **Teach the orchestrator how to delegate:** give sub-agents an objective, output format, tool guidance, and clear task boundaries — vague delegation produces duplicate or misinterpreted work.
3. **Scale effort to query complexity:** explicit rules in the prompt (1 agent / 3–10 calls for fact-finding; 2–4 sub-agents / 10–15 calls each for comparisons; 10+ sub-agents for complex research) prevent over-investment.
4. **Tool design and selection are critical:** explicit heuristics ("examine all available tools first, match tool usage to user intent, prefer specialized tools over generic ones") prevent the agent from sending itself down wrong paths.
5. **Let agents improve themselves:** a tool-testing agent that uses a flawed MCP tool, observes the failure, and rewrites the description produced a 40% decrease in task completion time on subsequent uses.
6. **Start wide, then narrow:** prompt agents to begin with short broad queries and progressively refine — the natural tendency is the opposite.
7. **Guide the thinking process:** extended thinking serves as a controllable scratchpad for planning; interleaved thinking helps sub-agents evaluate quality and refine queries between tool calls.
8. **Parallel tool calling transforms speed:** spinning up sub-agents in parallel and having sub-agents call multiple tools in parallel cut research time by up to 90% on complex queries.

7.9 Production Reliability for Stateful Agents

Anthropic's research-system post documents engineering challenges that emerge once agents run for long periods ([Anthropic — How We Built Our Multi-Agent Research System](#)):

- **Errors compound:** minor system failures can be catastrophic without checkpoint-and-resume infrastructure. Anthropic combines AI adaptability (letting the agent know when a tool is failing and trusting it to adapt) with deterministic safeguards like retry logic and regular checkpoints.
- **Debugging needs new tooling:** because agents are non-deterministic between runs, full production tracing — observing decision patterns and interaction structures without reading conversation contents — is the primary

diagnosis surface.

- **Deployment needs coordination:** rolling out a code change while many agents are running requires *rainbow deployments* that gradually shift traffic from old to new versions while keeping both alive.
- **Synchronous execution creates bottlenecks:** in their current architecture, the lead agent waits for sub-agents to finish before proceeding, simplifying coordination but blocking the system on the slowest sub-agent. Asynchronous execution would unlock more parallelism but adds challenges in result coordination, state consistency, and error propagation.

7.10 Durable Execution: Checkpoint, Replay, and Recovery

Section 7.9 noted that errors compound, and that Anthropic pairs AI adaptability with deterministic checkpoints and retries. *Durable execution* is the systems discipline that generalizes this. A durable-execution engine persists each step of a workflow to a log so that if the process dies — a machine failure, a timeout, a deploy, an exhausted context window — it resumes from the last recorded step instead of restarting from zero. The idea predates agents; it is how workflow engines such as Temporal and DBOS provide fault tolerance ([Temporal — Durable Execution Meets AI](#)). It maps directly onto the managed-agent split of §7.7: the durable event log is precisely what lets a fresh brain resume after the old one is gone.

For an agent, the unit of durability is the agent's state — its context, its tool-call results, and its position in the plan. LangGraph exposes this as a *checkpoint* that saves graph state at each super-step, enabling resume-after-failure, human-in-the-loop pauses, and even time-travel to a prior state, with selectable durability modes trading performance against how much work a crash can lose ([LangChain — Durable Execution](#)).

The design tension is *determinism vs. the model*. Replay-based durability assumes a step can be re-executed to reproduce its effect, but model calls and tool results are non-deterministic — re-running them would diverge from the recorded history. The standard resolution is to treat model and tool calls as *side-effecting activities* whose results are recorded once and then replayed from the log rather than recomputed. This is the same "record the observation, don't recompute it" logic behind ReWOO (§6.6) and context resets (§7.6), now made an infrastructure guarantee.

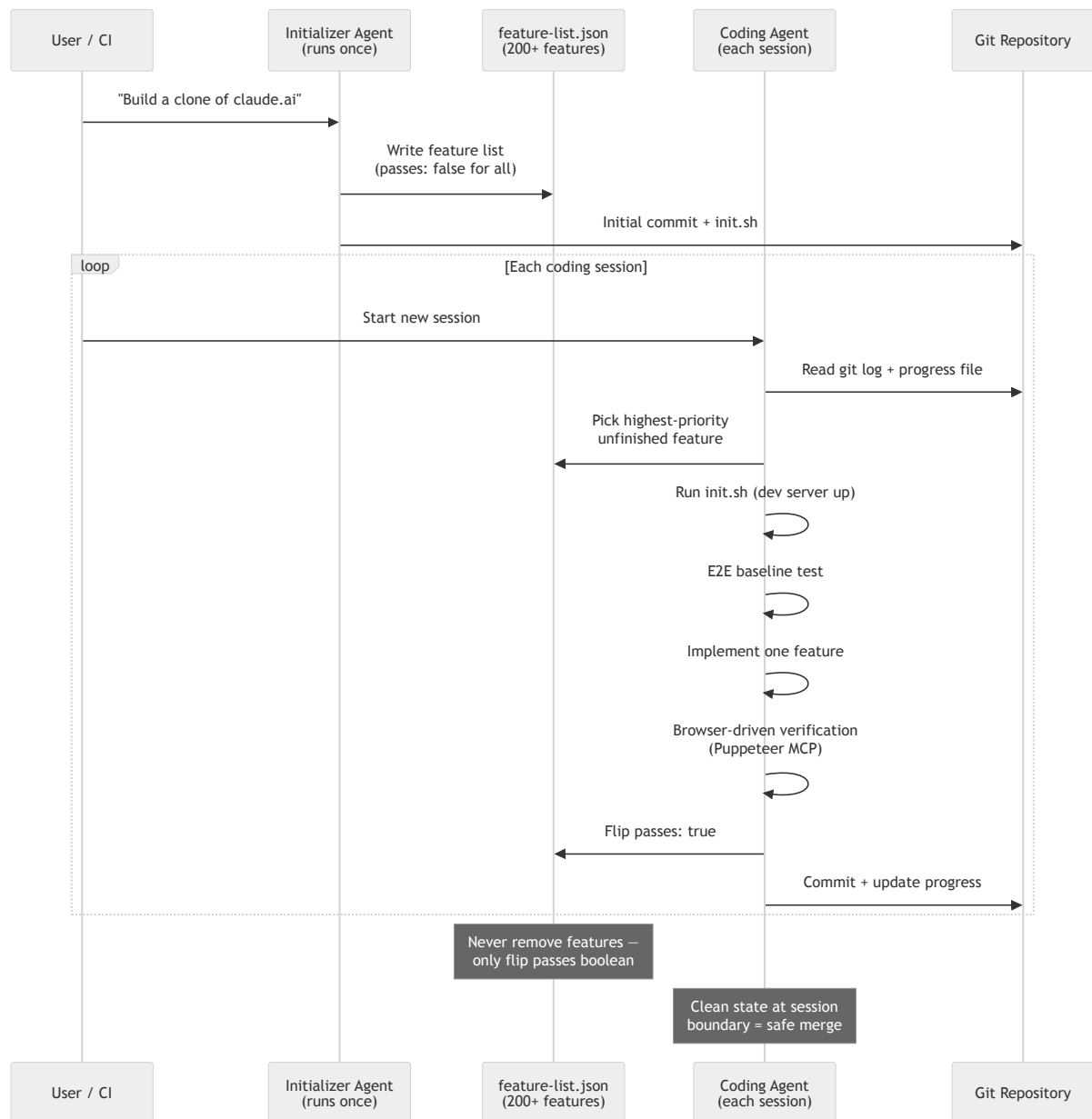
Seen this way, durable execution is what turns the "clean exit, resumable from artifacts" discipline of §7.3 from a convention the agent must remember into a property the platform enforces. It also bounds cost — a crash mid-task does not waste all the tokens spent before it — and it is the substrate for the rollback that Chapter 17 requires when a harness change misbehaves in production.

7.11 How Long Is "Long"? The Time-Horizon Metric

This chapter is about tasks that exceed a single context window, but "long" deserves a measure. METR proposes one: a model's *time horizon* is the length of task — measured by how long it takes a skilled human — that the model can complete with 50% reliability. A model with a "50-minute time horizon" succeeds, half the time, on tasks that take a human about fifty minutes. Measured across frontier models from 2019 to 2025, this horizon has roughly *doubled every seven months* ([Kwa et al. — Measuring AI Ability to Complete Long Tasks; METR](#)).

Two implications matter for this chapter. First, the horizon is a property of the *model plus its harness*, not the model alone: the handoff, checkpoint, and self-verification machinery here is how a harness stretches the effective horizon beyond what the raw model sustains solo — the model-harness coupling of Chapter 12, seen from the capability side. Second, the metric reframes when this machinery is worth building. As the intrinsic horizon grows, some scaffolding becomes unnecessary — Anthropic dropped context resets and, later, sprint decomposition as models improved (§7.6, Ch 12) — but the frontier of *interesting* long-horizon tasks moves out with it. The harness work relocates to harder problems rather than disappearing (Ch 18).

Diagram: Initializer Agent → Feature-List → Coding Agent Sessions



Key Takeaways

- **The shift-change problem is fundamental:** context limits mean agents need structured handoff mechanisms, not just bigger windows.
- **Initializer + coding agent is a useful long-horizon pattern:** separate roles for planning and incremental execution.
- **Clean exit is part of done:** each session should leave working startup paths, updated state artifacts, verification evidence, and no untracked mess for the next agent.
- **Separating generator from evaluator is a strong lever:** agents skew positive about their own output; an independent evaluator is more reliable.
- **Sprint contracts coordinate multi-agent work:** file-based communication with agreed success criteria before each build sprint.
- **Context resets cure "context anxiety":** sometimes a fresh start with a structured handoff outperforms compaction.
- **Managed agents decouple brain, hands, and state:** model context, sandbox execution, credentials, and event logs should fail and recover independently.
- **Self-verification is the headline lever:** forcing a verification pass before exit improved scores by 13.7 points with no model change.

- **Durable execution makes resumability an infrastructure guarantee:** persisting each step to a log lets a fresh agent resume after a crash, context exhaustion, or deploy — treat non-deterministic model/tool calls as recorded side effects, not steps to recompute.
- **The time horizon measures "how long":** METR's task-completion horizon (the human-task-length a model clears 50% of the time) has doubled roughly every seven months — and it is a model-plus-harness property, which is why this chapter's machinery extends it.

Further Reading

- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
- LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>

Chapter 8: Loop Engineering

Agent = Model + Harness. Chapter 1 introduced the agent loop as the cycle at the centre of every agent — assemble context, the model emits a tool call, the harness executes it, the observation is appended, repeat. The previous chapters engineered the *inside* of that loop: what the model sees, what tools it has, where it runs, how it recovers across sessions. This chapter is about engineering the loop *itself* as the primary unit of work — deciding what starts it, what it does each pass, who checks the result, and when it stops. In 2026 this practice acquired a name, *loop engineering*, and a slogan: stop prompting the agent, and build the system that prompts it.

8.1 From Prompting to Looping

Section 1.6 traced an arc — prompt engineering gave way to context engineering, which sits under harness engineering. Loop engineering is the next stop on the same road, and the one that made the shift concrete for practitioners. Addy Osmani named the pattern in a June 2026 essay, "Loop Engineering," which supplied the canonical anatomy and the vocabulary that now circulates ([Addy Osmani — Loop Engineering](#)). The same week, Peter Steinberger compressed it into a line that reached millions within a day: you should not be prompting coding agents anymore; you should be designing the loops that prompt them ([O'Reilly Radar — Loop Engineering](#)). Boris Cherny, who built Claude Code, put the practitioner's version bluntly: he no longer prompts the model turn by turn — his job is to write the loops that drive it ([The New Stack — Loop Engineering](#)).

The reframe is small but load-bearing. In prompt engineering the human is *in* the loop, pressing enter between every step and judging each result. Loop engineering removes the human from that inner position and asks a harder question: if you are not there to decide when the work is good enough and what to do next, *what is?* Everything in this chapter is an answer to that question. Nothing here is new relative to the mechanics of Chapter 1 — it is the same agent loop — but the emphasis moves from the model's turn to the surrounding control structure, which is squarely harness territory. Loop engineering and harness engineering are two names for closely related work; loop engineering is the operator's framing, focused on the *outer* loop that schedules, verifies, and bounds the agent over long horizons.

8.2 A Loop Is a Task With a Check

The field guide's one-sentence definition is the right anchor: a loop is a task with a check, and a task without a check is just hope ([The Agentic Loop — A Practical Field Guide](#)). Expanded, one pass of a well-formed loop is: observe the current state, take one bounded action, check the result against a fixed standard, and decide whether to continue or stop. That structure exposes four design levers, and loop engineering is largely the work of specifying them well:

- **The trigger** — what starts a pass: a human goal, a schedule, a webhook, or another agent.
- **The topology** — how loops nest and hand off: one agent, a maker plus a checker, or an orchestrator over workers.
- **The verifier** — the fixed standard that decides "good enough," and who applies it.
- **The stop rules** — the explicit conditions under which the loop succeeds, gives up, or asks for help.

Leave any one implicit and the failure is predictable. No trigger and the loop is just a chat. No verifier and it declares victory on garbage. No stop rule and it runs forever, or until the bill arrives. The rest of this chapter takes the four levers in turn.

8.3 The Trigger and the Nested Loops

A trigger is what promotes an agent from something you invoke to something that runs on its own. Osmani's anatomy calls it the *heartbeat* — a schedule or event that wakes the loop without a human prompt ([Addy Osmani — Loop Engineering](#)). This is where a coding agent becomes an operations concern rather than an editor: cron fires it nightly, a webhook fires it on a new issue, or a supervising agent fires it as a sub-task.

Andrew Ng gave the topology its clearest map by observing that these loops nest, each with a different owner and time horizon ([Andrew Ng — The Batch, June 2026](#)):

- The **agentic coding loop** runs in minutes: given a spec and evals, the agent writes code, tests it, and iterates until it meets the spec — no human between turns.
- The **developer feedback loop** runs in hours: a human inspects what was built and steers the agent toward what to do next.
- The **external feedback loop** runs in days: alpha testers, A/B tests, production signal.

The inner loop is the one loop engineering automates most aggressively; the outer loops are where human judgment stays irreplaceable, because the human holds a *context advantage* the agent lacks — knowledge of intent and of what

the product is really for. Ng's own example: a coding agent worked unattended for about an hour, checking its work in a browser several times, before coming back for direction. The design goal is to let each loop run as long as it productively can before it must hand up to the next.

8.4 The Verifier Is the Bottleneck

Of the four levers, the verifier is where loop engineering concentrates its effort, because it is what makes unattended operation safe. Chapter 7 established the headline result from the other direction: agents skew positive about their own work, so separating the agent that does the work from the agent that judges it is a strong lever. Loop engineering elevates that observation to a design law — the maker must not be the checker ([Loop Engineering Crash Course](#)). A separate reviewer agent, given the spec rather than the diff and prompted to be skeptical, catches what the generator rationalizes past. This is the generator–evaluator split of §7.4 and the evaluator–optimizer workflow of Chapter 6, promoted from a technique to the thing that determines whether the loop can be left alone.

The community slogan captures the leverage shift: writing the verifier is the new prompt engineering ([The Agentic Loop — A Practical Field Guide](#)). The scarce, valuable act is no longer phrasing the request; it is defining "done" precisely enough that a machine can check it. A useful distinction follows:

- A **closed loop** pins its acceptance criteria up front as hard, checkable passes — all tests green, the schema validates, the screenshot matches. It runs on a predictable budget and is safe to leave running.
- An **open loop** explores loosely toward a fuzzy goal. It needs an even stronger verifier, because without one it does not fail loudly — it succeeds at producing plausible garbage, confidently, hundreds of times.

The verifier should be as mechanical as the task allows: a test, a type check, a schema validation, a browser assertion, and only then an LLM-as-judge for what cannot be checked deterministically — the computational-before-inferential ordering of Chapter 5, and the grader taxonomy of Chapter 10. The strongest form uses a *fresh* model with no memory of how the work was produced, so it cannot inherit the maker's blind spots.

8.5 Stop Rules and the Three Hard Stops

A loop that can start itself must also be able to stop itself, and for reasons other than success. Every well-formed loop needs explicit stop conditions — success, no-op (nothing left to do), ask-for-approval, and blocked-or-exhausted — and three of these are non-negotiable hard stops that exist to bound a runaway ([The Agentic Loop — A Practical Field Guide](#)):

1. **A maximum iteration count** — a hard cap like "all tests green, or six rounds, whichever comes first."
2. **No-progress detection** — if N passes produce no measurable change, halt rather than spin.
3. **A budget ceiling** — a token or dollar limit past which the loop stops and asks.

This is the per-task budget of Chapter 17 seen from the loop's inside, and the reason it is load-bearing here is that the human is no longer present to notice the spin. The failure mode is concrete: a team at Uber capped agent spending at \$1,500/month after an unattended setup burned its annual AI budget in four months ([AI Builder Club — Loop Engineering Guide](#)). The ask-for-approval stop is the bridge to Chapter 15: modeling escalation as a tool call lets the loop suspend, hand a decision to a human, and resume from the event log when they respond — the same durable-approval pattern, now the loop's designated exit for anything consequential.

8.6 The Ralph Lineage

Loop engineering did not appear from nowhere; it is the current rung of a ladder the field has been climbing since 2022 ([The Agentic Loop — A Practical Field Guide](#)):

- **ReAct** (2022) established the basic reason–act–observe cycle that Chapter 1 calls the agent loop ([Yao et al. — ReAct](#)).
- **AutoGPT** (2023) made it goal-driven and autonomous — and exposed the failure the discipline now guards against: a loop with no verifier and no stop rule runs forever or wanders off.
- **The "Ralph Wiggum" loop** (2025) added the crucial fix seen in §7.5: reset to a clean context window each iteration, anchoring state to files on disk rather than to a swelling history, and reinject the goal so the agent keeps working against it.
- **Verifiable-completion commands** (2026), such as a `/goal` that gates exit on a separate validator model, made the stop rule a first-class, machine-checked step rather than the agent's own opinion.
- **Orchestration** (current) has loops supervising loops — scheduled, git-backed, and handing work up and down Ng's nested horizons.

Two through-lines connect the lineage to the rest of the book. The Ralph reset is why *memory lives on disk, not in context* (Chapters 2–3): a loop that resets each pass must reload its state from files, which is exactly the structured note-taking and recitation those chapters prescribe. And verifiable completion is why *the event log matters* (Chapter 9): a loop whose state is an append-only log can stop, resume, and be replayed, which is what makes long-horizon autonomy debuggable at all.

8.7 What a Loop Is Made Of

Osmani's anatomy lists the parts a durable loop assembles, and each maps onto a capability the earlier chapters already built ([Addy Osmani — Loop Engineering](#)):

- **Heartbeat** — the schedule or event that triggers a pass (§8.3).
- **Worktrees** — isolated working directories so parallel passes do not collide, drawing on the sandbox isolation of Chapter 5.
- **Skills** — reusable, file-backed project knowledge written once and loaded on demand, the `SKILL.md` pattern of Chapter 4.
- **Connectors** — MCP servers and plugins that reach the real tools the work depends on (Chapter 4).
- **Sub-agents** — the maker and the checker as separate roles (§8.4, Chapter 3).
- **The spine** — a persistent state file that survives between runs and carries the loop's memory across resets (Chapters 2–3, and the structured handoff of §7.6).

The loop is only as good as the codebase it runs against, which is why practitioners describe three properties a repository needs to be loop-ready ([AI Builder Club — Loop Engineering Guide](#)). It must be **legible** — a compact `AGENTS.md` index and custom lints so the agent knows the shape of the code and what not to touch. It must be **executable** — a dev server that comes up at near-zero token cost and tolerates parallel worktrees. And it must be **verifiable** — end-to-end tests and browser-driven checks on the core flows, so the verifier of §8.4 has something mechanical to assert against. These are the *ambient affordances* of Chapter 5, now prerequisites for autonomy rather than niceties.

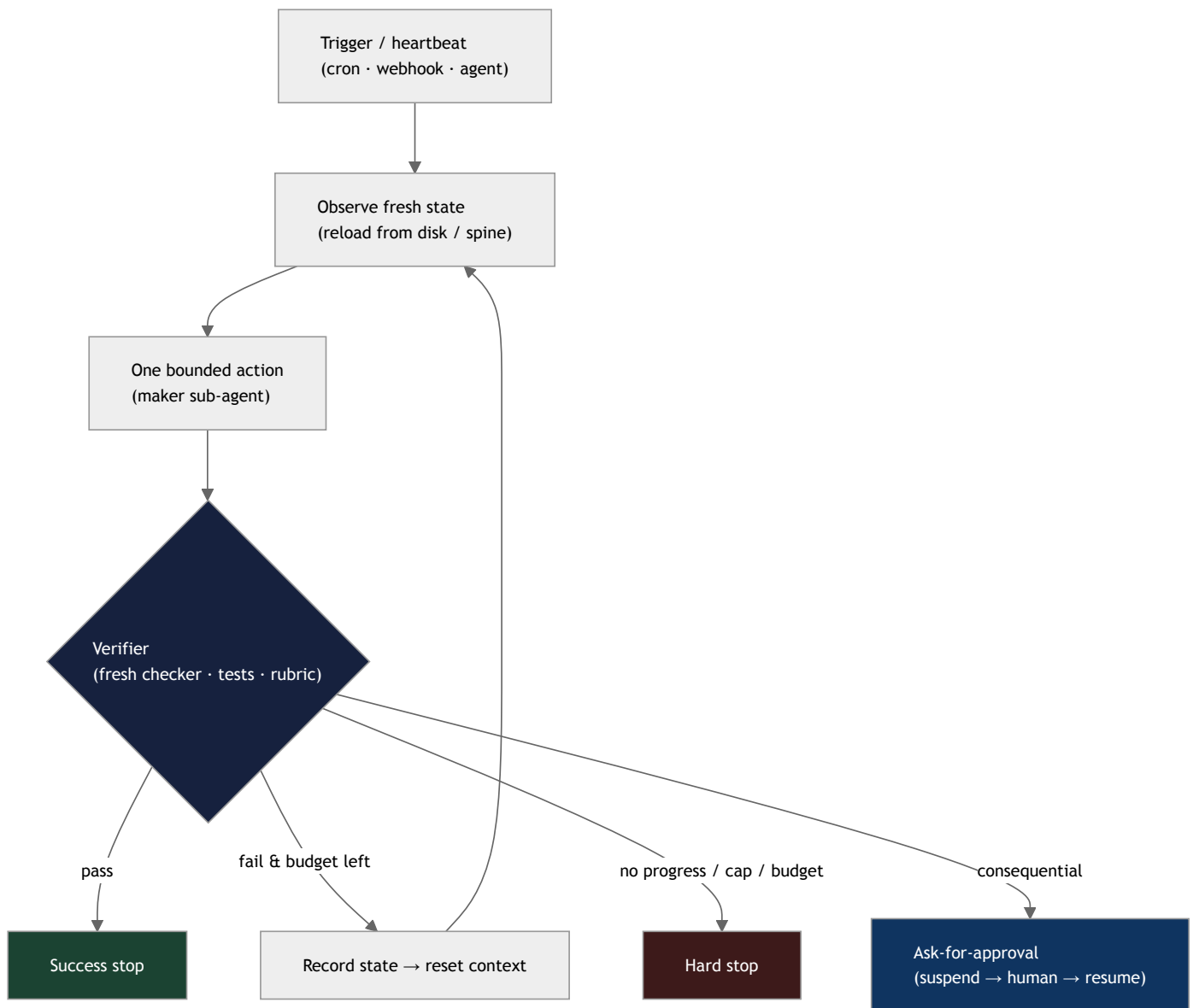
8.8 The Maturity Ladder and Unattended Risk

Because a loop compounds whatever it does, adoption should be staged. The community's maturity ladder climbs one rung at a time, and only when the current rung already produces work you would have done by hand ([The Agentic Loop — A Practical Field Guide](#)):

0. **Manual** — you prompt every turn.
1. **Triage** — the loop reports findings to a markdown file, changing nothing.
2. **Draft** — it makes fixes on an isolated branch.
3. **Verified PR** — a separate verifier gates the change before a human reviews it.
4. **Auto-merge** — reserved for low-risk classes only.

The discipline exists because loop engineering does not remove the hard problem; it relocates it. Speed compounds both wins and mistakes: a loop running unattended is also a loop making mistakes unattended, and it can ship code faster than a human reads it, accumulating *comprehension debt* — a codebase whose owners no longer fully understand it ([The New Stack — Loop Engineering](#)). Verification and accountability remain human, at two irreducible endpoints: the *intent* that specifies what "good" means, and the *ownership* of what ships ([Loop Engineering Crash Course](#)). The corollary is a scoping rule: reach for a structured loop only for work that is repeated, unattended, scheduled, and consequential. For anything you are watching interactively, the check is your own eyes, and a plain conversation is the better tool — over-engineering a loop for a one-off is its own failure mode.

Diagram: The Engineered Loop



Key Takeaways

- **Loop engineering is the outer-loop framing of harness engineering:** stop prompting the agent turn by turn; design the system that prompts it, and specify what decides when the work is good enough.
- **A loop is a task with a check:** observe → one bounded action → verify against a fixed standard → decide continue or stop. A task without a check is just hope.
- **Four levers define a loop:** the trigger, the topology, the verifier, and the stop rules. Leave any implicit and the failure is predictable.
- **Loops nest:** agentic coding in minutes, developer feedback in hours, external feedback in days — automate the inner loop, keep human judgment on the outer ones.
- **The verifier is the bottleneck:** writing it is the new prompt engineering, the maker must not be the checker, and a weak check fails silently by shipping confident garbage.
- **Stop rules are mandatory:** a max iteration count, no-progress detection, and a budget ceiling — because no human is present to notice a runaway.
- **The Ralph lineage is the ancestry:** ReAct → AutoGPT → clean-context reset → verifiable completion → orchestration; memory lives on disk and state lives in an event log.
- **Climb the maturity ladder slowly:** triage before draft before auto-merge, and only for work that is repeated, unattended, and consequential — unattended speed compounds mistakes and comprehension debt.

Further Reading

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026. <https://addyosmani.com/blog/loop-engineering/>
- *Loop Engineering*, O'Reilly Radar, 2026. <https://www.oreilly.com/radar/loop-engineering/>
- Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026. <https://www.deeplearning.ai/the-batch/>
- *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026. <https://thenewstack.io/loop-engineering/>
- *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026. <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
- *Loop Engineering Guide (2026)*, AI Builder Club. <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
- Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022. <https://arxiv.org/abs/2210.03629>

Chapter 9: Twelve Factors for Production Agents

The previous chapters described individual harness techniques: context management, tools, sandboxing, workflows, and long-running handoffs. This chapter steps back: HumanLayer's "12 Factor Agents" is best read as a production checklist that ties those techniques back to ordinary software architecture. It borrows the naming style of the classic Twelve-Factor App, but the factors are specific to LLM agents — and it is a manifesto, not a complete reference architecture.

Two software concepts do a lot of work in this chapter. *State* is the information needed to continue execution: the current step, retry counts, approvals, user messages, tool results, and business objects touched so far. An *event log* is the append-only record from which that state can be reconstructed. When an agent is modeled as a reducer over events, pause/resume, replay, debugging, and testing become ordinary software problems rather than hidden conversation state.

9.1 The Twelve Factors as Software Architecture

The twelve principles, drawn from many production deployments ([HumanLayer — 12-Factor Agents](#)):

1. **Natural Language to Tool Calls:** the atomic pattern is converting user phrasing into a structured JSON call that deterministic code executes.
2. **Own Your Prompts:** do not outsource prompt engineering to a framework's black box. Write prompts as first-class code so they can be tested, evaluated, and tuned.
3. **Own Your Context Window:** standard message-format is one option; custom XML-tagged event logs that pack history into a single user message are another. The aim is maximum information density with minimum tokens.
4. **Tools Are Just Structured Outputs:** a tool call is a model emitting JSON that names an intent and parameters. Deterministic code decides what to do with it.
5. **Unify Execution State and Business State:** don't separate "current step / next step / retry count" from "what happened in the conversation." Infer execution state from a single event log.
6. **Launch / Pause / Resume with Simple APIs:** agents are programs; they should support standard lifecycle operations, including pause-between-tool-selection-and-execution.
7. **Contact Humans with Tool Calls:** rather than relying on the model's choice of plain-text vs. structured output, give it explicit `request_human_input` tools with structured options (urgency, format, choices).
8. **Own Your Control Flow:** hijack the loop to break for approval, summarize tool results, run LLM-as-judge over outputs, manage memory, log and trace, rate-limit, or sleep durably.
9. **Compact Errors into Context Window:** leaving errors visible enables self-healing; with a counter on consecutive errors, escalate to a human after a threshold. The verb is *compact* because accumulating raw stack traces blows the token budget and feeds context rot (see Foundations ch 9) — keep the latest or a summarized error in context, and drop or fold older repeated traces so the error history does not crowd out the working context.
10. **Small, Focused Agents:** keep individual agent scope to 3–10, maybe 20 steps. Larger context = worse performance.
11. **Trigger from Anywhere:** enable launches from Slack, email, SMS, webhooks, crons. Combined with factor 7, this enables the *outer loop* — agents kicked off by events that contact humans for help when they reach critical points.
12. **Make Your Agent a Stateless Reducer:** a fold over events. Pure, serializable, replay-able. Replay is only deterministic if every LLM response and tool result is captured as an event: on resume you fold over the recorded results, you do not re-invoke the model or re-run side-effecting tools.

The deeper claim binding these together is that "agents, at least the good ones, don't follow the 'here's your prompt, here's a bag of tools, loop until you hit the goal' pattern. Rather, they are comprised of mostly just software" ([HumanLayer — 12-Factor Agents](#)). The factors are mostly software-engineering hygiene applied to a stateful, non-deterministic component. They should not be treated as universal laws: a research prototype, a local coding assistant, and a regulated customer-support agent will need different trade-offs. The useful lesson is the direction of travel — make state explicit, make control flow inspectable, and put human interaction behind structured interfaces.

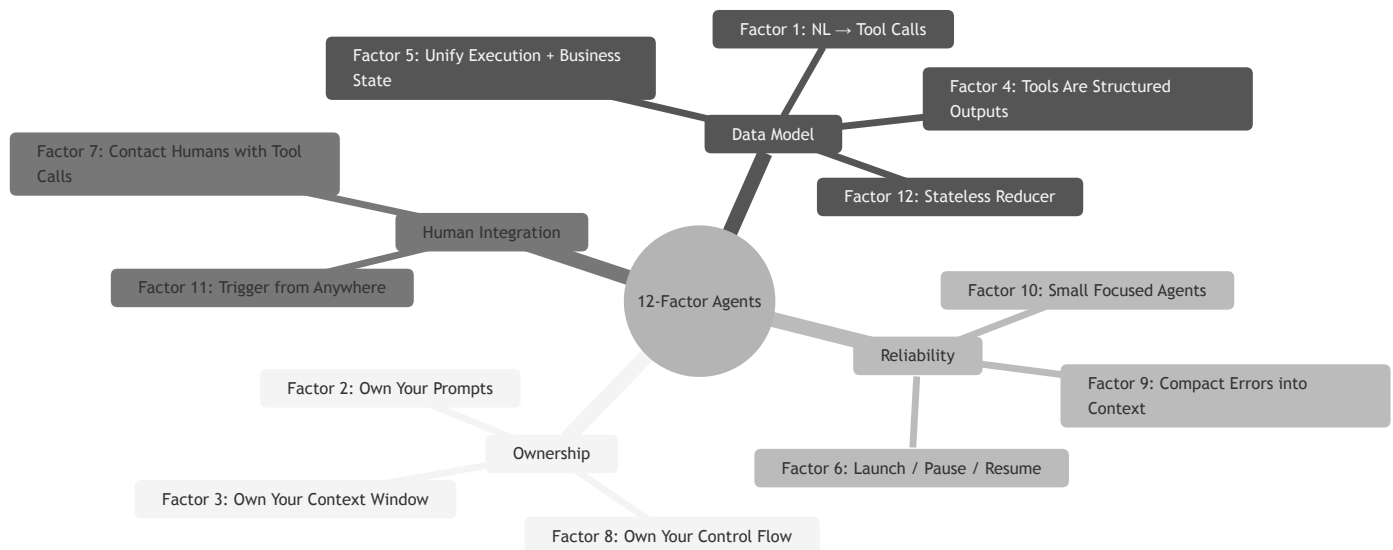
9.2 From Agent Programs to Agent Platforms

The OpenReview survey describes the ecosystem moving from agent frameworks toward agent platforms ([OpenReview — Agent Harness Engineering: A Survey](#)). A framework packages local abstractions — agents, tools, memory stores, and loops. A platform adds durable workspaces, managed sandboxes, identity, billing, observability, evaluation, governance, and human handoff across many runs and users.

This does not replace the twelve factors; it raises their scope. "Launch / pause / resume" becomes a platform API. "Unify execution state and business state" becomes event-log storage with tenancy and migration semantics. "Contact humans with tool calls" becomes a handoff interface with audit and permission state. "Own your control flow" becomes explicit decisions about which checks run synchronously, which run offline, and which failures justify costly recovery.

The platform boundary also changes responsibility. A local agent can get away with ad hoc state files; a shared platform needs state ownership, retention policy, billing attribution, credential scoping, and replayable audit trails. The harness is no longer just the thing around one model call. It becomes the control system around many agents, many environments, and many human stakeholders.

Diagram: The 12 Factors Grouped by Theme



Key Takeaways

- **"Mostly just software"**: good agents are deterministic programs with a non-deterministic LLM component — not bags of tools looping until done.
- **Own your prompts**: frameworks hide prompts; prompts should be first-class code under version control.
- **Stateless reducer pattern**: treating the agent as a fold over an event log makes it serializable, replay-able, and testable.
- **Platform scope changes the factors**: lifecycle, state, identity, billing, observability, and human handoff become shared infrastructure concerns.
- **Small, focused agents**: 3–20 steps per agent; performance degrades with context length.
- **Contact humans with tool calls**: structured `request_human_input` tools beat relying on the model's unstructured text choices.
- **Compact errors, don't hide them**: visible error traces enable self-healing; a consecutive-error counter provides a safety escalation path.

Further Reading

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

Chapter 10: Evaluation

10.1 Why Evals

Without evals, debugging is reactive: wait for complaints, reproduce manually, fix, hope nothing else regressed ([Anthropic — Demystifying Evals for AI Agents](#)). Teams cannot distinguish real regressions from noise, automate testing of changes against many scenarios, or measure improvements. They also adopt new models slowly — without evals, taking advantage of a new model means weeks of manual testing, while evaluated teams can verify strengths and tune prompts in days.

Recall that an eval grades the unit that matters — for an agent, the unit is the loop, not one model call (see Foundations ch 13). Agent evals run the whole model-plus-harness system in an environment, then judge whether the final state satisfies the task. That matters because an agent can pass an intermediate test, produce a fluent answer, or take an unusual path while still failing the user's real goal.

Anthropic positions evals as compounding infrastructure: the costs are visible up front, the benefits accumulate over the agent's lifecycle. Their advice: start early, even with 20–50 simple tasks. Effect sizes in early agent development are large, so small samples suffice; mature agents need larger evals to detect smaller effects.

10.2 The Anatomy of an Evaluation

The eval vocabulary — graders (code/model/human), traces, regression vs capability, pass@k vs pass^k, reward hacking — is established in Foundations ch 13; this chapter recaps it briefly and builds the harness-specific apparatus on top.

Anthropic's vocabulary ([Anthropic — Demystifying Evals for AI Agents](#)):

- A **task** has defined inputs and success criteria.
- A **trial** is a single attempt at a task — multiple trials per task because outputs vary.
- A **grader** scores some aspect of performance; tasks can have multiple, each containing assertions.
- A **transcript** (trace, trajectory) is the full record of a trial.
- An **outcome** is the final environmental state at end of trial — distinct from the agent's text response. (A flight-booking agent's "your flight is booked" is the response; whether a row exists in the SQL database is the outcome.)
- An **evaluation harness** (distinct from the agent harness) is the infrastructure that runs the eval end-to-end.
- An **agent harness** (or scaffold) is the system being evaluated alongside the model. *"When we evaluate 'an agent,' we're evaluating the harness and the model working together."*

10.3 Three Types of Graders

- **Code-based**: string match, binary tests, static analysis, outcome verification, tool-call verification, transcript analysis. Fast, cheap, objective, reproducible — but brittle to valid variations.
- **Model-based**: rubric scoring, natural-language assertions, pairwise comparison, multi-judge consensus. Flexible, scalable, handles open-ended tasks — but non-deterministic, requires human calibration.
- **Human**: SME review, crowdsourced judgment, A/B testing. Best for calibration and subjective judgments — but expensive, slow, and still vulnerable to inconsistency if rubrics are weak.

Anthropic recommends deterministic graders where possible, model-based where necessary, human for periodic calibration. They also caution against grading the *path* the agent took rather than what it produced — agents regularly find valid approaches the eval designer did not anticipate, and grading paths makes the eval brittle.

10.4 Capability vs. Regression Evals

Two distinct purposes ([Anthropic — Demystifying Evals for AI Agents](#)):

- **Capability evals** ask "what can this agent do well?" They start at low pass rates, targeting tasks the agent struggles with, giving teams a hill to climb.
- **Regression evals** ask "does the agent still handle what it used to?" They should run near 100%, protecting against backsliding.

After an agent matures, capability evals with high pass rates *graduate* into the regression suite. Tasks that once measured "can we do this at all?" then measure "can we still do this reliably?"

10.5 Pass@k vs. Pass^k

For agents whose behavior varies between runs, two metrics with opposite slopes ([Anthropic — Demystifying Evals for AI Agents](#)):

- **pass@k**: probability of at least one correct solution in k attempts. Rises as k increases — more shots on goal means higher odds of success.
- **pass^k**: probability that *all* k trials succeed. Falls as k increases — demanding consistency across more trials raises the bar.

These metrics exist because agent runs are stochastic. The same prompt, model, and harness can produce different tool orders, search paths, or final answers across trials. A single run is therefore weak evidence; repeated trials tell you whether you have occasional success, consistent reliability, or a brittle lucky path.

A 75% per-trial success rate gives pass^3 of about 42% and pass^10 about 5.6%, while pass@10 is about 99.9999%. The right metric depends on the product: one success matters when the system can generate multiple candidates and select or show the best one; every success matters for a customer-facing agent that must behave reliably on repeated runs.

10.6 The Eight-Step Roadmap

Anthropic's distilled roadmap to going from no evals to evals you trust ([Anthropic — Demystifying Evals for AI Agents](#)):

0. **Start early** — 20–50 tasks from real failures.
1. **Begin with what you already test manually** — pre-release checks and bug-tracker queue items.
2. **Write unambiguous tasks with reference solutions** — two domain experts should reach the same verdict; a 0% pass rate across many trials usually means a broken task, not an incapable agent.
3. **Build balanced problem sets** — both cases where a behavior should and should not occur. One-sided evals create one-sided optimization.
4. **Build a robust eval harness with a stable environment** — isolate trials, no shared state. Anthropic observed Claude gaining unfair advantage from inspecting git history left over from previous trials.
5. **Design graders thoughtfully** — deterministic where possible, partial credit for multi-component tasks, calibrated LLM-as-judge with structured rubrics, escape hatches for "Unknown" to avoid hallucination, anti-hacking design.
6. **Read transcripts** — failures should look fair; if scores stop climbing, the question is whether the agent regressed or the eval itself is now unfair.
7. **Monitor for capability eval saturation** — an eval at 100% provides no improvement signal. SWE-bench Verified started at 30% and is now nearing 80%, with deceptive small score increases now hiding large capability gains.
8. **Maintain through open contribution** — domain experts and product teams should contribute eval tasks; product managers, customer success, salespeople can use Claude Code to file evals as PRs.

10.7 What Real Evals Look Like for Different Agent Types

These canonical examples per agent type follow Anthropic's survey ([Anthropic — Demystifying Evals for AI Agents](#)):

- **Coding agents**: deterministic graders are natural — does the code run, do the tests pass? SWE-bench Verified runs the test suite from a fixed GitHub issue; Terminal-Bench tests end-to-end tasks like building a Linux kernel from source.
- **Conversational agents**: success is multidimensional — ticket resolved (state check), conversation under 10 turns (transcript constraint), tone appropriate (LLM rubric). Often require a second LLM to simulate the user (τ-Bench, τ²-Bench).
- **Research agents**: groundedness checks (claims supported by sources), coverage checks (key facts included), source-quality checks (authoritative sources, not first-retrieved). Require frequent calibration against expert humans.
- **Computer-use agents**: real or sandboxed environment, URL/page-state checks, backend state verification (was an order actually placed, or did just a confirmation page appear?). WebArena and OSWorld are the canonical examples.

10.8 Verification Feedback for Coding Agents

For coding agents, the most useful graders often double as repair signals. A failing check that says only "test failed" confirms that the outcome is bad but gives the agent little traction. A better failure message names the violated path, the expected state, the observed state, and the next place to inspect. OpenAI's Codex harness guidance emphasizes

turning recurring review comments and architectural rules into repository-local checks so agents receive specific feedback while they are still able to fix the work ([OpenAI — Harness Engineering](#)).

End-to-end verification should also be treated as a done gate, not a ceremonial final step. Anthropic's long-running application harness required the coding agent to start the app and verify the feature through a browser-driven path because agents were otherwise prone to declaring features complete after local tests or visual inspection while the actual user flow remained broken ([Anthropic — Effective Harnesses for Long-Running Agents](#)). The general rule from section 9.2 applies directly: grade the environment state, not the agent's confidence.

10.9 Reading Transcripts Is the Skill

A repeated theme: do not take eval scores at face value until someone reads the transcripts. Anthropic recounts a case where Opus 4.5 initially scored 42% on CORE-Bench, but investigation revealed rigid grading penalizing "96.12" when the expected answer was "96.124991...", ambiguous task specs, and stochastic tasks that were impossible to reproduce exactly. After fixing the grading bugs and running with a less constrained scaffold, the score jumped to 95% ([Anthropic — Demystifying Evals for AI Agents](#)). Similarly, METR found tasks in their time-horizon benchmark that asked agents to optimize to a stated score threshold, but where the grading required exceeding the threshold — penalizing models that followed instructions and rewarding ones that ignored them ([Anthropic — Demystifying Evals for AI Agents](#)).

The general rule: failures should seem fair. When scores plateau, the question to ask is whether the eval is measuring what it should.

10.10 Evals Are One Layer of Many

Automated evals are not a complete picture. Anthropic compares the situation to the Swiss-cheese model from safety engineering: no single layer catches every issue ([Anthropic — Demystifying Evals for AI Agents](#)). The complete stack:

- **Automated evals** for fast iteration, regression detection, model upgrades.
- **Production monitoring** for ground truth and unanticipated real-world failures.
- **A/B testing** for validating significant changes once traffic is sufficient.
- **User feedback** for surfacing problems no one anticipated.
- **Manual transcript review** for building intuition about failure modes.
- **Systematic human studies** for calibrating LLM graders or grading subjective output.

10.11 Readiness Validation and Failure Attribution

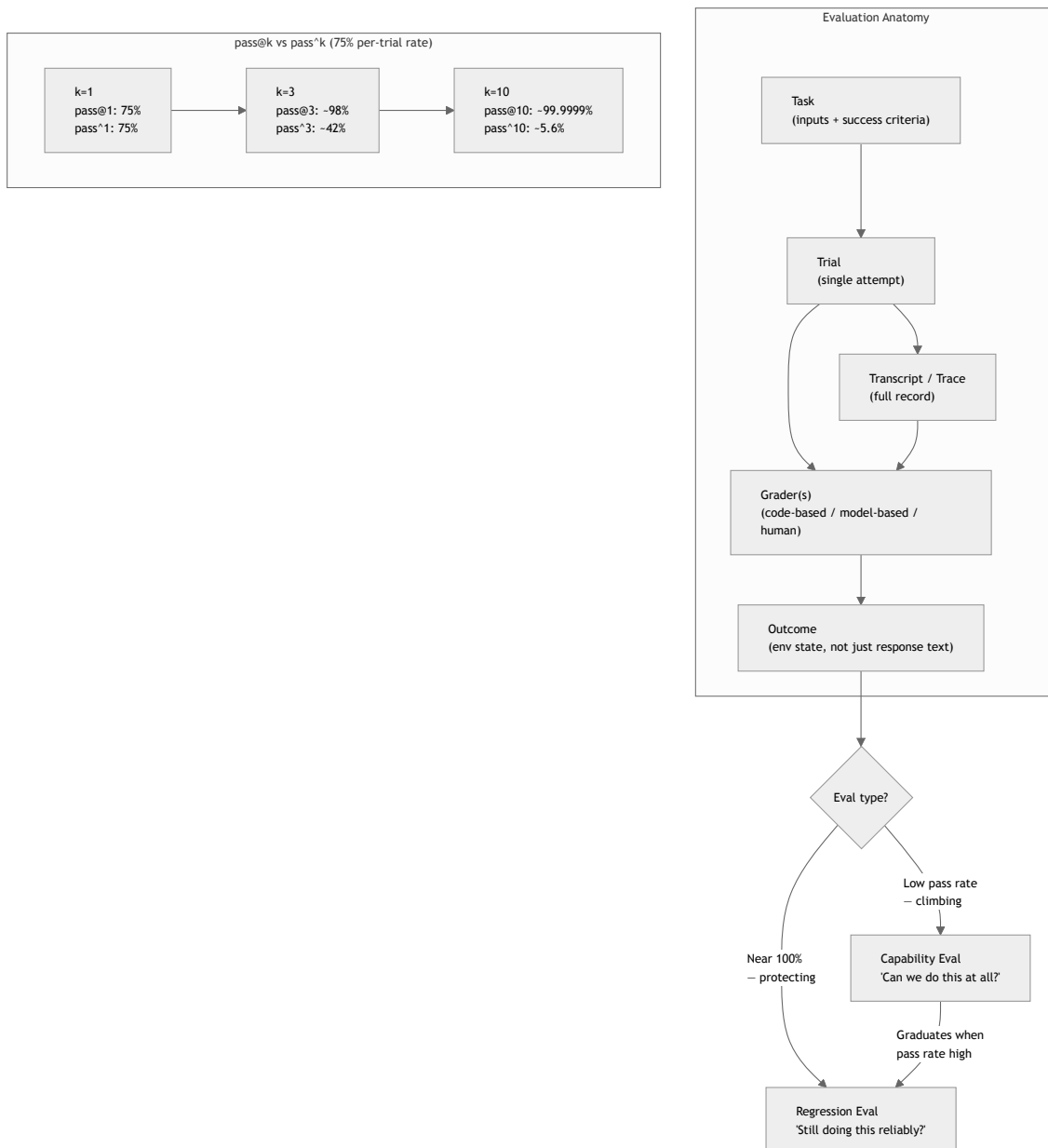
The OpenReview survey separates the **Verification** layer from generic evaluation because harnesses need more than scorekeeping. Verification asks whether a deployed agent-harness combination is ready for a specific task distribution, under a specific environment, budget, and governance regime ([OpenReview — Agent Harness Engineering: A Survey](#)).

That framing adds two practical requirements to ordinary eval suites:

- **Readiness validation**: a release gate should bind tasks, environment reset rules, tool availability, context policy, budget limits, and governance checks together. A score measured under a different execution substrate or tool menu is not automatically portable.
- **Failure attribution**: failures should be labeled by likely layer — execution, tool interface, context, lifecycle, observability, verification, or governance. Otherwise teams overfit prompts when the real defect is an unstable sandbox, oversized tool surface, missing checkpoint, or weak policy hook.

This also explains why benchmark numbers are fragile. Infrastructure changes, cost optimizations, and altered tool boundaries can change the measured capability of the same model. A useful eval report should therefore record the harness configuration alongside the model name: environment image, resource limits, tool catalog, context assembly policy, retries, graders, and human-approval rules.

Diagram: Eval Anatomy and pass@k vs pass^k



Key Takeaways

- **Evals are compounding infrastructure:** start with 20–50 tasks from real failures, even before the agent is mature.
- **Eval is broader than unit testing:** it checks the model-plus-harness system against task outcomes in an environment.
- **Outcome ≠ response:** measure the environmental state (database row, URL, file) not just what the agent said.
- **Repair-oriented feedback improves self-correction:** checks should tell the agent what failed, where, and what evidence would count as fixed.
- **Three grader types form a pyramid:** code-based for speed, model-based for nuance, human for calibration.
- **pass@k and pass^k serve different products:** multi-attempt generation can use pass@k; repeated customer-facing execution needs pass^k-style reliability.
- **Reading transcripts is the skill:** scores plateau for two reasons — agent regression or eval unfairness — only transcripts distinguish them.
- **Readiness is configuration-specific:** eval results should travel with the harness configuration that produced them.
- **Evals are one layer of many:** automated evals + production monitoring + A/B testing + user feedback + human review.

Further Reading

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025.
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026.
<https://openai.com/index/harness-engineering/>
- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

Chapter 11: Infrastructure Noise

A practitioner reading benchmark leaderboards needs to know that small score differences carry more uncertainty than the precision of the reported numbers suggests. Anthropic's "Quantifying Infrastructure Noise" documents the size of this effect ([Anthropic — Quantifying Infrastructure Noise in Agentic Coding Evals](#)).

Static benchmarks score a model's output directly. Agentic coding evals are different: the model writes programs, runs tests, installs dependencies, iterates over many turns. The runtime is not a passive container but an integral component of the problem-solving process. Two agents with different resource budgets are not taking the same test.

11.1 The Headline Result

Running Terminal-Bench 2.0 across six resource configurations on a Google Kubernetes Engine cluster — same Claude model, same harness, same task set, varying only the resource floor and ceiling — the gap between the most- and least-resourced setups was 6 percentage points ($p < 0.01$) ([Anthropic — Quantifying Infrastructure Noise](#)).

This is more than the typical leaderboard gap between top models. The implication is direct: a 2-point lead on a leaderboard might reflect a real capability gap, or it might reflect that one eval ran on beefier hardware.

11.2 The Two Regimes

The data reveals two regimes:

- **From 1× to 3× the per-task resource specs**, scores fluctuated within noise ($p = 0.40$), but infrastructure error rates dropped monotonically — from 5.8% at strict enforcement to 2.1% at 3× headroom, $p < 0.001$. Tasks that crashed at 1× would have failed regardless. The extra resources fixed transient memory spikes that were OOM-killing containers, without making the eval itself easier.
- **Above 3×**, scores climbed faster than infrastructure errors declined. From 3× to uncapped, infra errors dropped 1.6 percentage points but success jumped almost 4 points. Extra resources let the agent try approaches that only work with generous allocations: pulling in large dependencies, running memory-intensive test suites, brute-forcing solutions with heavyweight tools.

11.3 What This Means for Measurement

Tight resource limits inadvertently reward efficient strategies; generous limits reward agents that exploit available resources. Both are legitimate things to test, but collapsing them into a single score without specifying configuration makes interpretation difficult.

Anthropic's `bn-fit-modify` example illustrates: under generous limits, some models default to installing the entire Python data-science stack (pandas, networkx, scikit-learn) before writing any solution code. Under tight limits, the pod runs out of memory during installation. A leaner strategy exists — implementing the math from scratch using only the standard library — and some models default to it. The resource configuration determines which default succeeds ([Anthropic — Quantifying Infrastructure Noise](#)).

The same effect holds outside Terminal-Bench, though with smaller magnitude. Anthropic's SWE-bench experiment with 5× RAM showed scores 1.54 percentage points higher at 5× than 1× across 227 problems — smaller than Terminal-Bench because SWE-bench tasks are less resource-intensive, but non-neutral ([Anthropic — Quantifying Infrastructure Noise](#)).

11.4 The Recommendation

Evals should specify both a guaranteed allocation (the floor) and a hard ceiling separately, not a single pinned value. A 3× ceiling over per-task specs is a reasonable default for Terminal-Bench: it cut infra errors by two-thirds while keeping the score lift well within noise ([Anthropic — Quantifying Infrastructure Noise](#)). The exact multiplier depends on benchmark and task distribution and should be reported.

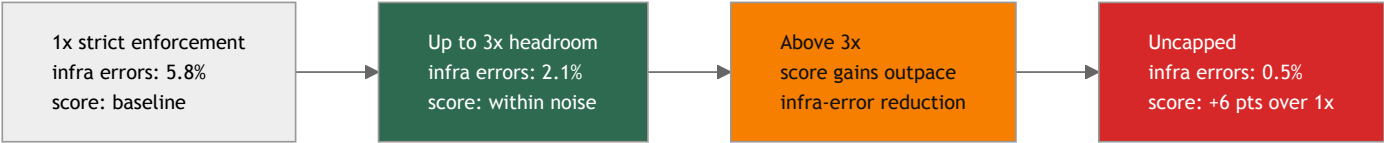
For consumers of benchmark results, the operational rule: leaderboard differences below 3 percentage points deserve skepticism until configuration is documented and matched. A few-point lead might signal a real capability gap, or it might just be a bigger VM.

Diagram: Resource Configuration vs. Score (Terminal-Bench 2.0 Summary)

The following table summarizes the two regimes reported by Anthropic. Only the 1×, 3×, and uncapped error rates are explicitly quantified in the article; intermediate rows are qualitative to avoid implying more precision than the source

provides.

Resource Level	Infra Error Rate	Score Change	Interpretation
1× (strict)	5.8%	baseline	OOM kills mask real failures
3×	2.1%	+noise	Sweet spot: infra errors cut 2/3
Above 3×	lower	score begins rising faster than infra errors fall	Agents start exploiting extra RAM
Uncapped	0.5%	+6 pts over 1×	Resource-intensive defaults succeed



Above 3×, score gains outpace the decline in infrastructure errors — extra resources enable strategies, not just stability.

Key Takeaways

- **6-point gap from hardware alone:** the most- vs. least-resourced configuration on Terminal-Bench 2.0 produces a difference larger than typical top-model gaps on leaderboards.
- **Two regimes:** 1×–3× fixes infrastructure instability; above 3× enables resource-intensive strategies.
- **3× ceiling is the practical default:** cuts infrastructure errors by two-thirds while keeping score lift within noise.
- **Leaderboard skepticism threshold:** differences below 3 percentage points are suspect until resource configurations are documented and matched.
- **Resource limits shape strategy:** tight limits reward efficiency; generous limits reward exploitation — both are valid but should be distinguished.

Further Reading

- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>

Chapter 12: Trace-Driven Iteration and Model–Harness Co-Evolution

12.1 Traces Are the Feedback Loop

Models today are largely black boxes; their inner mechanisms are hard to interpret. But their inputs and outputs are visible in text, and that is enough to drive systematic improvement. LangChain treats traces as the primary surface for harness debugging: every agent action is stored, including latency, token counts, costs, and tool invocations ([LangChain — Improving Deep Agents with Harness Engineering](#)).

Their *Trace Analyzer Skill* automates the loop:

1. Fetch experiment traces from LangSmith.
2. Spawn parallel error-analysis agents; the main agent synthesizes findings and suggestions.
3. Aggregate feedback and make targeted changes to the harness.

This is structurally similar to boosting in classical machine learning — iteration focuses on mistakes from previous runs. Human review at step 3 is helpful but not strictly required, mainly to catch changes that overfit to specific tasks at the cost of generalization.

12.2 Runtime and Process Observability

Trace-driven iteration needs two kinds of observability. **Runtime observability** records what the system did: tool calls, command outputs, browser actions, latency, token usage, errors, retries, and final environment state. This is the layer LangChain emphasizes when it stores every agent action and aggregates failure patterns from traces.

Process observability records why the current work should be accepted: task scope, sprint contracts, rubrics, verification evidence, excluded work, and handoff notes. Anthropic's generator-evaluator harness made this explicit with sprint contracts negotiated before implementation and evaluator rubrics that produced specific feedback when a sprint failed ([Anthropic — Harness Design for Long-Running Application Development](#)).

The two layers reinforce each other. Runtime signals without process artifacts tell you what happened but not whether it satisfied the intended scope. Process artifacts without runtime signals can become plausible paperwork around broken behavior. A production harness should make both inspectable: the task trajectory, the acceptance criteria, and the evidence that the environment actually reached the desired state.

The OpenReview survey adds a useful implementation detail: agent traces should be structured as trees of spans, not flat logs ([OpenReview — Agent Harness Engineering: A Survey](#)). At minimum, trace spans should cover model calls, tool invocations, retrieval steps, context-assembly operations, latency, token usage, cost, retries, permission decisions, and final outcome state.

That structure is now being standardized. OpenTelemetry — the vendor-neutral observability standard already used across distributed systems — is defining *GenAI semantic conventions*: an agreed schema of span and attribute names for LLM and agent telemetry ([OpenTelemetry — Semantic conventions for generative AI spans](#)). The conventions name the spans a harness produces — a top-level `invoke_agent`, child `chat` spans for each model call, and `execute_tool` spans for each tool invocation — plus standard metrics such as operation duration and token usage. Adopting them buys two things. First, agent traces join the *same* observability stack as the rest of a system's services, so latency, cost, and errors are queryable with existing tooling rather than a bespoke agent logger. Second, the standard confronts the privacy problem directly: it defines explicit patterns for whether to record prompt and completion content at all — omit it, attach it to the span, or store it externally and reference it — and recommends against capturing sensitive payloads by default, which is exactly the redaction discipline §12.3 and Chapter 17 require. (As of this writing the GenAI conventions are still marked *Development*, so exact names will shift; the value is the direction, not the version number.)

12.3 From Production Trace to Regression Case

Observability should feed verification. A trace that exposes a real production failure is too valuable to remain only a debugging artifact. The mature loop is:

1. Capture a failed or surprising production trajectory.
2. Redact sensitive data and freeze the relevant environment or fixture.
3. Extract the user intent, tool sequence, intermediate state, and final outcome.
4. Write a deterministic or model-graded assertion for the corrected behavior.

5. Add the case to the regression suite, with the original trace as evidence.

This turns traces into a source of eval tasks. It also prevents teams from optimizing against synthetic benchmark tasks while missing the failures their own users generate. The boundary with governance matters: trace-to-eval pipelines must preserve privacy, provenance, and permission metadata, or they will produce tests that are useful technically but unsafe operationally.

12.4 Stress-Test Load-Bearing Components

Anthropic's harness-design follow-up adds a complementary discipline ([Anthropic — Harness Design for Long-Running Application Development](#)). Every component in a harness encodes an assumption about what the model cannot do on its own. As models improve, those assumptions go stale. The recommended approach: remove one component at a time, run the eval, observe.

When Opus 4.6 launched with stronger long-context retrieval and better long-horizon coding behavior, Anthropic was able to remove the sprint construct in one version of the harness. The generator ran coherently for over two hours without sprint decomposition. The evaluator, which had been more load-bearing on earlier models, became more situational on 4.6 — useful for tasks at the edge of what the generator could do solo, unnecessary overhead within that boundary. The general principle the team articulates: "the evaluator is not a fixed yes-or-no decision. It is worth the cost when the task sits beyond what the current model does reliably solo."

12.5 Model–Harness Co-Evolution

Today's frontier coding models are post-trained with their harnesses in the loop ([LangChain — The Anatomy of an Agent Harness](#)). Useful primitives are discovered, added to the harness, and used in training the next generation, which becomes more capable within that harness. This creates a feedback loop with side effects: changing harness logic can produce worse model performance even when the change should be neutral.

The Codex `apply_patch` tool is the canonical example. Codex models are post-trained on this specific patching format, and OpenCode — built as an open-source alternative to Claude Code — had to add an `apply_patch` tool specifically for GPT/Codex models to mimic the Codex harness; Claude and other models still use normal `edit` and `write` tools ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)).

12.6 But the Best Harness Is Not Always the One the Model Was Trained In

The corollary, and the practical license to iterate: the harness a model was trained in is often *not* optimal for a given task. Terminal-Bench 2.0 has been a recurring data point in the practitioner discussion — HumanLayer cites Opus 4.6 in Claude Code at position 33, while the same model in a different harness places at position 5, with about four positions of leaderboard noise ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#); [LangChain — The Anatomy of an Agent Harness](#)). Treat the exact ranks as leaderboard snapshots, not timeless model facts.

LangChain's case study reaches the same conclusion experimentally. They ran a Claude Opus 4.6 test on an early version of their harness that scored 59.6%, competitive but worse than their tuned Codex configuration. The principles generalized — context preparation, verification — but a few rounds of harness iteration tailored to the model would have closed the gap ([LangChain — Improving Deep Agents with Harness Engineering](#)).

The pragmatic rule: if you change the model, re-examine the harness. Tune what is now load-bearing, strip what is no longer.

12.7 Practical Takeaways

LangChain's distilled principles for harness iteration ([LangChain — Improving Deep Agents with Harness Engineering](#)):

1. **Context engineering on behalf of agents** — onboard the model into its environment with directory structures, available tools, coding best practices, problem-solving strategies.
2. **Help agents self-verify their work** — models bias toward their first plausible solution; prompt aggressively to verify by running tests.
3. **Tracing as a feedback signal** — debug tooling and reasoning together (models go down wrong paths because they lack a tool or the instructions for one).
4. **Detect and fix bad patterns in the short term** — guardrails like loop detection are crutches that will dissolve as models improve, but are useful today.
5. **Tailor harnesses to models** — Claude and Codex prompting guides differ for a reason; principles generalize, specifics do not.

HumanLayer's parallel set ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)):

What worked: starting simple and adding configuration only after real failures; iterating and throwing away things that did not help; distributing battle-tested configurations across the team via repository-level config; optimizing for iteration speed rather than likelihood of one-shotting; pruning capabilities once you know what you actually need.

What did not work: designing the ideal harness upfront; installing dozens of skills and MCP servers "just in case"; running the full test suite at the end of every session; micro-optimizing which sub-agents could access which tools.

12.8 The Misleading Data on AGENTS.md

A worth-reading detail: an ETH Zurich study tested 138 agentfiles (the generic term for AGENTS.md / CLAUDE.md-style instruction files) across various repos and found that LLM-generated ones hurt performance while costing 20% more, that human-written ones helped only about 4%, that agents spent 14–22% more reasoning tokens processing context-file instructions, and that codebase overviews and directory listings did not help in that benchmark because agents could discover repository structure on their own ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#) citing the ETH Zurich paper).

HumanLayer reads this as confirming their own AGENTS.md guidance — keep files concise, avoid auto-generation, use progressive disclosure rather than dumping every instruction up front, keep contents universally applicable rather than full of conditional rules. Their own CLAUDE.md is under 60 lines.

This should not be read as "do not write repository instructions." It means the root instruction file should be a router, not an encyclopedia. OpenAI's Codex harness guidance takes the same shape: keep essential context repo-local, keep AGENTS.md concise, point agents to the right deeper documents, and enforce the most important rules mechanically where possible ([OpenAI — Harness Engineering](#)).

The general principle: more configuration is not better. Every irrelevant instruction is an instruction the agent must process for no benefit, and the *instruction budget* matters as much as the token budget.

12.9 Reusable Harness Packages and Skills

Once a harness pattern has survived real use, it should not remain tribal knowledge in one repository. Package it. The practical unit can be a skill, a template bundle, a small scaffold generator, or a set of repo checks. The important property is that it carries both instructions and working artifacts: not just "remember to maintain state," but a progress-log template, a feature-list schema, a startup script, and a validation command.

The Learn Harness Engineering course demonstrates this packaging shape with `harness-creator`, a skill for creating, assessing, and improving five harness subsystems: instructions, state, verification, scope, and session lifecycle ([Learn Harness Engineering — Skills](#)). This is a useful engineering boundary. A reusable harness package should not freeze one ideal workflow forever; it should make proven defaults easy to install, easy to inspect, and easy to remove when traces show that a component is no longer load-bearing.

12.10 Meta-Harness: Optimizing the Harness Itself

Once evals and traces exist, the harness itself becomes an optimization object. The OpenReview survey points to *meta-harness* work that searches over harness structure, prompting strategy, tool interface, and control loop choices rather than treating the harness as fixed ([OpenReview — Agent Harness Engineering: A Survey](#)).

The pragmatic version is not fully automated architecture search. It is disciplined experimentation:

- A/B test a tool schema or prompt change against the same task suite.
- Ablate one component, such as a planner, context reset, memory layer, or evaluator.
- Vary cost controls, retry policy, and tool-response truncation to see where quality degrades.
- Measure the whole closed loop, not only model output: success rate, pass^k reliability, latency, cost, human escalations, and security false positives.

This extends the load-bearing idea. Harness components should earn their keep continuously. If a component improves reliability only for rare high-value tasks, route it selectively. If it no longer helps after a model upgrade, remove it.

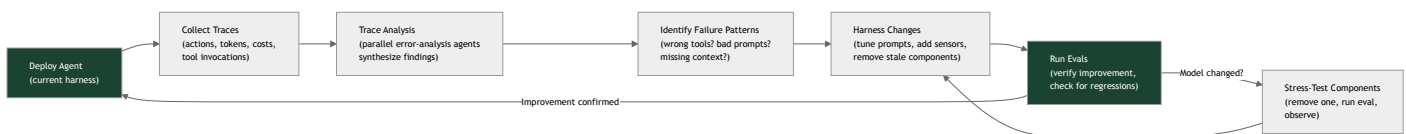
12.11 Cross-Layer Coupling

ETCLOVG is also a debugging map. When traces show a failure, the apparent bad model decision may be a symptom of another layer ([OpenReview — Agent Harness Engineering: A Survey](#)):

- A tool-selection error may come from the **Tool** layer, because the action space is too large or the schema hides the important affordance.
- A premature completion may come from the **Lifecycle** layer, because exit criteria are not represented as a durable state machine.
- A regression after cost tuning may come from the **Observability** and **Execution** layers, because resource limits changed latency, timeouts, or benchmark fidelity.
- A security failure may come from the **Governance** layer, because policy, permissions, and audit hooks are inconsistent across training-time alignment, deployment configuration, and runtime enforcement.

The practical move is to annotate trace reviews with layer hypotheses before changing prompts. Good harness iteration asks: which layer made the wrong behavior easy, invisible, or cheap? Only then should the team decide whether the fix is a prompt, a tool redesign, a sandbox change, a new metric, a stronger grader, or a governance hook.

Diagram: Trace-Driven Iteration Loop



Key Takeaways

- **Traces are the primary debugging surface:** text I/O is visible even when model internals are not — systematic trace analysis drives harness improvement.
- **Observability has two layers:** runtime traces show what happened; process artifacts show why the work should be accepted.
- **Trace spans should be structured:** model calls, tool calls, retrieval, context assembly, permissions, cost, and outcome state need machine-readable telemetry.
- **A standard is emerging for that telemetry:** OpenTelemetry's GenAI semantic conventions (`invoke_agent` , `chat` , `execute_tool` spans) let agent traces join the ordinary observability stack and codify content-capture patterns for privacy.
- **Production traces should become regression cases:** real failures are the highest-signal eval tasks if privacy and provenance are preserved.
- **The trained harness is not automatically optimal:** leaderboard snapshots show the same model moving substantially under different harnesses.
- **Stress-test components when models change:** every harness component encodes an assumption that may go stale as models improve.
- **Model-harness co-evolution is real:** post-training loops the harness into model training, creating coupling that breaks when either side changes unexpectedly.
- **AGENTS.md has limited ROI when bloated:** keep it concise and human-written, then use progressive disclosure into repo-local docs and mechanical checks.
- **Reusable harness packages preserve hard-won practice:** promote stable patterns into skills, templates, scaffolds, and checks once traces show they help.
- **Meta-harness turns evals into design search:** prompts, tools, retries, context policies, and evaluators can be ablated and optimized like any other system component.
- **Layer attribution prevents prompt-only fixes:** use ETCLOVG to ask which harness layer made a failure possible before changing instructions.
- **Iteration speed beats upfront design:** start simple, add only after real failures, prune aggressively.

Further Reading

- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026.
<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>

- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Walking Labs, *Learn Harness Engineering — Skills*. <https://walkinglabs.github.io/learn-harness-engineering/en/skills/>
- OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

Chapter 13: System Prompts and Instruction Architecture

The core diagram in the [Preface](#) lists *System Prompts* as the first component of the harness, yet the preceding chapters have treated them as a given. This chapter looks at the instruction layer directly: what belongs in it, how competing instructions are prioritized, how the layer is assembled at runtime, and why it should be engineered with the same discipline as any other part of the system.

13.1 The System Prompt Is a Harness Layer, Not a Prompt

A "prompt" in the casual sense is a question typed into a chat box. The system prompt of an agent is something else: a persistent program that ships with the harness and frames every single turn of the agent loop. HumanLayer's twelve-factor manifesto names this directly — *own your prompts* — and argues that for production agents the prompt is core engineering logic, not a string to outsource to a framework's hidden default ([HumanLayer — 12-Factor Agents](#)). Treating it as application code is the premise of everything below.

The distinction matters because the system prompt does work that no other layer can. It establishes the agent's role and objective, declares the available tools and when to use them, encodes policy the agent must not violate, and sets the output contract. When any of these is vague, the failure does not appear as a syntax error; it appears as drift, over-eagerness, refusal, or a tool used at the wrong moment — failures that are hard to attribute precisely (Ch 10, [Ch 12](#)).

13.2 The Instruction Hierarchy

An agent receives instructions from several origins at once: the platform's system prompt, a developer's configuration, the user's request, and — critically — text that arrives inside tool results and retrieved documents. These do not carry equal authority, and the model must be told so. OpenAI formalized this as the *instruction hierarchy*: models can be trained to treat system-level instructions as privileged over user-level instructions, and both as privileged over content encountered in tool outputs, so that a lower-priority instruction cannot override a higher-priority one ([OpenAI — The Instruction Hierarchy](#)).

For the harness engineer this is the same boundary that [Chapter 5](#) drew for security, seen from the instruction side. Prompt injection is precisely a failure of the hierarchy: text in a web page or file ([the lethal trifecta](#), Ch 5) tries to promote itself to instruction level. The hierarchy gives two complementary defenses:

- **Rely on the model's trained priority** by placing genuine policy in the system position, never in user-editable or tool-supplied positions.
- **Reinforce it structurally** by labeling untrusted spans as data, because a trained priority is a tendency, not a guarantee.

The practical rule: an instruction's authority should come from *where the harness puts it*, not from how forcefully it is phrased. A constraint that matters belongs in the system layer; a constraint written into a retrieved document is a suggestion the agent is free to ignore — and should be made to ignore.

13.3 What Belongs in the System Prompt

Not everything that is true needs to be stated, and an overstuffed system prompt spends the attention budget of [Chapter 2](#) before the task even begins. A useful division:

- **Belongs in the system prompt:** the agent's role and goal, the high-value rules it must always follow, the tool-use policy that a tool description cannot express on its own (when *not* to use a tool, how tools relate), and the output contract.
- **Belongs in tool descriptions, not the system prompt:** the mechanics of each individual tool (Ch 4). Duplicating tool detail into the system prompt creates two sources of truth that drift apart.
- **Belongs in retrieved context, not the system prompt:** facts that change, that are large, or that are needed only sometimes. These should arrive through just-in-time retrieval (Ch 2), not be baked into a static prefix.

13.4 Dynamic Assembly and the Stable-Prefix Constraint

Most real agents do not ship one fixed string. The system prompt is *assembled* per call from parts: a base policy, the current tool set, project-specific instructions, possibly a retrieved skill ([Chapter 4](#)). This assembly collides with the KV-cache economics of [Chapter 2](#): the cache is reused only for a byte-identical prefix, so anything that changes between calls should live *after* everything that does not.

This turns instruction architecture into a layout problem. Stable, reusable material — base policy, the standing tool catalog — goes first. Volatile material — the current task, freshly retrieved facts, a timestamp — goes last. A system prompt that interpolates the current time or a request ID near its top quietly defeats prefix caching on every turn, paying a latency and cost penalty for no behavioral benefit.

13.5 Treat Prompts as Versioned Code

Because the system prompt is load-bearing, a change to it is a change to the system and carries the same regression risk as a code change. The discipline of [Chapter 10](#) applies without modification: a prompt edit should be run against the eval suite before and after, with pass rates, failure categories, cost, and latency compared. The model-harness coupling of [Chapter 12](#) sharpens the point — a prompt tuned for one model can regress on the next, so prompts are versioned alongside the model they were validated against.

Concretely, the instruction layer deserves: version control, a changelog tied to eval results, and an owner. "Own your prompts" means exactly this — the prompt is a maintained artifact with a history, not a string someone edited live in production ([HumanLayer — 12-Factor Agents](#)).

13.6 The Right Altitude

The hardest judgment in writing a system prompt is its level of specificity. Anthropic frames this as finding the *right altitude*: too low, and the prompt becomes a brittle pile of hardcoded if-then rules that break on the first unanticipated case and grow without bound; too high, and the guidance is so vague that the model has no concrete signal to act on ([Anthropic — Effective Context Engineering for AI Agents](#)). The target is guidance specific enough to shape behavior reliably but general enough to transfer across the cases the agent will actually meet.

Altitude is also where over-engineering hides. Every special-case rule added in response to a single bad trace is a hostage to fortune: it narrows behavior, consumes context, and may conflict with a rule added next month. Often the better fix is not another sentence in the system prompt but a tool change, a sensor (Ch 5), or an eval case that pins the behavior down (Ch 10). The transcript tells you which ([Chapter 12](#)).

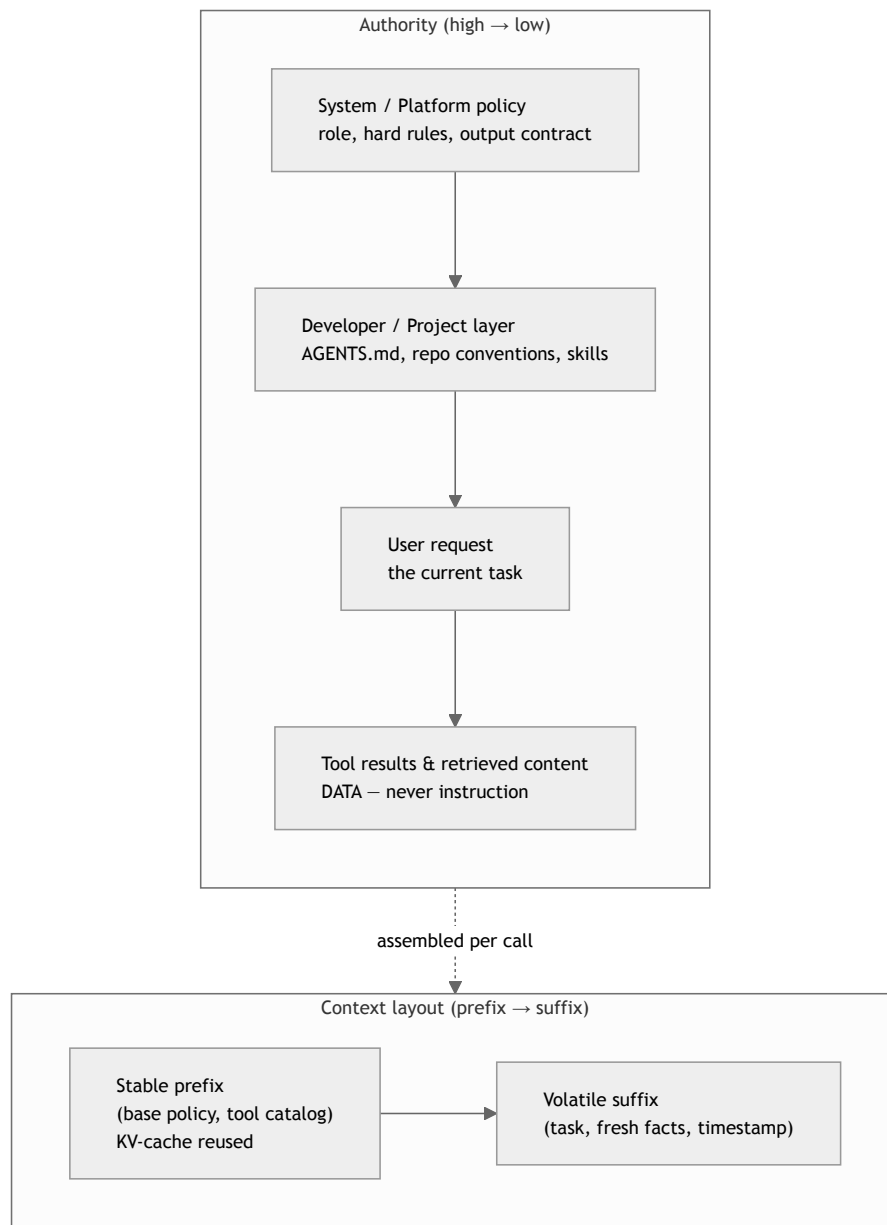
13.7 Layered Instructions Across the Three Harness Rings

The instruction layer is not monolithic; it mirrors the inner/outer harness structure of [Chapter 1](#). A coding agent typically composes:

- the **builder harness** system prompt shipped by the lab,
- the **user harness** project instructions a team adds — `AGENTS.md` files, repo conventions, review rules (Ch 12) — and
- on-demand **skills** loaded for a particular task (Ch 4).

These compose at runtime into the effective instruction set the model sees. The lesson of [Chapter 12](#) carries over: do not assume more project instruction is always better. The evidence on sprawling `AGENTS.md` files is mixed, and an over-specified project layer can fight the builder harness it sits on top of. The instruction architecture is something to measure, not to maximize.

Diagram: The Instruction Stack



Authority flows top-down and is enforced by position, not phrasing; layout flows prefix-to-suffix and is governed by cache economics. The two axes are independent and both must be designed.

Key Takeaways

- **The system prompt is application code:** a persistent, load-bearing harness layer, not a casual string — own it, version it, and test changes against evals.
- **Authority comes from position, not emphasis:** the instruction hierarchy makes system instructions privileged over user input and tool content; prompt injection is a hierarchy failure, so label untrusted spans as data.
- **Separate what belongs where:** role and policy in the system prompt, mechanics in tool descriptions, changing facts in just-in-time retrieval.
- **Assemble for the cache:** stable instructions first, volatile material last, or prefix caching silently breaks.
- **Aim for the right altitude:** specific enough to steer, general enough to transfer; resist patching every trace with a new hardcoded rule.

Further Reading

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>

Chapter 14: Model Selection, Routing, and Reasoning Models

Agent = Model + Harness. The previous chapters engineered the harness while treating "the model" as a single fixed choice. In practice the model side is itself a design surface: which model runs a given step, whether a cheaper one would do, and — since the arrival of reasoning models — how much the model should be allowed to think before it acts. This chapter covers the model-facing decisions the harness owns. For what these models are doing internally, the companion volume *LLM Foundations* is the reference; here the concern is operational.

14.1 Model Choice Is a Harness Decision

The first principle of [Chapter 6](#) — do the simplest thing that works — applies to models as much as to control flow. The largest, most capable model is rarely the right default for every step of an agent loop. Most loops contain a mix of work: a hard planning or synthesis step that genuinely needs a frontier model, surrounded by easy steps — classifying an input, formatting a result, extracting a field — that a smaller, faster, cheaper model handles just as well ([Anthropic — Building Effective Agents](#)).

Treating the model as a single global setting leaves this on the table. The harness can select per step, and the unit of selection is the call, not the application. This is the same instinct as the model-routing note in [Chapter 3](#), generalized: capability spent where it does not change the outcome is capability wasted, and in production that waste is measured in latency and dollars on every request.

14.2 Routing: Match Model Strength to Task Difficulty

Routing in the workflow sense (Ch 6) dispatches an input to a specialized branch. *Model routing* is the same idea applied to model choice: classify the difficulty of a request and send easy ones to a weak, cheap model and hard ones to a strong, expensive one. The challenge is the classifier — routing only pays off if deciding the route is much cheaper than the saving it produces.

RouteLLM showed this can be learned: a router trained on preference data sends queries to a strong or weak model and recovers most of the strong model's quality at a fraction of the cost on standard benchmarks ([RouteLLM](#)). The harness-level takeaway is not a specific router but the shape of the lever: a small, fast decision in front of an expensive call can move the cost-quality frontier, *if* the routing decision is itself reliable and cheap. A misrouted hard task that fails silently can cost far more than the model it saved.

14.3 Cascades and Fallbacks

Routing decides up front. A *cascade* defers the decision: try the cheap model first, accept its answer if a verifier is satisfied, and escalate to a stronger model only when it is not. FrugalGPT demonstrated cascades that match a frontier model's accuracy at a large cost reduction by sending most queries to cheaper models and escalating only the hard residual ([FrugalGPT](#)).

A cascade is only as good as its escalation signal — the verifier that decides "this answer is not good enough." That is the same verification machinery as everywhere else in this book: a code-based check, a test, a schema validation, or a model-based grader (Ch 5, [Ch 10](#)). Without a trustworthy signal, a cascade just adds latency before producing the same wrong answer. The related production pattern is the *fallback*: when the primary model errors, times out, or is rate-limited, fail over to an alternate so the agent degrades gracefully instead of stalling — a reliability concern that belongs with the checkpoint/resume discipline of [Chapter 7](#).

14.4 Reasoning Models and Test-Time Compute

A reasoning model is post-trained — often with reinforcement learning on checkable problems — to generate a long internal chain of reasoning before committing to an answer, spending extra inference tokens to do better on hard problems. DeepSeek-R1 documented that strong reasoning behavior can be elicited largely through RL on verifiable rewards ([DeepSeek-R1](#)), and a line of work has shown that *test-time compute* — letting a model think longer at inference — can be a more effective use of budget than a larger model on some problems ([Snell et al. — Scaling LLM Test-Time Compute](#)). The companion volume covers the mechanism (*LLM Foundations*, Ch 7–8); the consequence here is that "how long the model thinks" has become a harness-controllable axis, distinct from "which model."

This is a different scaling axis from anything in [Chapter 11](#). There, more hardware enabled more agent *actions*. Here, more inference budget buys more *thinking* inside a single call, before any tool is touched. Both cost money; they are not interchangeable.

14.5 Harnessing Reasoning Models: Don't Fight the Training

A reasoning model changes several harness assumptions, and the failure mode is fighting behavior the model already has:

- **Do not hand-prompt the reasoning it does natively.** Forcing "think step by step" onto a model trained to reason can waste tokens or conflict with its trained behavior. Follow the provider's guidance for that model class.
- **Budget for invisible tokens.** A reasoning model can emit thousands of hidden reasoning tokens before its first visible word. That is real latency and real cost. Where the model exposes a reasoning-effort control, treat it as a first-class harness parameter and, when appropriate, surface it to the user.
- **Do not trust the trace as an explanation.** The reasoning text may be hidden, summarized, or unfaithful to the model's actual computation. Treat any exposed reasoning as a debugging aid, not verification — the same caution the eval chapter applies to model self-reports (Ch 10).
- **Reasoning is not grounding.** A longer internal monologue is still ungrounded generation. It can reason more carefully over evidence the harness supplies, but it cannot manufacture facts it was never given. Tools, retrieval, and verification remain mandatory (Ch 4, Ch 10).

14.6 Routing to Reasoning: When the Extra Tokens Pay Off

Reasoning models help most on math, code, planning, and multi-step analysis — work where a checkable, hard sub-problem sits inside the loop. They help least on simple extraction, formatting, and classification, where they are slower and more expensive for no quality gain. This makes reasoning a routing target like any other: send the hard, verifiable step to a reasoning model and keep the surrounding easy steps on a fast non-reasoning model (§14.2).

The micro-agent pattern of Chapter 6 composes naturally here. A deterministic DAG can place a reasoning-model call at exactly the node that needs deliberation — a "reasoning sandwich" with cheap deterministic work on either side — rather than paying for deliberation on every turn of an open-ended loop.

14.7 Model Upgrades Are Harness Events

Because frontier models are increasingly post-trained with their harnesses in the loop, the model and the harness are coupled (Chapter 12). Swapping the model — an upgrade, a cheaper provider, a quantized variant, a different reasoning class — is therefore a change to the system, not a drop-in. A model that scores higher on public benchmarks can still regress on your workflow because it follows tool descriptions differently, has different verbosity, or reasons when you wanted a fast answer.

The discipline is the one from Chapter 10: every model change is a regression event. Run the eval suite before and after, watch failure categories and cost as well as pass rate, and keep the prompts and tools that were validated against the old model versioned alongside it. Readiness validation is per model-plus-harness configuration, not per model.

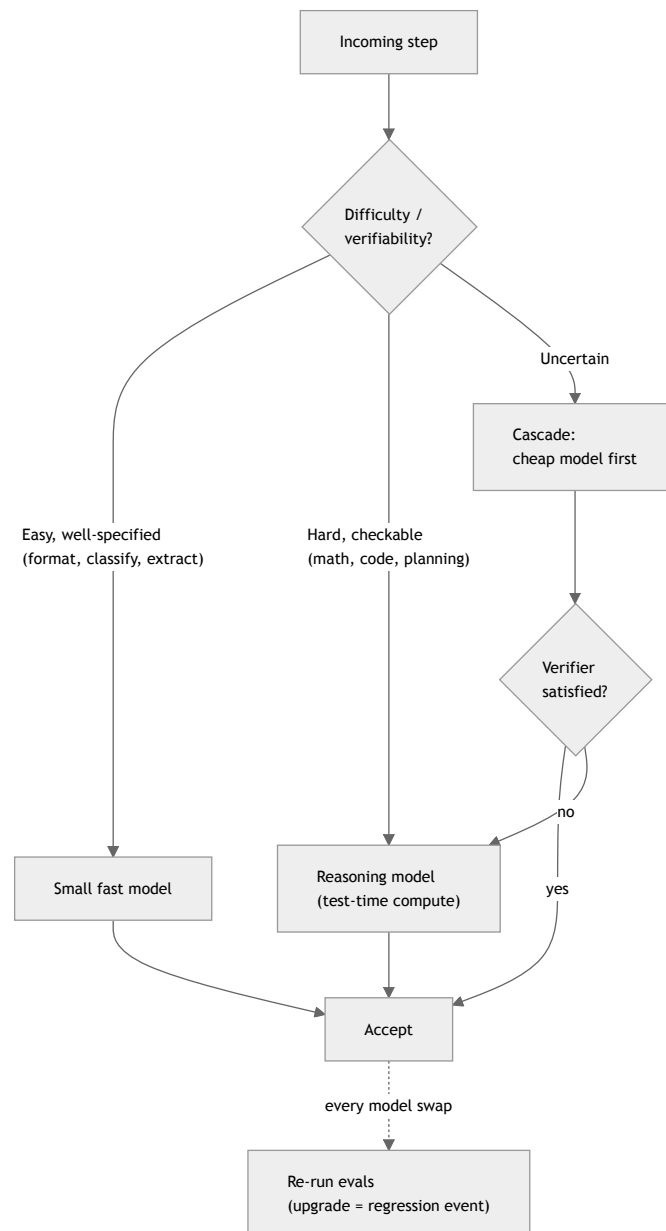
14.8 The AI Gateway: Routing as Infrastructure

Sections 14.2–14.3 treated routing, cascades, and fallbacks as *logic*. In production, that logic increasingly lives in a dedicated infrastructure component — the *AI gateway* (also LLM gateway or LLM proxy). A gateway sits between the harness and the model providers and presents a single, usually OpenAI-compatible, endpoint over many providers, while absorbing cross-cutting concerns: model routing and fallback, load balancing, retries, per-key budgets and rate limits, caching, and request/response logging. Open-source examples include LiteLLM, which fronts 100-plus providers behind one interface with cost tracking and load balancing (LiteLLM), and Portkey, which additionally folds guardrails and PII redaction into the gateway layer (Portkey — AI Gateway).

The reason it belongs in a harness discussion is that it externalizes several harness responsibilities into one place. The fallback pattern of §14.3 becomes a gateway configuration rather than per-agent code. The cost budgets and per-span attribution of Chapter 17 attach naturally at the gateway, which sees every call. And the model-swap-as-regression-event discipline of §14.7 is easier to run when the swap is a single gateway setting behind a stable interface — you can canary a new model on a slice of traffic without touching agent code.

The tradeoffs are the obvious ones. A gateway is a new dependency in the hot path, so it is itself a place to apply the circuit breakers and timeouts of §5.11 — an outage in the gateway is an outage in every agent behind it. And centralizing routing does not remove the need for a *reliable routing decision* (§14.2); it only gives that decision a home. The gateway is plumbing, in the same sense §4.3 called MCP plumbing: it makes good routing convenient to operate, but it does not make a bad routing policy good.

Diagram: The Model-Selection Decision



Route easy steps down, hard checkable steps up; cascade when unsure and let a verifier decide escalation. Any change to the model is a regression event for the whole configuration.

Key Takeaways

- **Model choice is per-step, not per-app:** spend frontier capability only where it changes the outcome; route easy steps to small fast models.
- **Routing decides up front, cascades decide after:** a learned router can shift the cost-quality frontier ([RouteLLM](#)); a verifier-gated cascade matches strong-model accuracy at lower cost ([FrugalGPT](#)) — both depend on a cheap, reliable decision signal.
- **Reasoning models add a new axis:** test-time compute trades inference tokens for quality on hard, checkable tasks — distinct from picking a bigger model or buying more hardware.
- **Don't fight the training:** don't hand-prompt native reasoning, budget for hidden tokens, don't trust the trace as explanation, and remember reasoning is not grounding.
- **Model upgrades are harness events:** re-run evals on every swap; readiness is per model-plus-harness configuration.
- **The AI gateway gives routing a home:** LiteLLM, Portkey, and similar proxies host fallback, budgets, caching, and logging behind one interface — convenient infrastructure, but it still needs a reliable routing decision and its own circuit breakers.

Further Reading

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
- DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- *LLM Foundations for Harness Engineering* (companion volume), Chapters 5–8. [../llm-foundations/](https://llm-foundations.com/)
- LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
- Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>

Chapter 15: Human-Agent Interaction

Most of this book is about the machinery between the model and the world: context, tools, sandboxes, state, evaluation. But for any agent that does consequential work, a human is also inside the loop — approving actions, steering mid-task, reviewing results, deciding when to trust the output. The interface to that human is a harness layer as real as the agent-computer interface of [Chapter 4](#), and it is engineered, not incidental. [Chapter 5](#) introduced one corner of it — permission fatigue. This chapter treats the human interface as a whole.

15.1 The Human Is Inside the Loop

The agent-computer interface chapter argued that as much engineering should go into how an agent uses tools as into how a human uses a screen. The human-agent interface is the mirror: as much engineering should go into how a human supervises an agent as into how the agent acts. Anthropic's three implementation principles already point here — maintain simplicity, *prioritize transparency by showing the agent's planning steps*, and craft the interface carefully ([Anthropic — Building Effective Agents](#)). Transparency is not a UX nicety; it is what makes supervision possible at all.

The premise is older than agents. Horvitz's principles of *mixed-initiative* interfaces, from 1999, already framed the core problem: a system that acts on a user's behalf must decide when to act autonomously and when to defer, must manage the cost of interrupting the user, and must remember and use the context of the interaction ([Horvitz — Principles of Mixed-Initiative User Interfaces](#)). Every one of those questions reappears, sharper, in an agent that can run a shell.

15.2 Two Failure Modes: Permission Fatigue and Blind Trust

The human interface fails in two opposite directions, and a good design has to avoid both at once.

- **Permission fatigue** is the failure of asking too much. When an agent prompts for approval on every action, the human habituates and clicks "allow" without reading, which is worse than no prompt because it manufactures the *appearance* of oversight without the substance ([Anthropic — Beyond Permission Prompts](#), Ch 5). The sandbox exists precisely so that low-stakes actions inside a boundary need no prompt at all.
- **Blind trust** is the failure of asking too little. A fluent, confident output invites the human to rubber-stamp it. When the agent is wrong in a plausible way — a hallucinated citation, a subtly broken refactor — a review surface that does not expose the evidence lets the error through.

These are the two ends of one dial. Turning it toward fewer prompts risks blind trust; toward more prompts risks fatigue. The resolution is not a global setting but *stake-proportionate* interaction: calibrate how much the human is asked to the reversibility and blast radius of the action (§15.3).

15.3 Mixed-Initiative: When to Ask vs Act

The central design question is per action: should the agent do it, or ask first? A workable default scales the answer to consequence:

- **Act silently** inside the sandbox for reversible, low-stakes actions — reading files, running tests, editing a scratch workspace (Ch 5). Asking here only trains the human to stop reading.
- **Act and report** for actions that are consequential but observable and reversible — leave a clear trace the human can review after the fact rather than blocking on approval.
- **Ask first** for irreversible or high-blast-radius actions: deleting data, sending an external message, spending money, touching production. This is exactly the *lethal trifecta* boundary (Ch 5) — the moment an agent is about to combine private data, untrusted input, and external reach is the moment a human should be in the loop.

The map is the same capability-control tradeoff the book returns to in the [Outlook](#): more autonomy is more capability and more control burden. The interface is where that tradeoff is made concrete, action by action.

15.4 Approval as a Tool Call

HumanLayer's twelve-factor manifesto makes the cleanest structural move: *contact humans with tool calls*. Instead of treating human approval as a special control-flow exception, model it as just another tool the agent can invoke — `request_human_approval(action, context)` — whose result comes back into the loop like any other observation ([HumanLayer — 12-Factor Agents](#)).

This unifies human interaction with the rest of the architecture. The approval request is an event in the event log (Ch 9), so it is durable, replayable, and auditable. It composes with the long-running patterns of [Chapter 7](#): an agent can request approval, suspend, and resume hours later when a human responds, because its state lives in the log, not in a

held-open connection. And it makes the human a first-class participant in the trace ([Chapter 12](#)) rather than an out-of-band interruption.

15.5 Designing the Review Surface

When a human does review, the quality of the decision is bounded by what the interface shows. A review surface that presents only the agent's conclusion invites blind trust; one that presents the *evidence behind the conclusion* enables real judgment. For a coding agent this means the diff, the tests that ran, and the commands executed — not just "done." For a research agent it means the sources, not just the summary.

This is the supervision-facing use of the same span telemetry that powers debugging (Ch 12): a good trace is also a good review artifact. The guidance from human-AI interaction research applies directly — make clear what the system can do, make clear how well it did it, and support efficient correction and dismissal of wrong output ([Amershi et al. — Guidelines for Human-AI Interaction](#)). An agent that makes its work cheap to verify and cheap to reject is one a human can actually supervise.

15.6 Steering and Interruption

Supervision is not only before and after; it is also during. A human watching an agent head down a wrong path needs to redirect it without killing the session and losing all state. Steering is a harness capability: the ability to inject a new instruction into a running agent and have it incorporated on the next turn.

The mechanics connect to earlier chapters. Because the agent loop reassembles context each turn (Ch 1), a steering message is simply added to the context for the next iteration — and, by the recitation logic of [Chapter 3](#), placing the correction near the end of context keeps it in the model's most reliable attention span. Interruptibility also depends on clean checkpointing (Ch 7): an agent that can be paused and resumed can be steered; one that holds all its state in a single uninterruptible call cannot.

15.7 Calibrating Trust: Transparency and Uncertainty

The goal of the human interface is *calibrated* trust: the human trusts the agent exactly as much as it deserves on this task. Over-trust produces unreviewed errors; under-trust produces fatigue and wasted human time. Two harness levers move calibration:

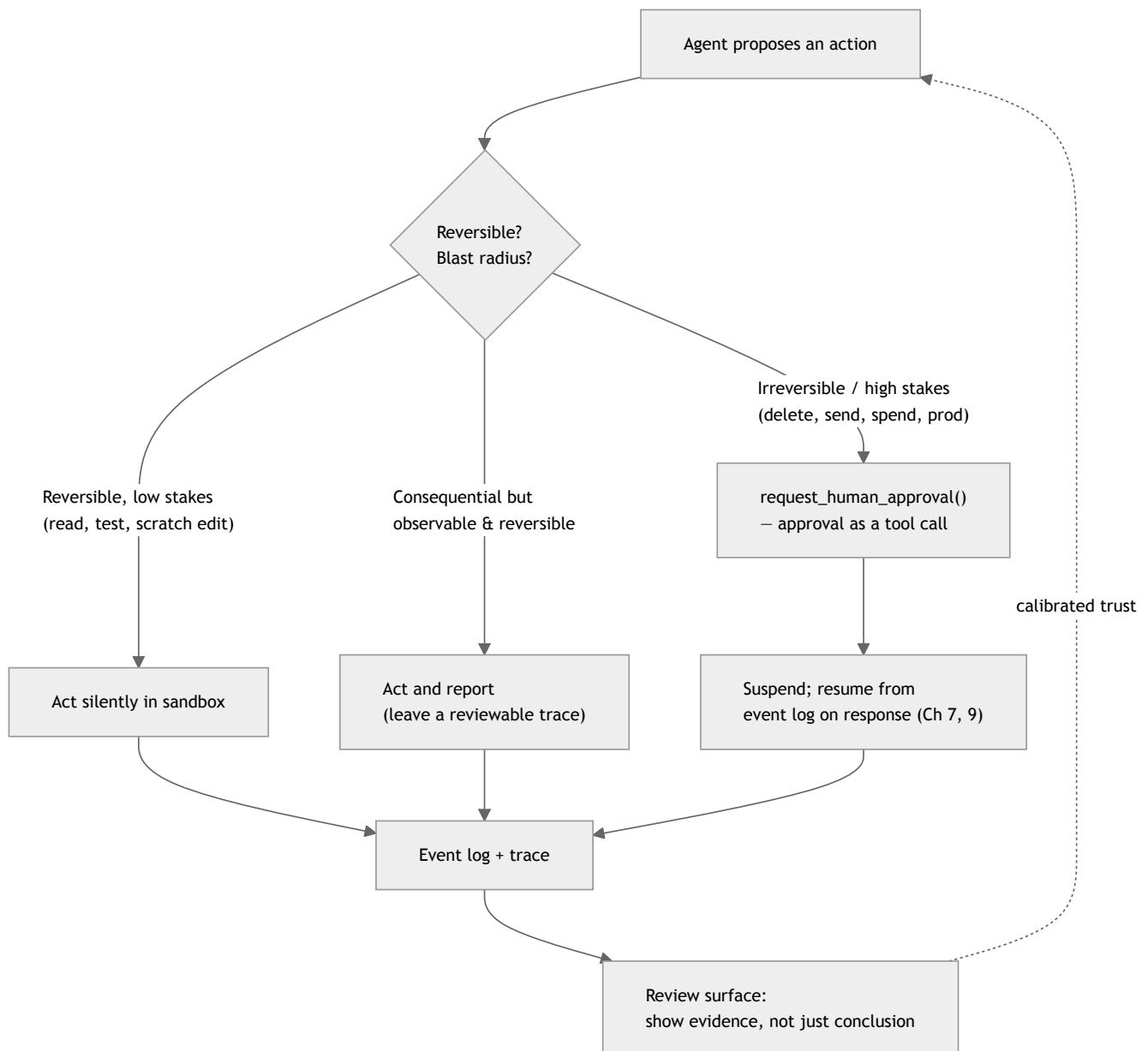
- **Transparency** — showing planning steps, tool calls, and evidence — lets the human's trust track the agent's actual reasoning rather than its fluency ([Anthropic — Building Effective Agents](#)).
- **Uncertainty signaling** — making it clear when the agent is unsure — directs human attention to where it is needed. This depends on the agent's calibration being trustworthy, which the companion volume warns is itself unreliable (*LLM Foundations*, Ch 10); self-reported confidence is a weak signal, so the harness should prefer grounding the uncertainty in verification (did the tests pass? does a cited source exist?) over the model's own hedging.

15.8 Supervising Many Agents

As agents proliferate, the human role shifts from doing the work to supervising agents that do it — and then to supervising *many* agents at once. This is the leverage the field is moving toward, and it changes the interface requirements. One human overseeing ten agents cannot read every trace; the interface must surface what needs attention: which agents are blocked on approval, which are uncertain, which produced results that failed verification.

This raises supervision to a fleet-level concern that the [Outlook](#) flags as open — human approval interfaces and harness coherence at scale are not solved. The principle that does hold: make supervision *cheap and meaningful*. Cheap, so a human can oversee many agents without drowning; meaningful, so the oversight is real and not a rubber stamp. Every technique in this chapter — stake-proportionate prompting, approval-as-tool-call, evidence-rich review surfaces, steering, calibrated transparency — is in service of that one goal.

Diagram: Stake-Proportionate Interaction



The dial runs from silent action to mandatory approval; position each action on it by reversibility and blast radius. Approval is a tool call, and review shows evidence — avoiding both permission fatigue and blind trust.

Key Takeaways

- **The human interface is a harness layer:** as much engineering belongs in how a human supervises an agent as in how the agent acts.
- **Avoid both failure modes:** permission fatigue (asking too much trains rubber-stamping) and blind trust (asking too little lets plausible errors through) are two ends of one dial.
- **Make interaction stake-proportionate:** act silently for reversible low-stakes actions, act-and-report for observable ones, ask first for irreversible or high-blast-radius ones — the lethal-trifecta boundary.
- **Model approval as a tool call:** it becomes durable, replayable, auditable, and composes with long-running suspend/resume.
- **Show evidence, not just conclusions:** an agent whose work is cheap to verify and reject is one a human can actually supervise — at one agent or at a fleet.

Further Reading

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024.
<https://www.anthropic.com/engineering/building-effective-agents>

- Dex Horthy, *12-Factor Agents* (Factor 7: contact humans with tool calls), HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>

Chapter 16: Computer-Use and Multimodal Agents

[Chapter 4](#) built the agent-computer interface out of well-defined tools: functions with schemas, MCP servers, code APIs. A growing class of agents works differently. Instead of calling a clean API, they operate software the way a person does — looking at a screen and moving a cursor, typing into fields, clicking buttons. These *computer-use* agents, and the multimodal perception they depend on, raise harness problems the API-tool chapters do not, and they are the frontier where the most ambitious general-purpose agents now live.

16.1 Beyond the API: Agents That Operate Software

The motivation is reach. Most software has no agent-friendly API; it has a GUI built for humans. An agent that can see a screen and act on it can use anything a person can — legacy desktop apps, web sites without an API, internal tools no one will ever wrap in MCP. Anthropic's computer-use models and OpenAI's Computer-Using Agent both take this route: the model receives screenshots, reasons about them, and emits low-level actions like "click at (x, y)" or "type this string" ([Anthropic — Computer use](#); [OpenAI — Computer-Using Agent](#)).

This is a genuinely different agent loop from the one in [Chapter 1](#). The observation is an image rather than a tool result; the action is a UI gesture rather than a function call; and the environment is an entire operating system or browser rather than a curated tool set. Everything the book has said about tools, context, and safety still applies, but each acquires a visual, lower-level character.

16.2 The Environment Is the Tool Surface

In the ACI chapter, the harness engineer *chose* the tools and could keep them few and high-affordance (Ch 4). A computer-use agent inherits its action surface from the environment: every button, menu, and field on the screen is a potential target. The careful tool curation of [Chapter 4](#) is no longer available the same way, because the "tools" are whatever the application happens to expose visually.

This inverts a key lever. The harness can no longer reduce the action space by offering fewer tools; instead it must help the agent *perceive* the action space accurately and constrain *where* the agent is allowed to act. The design work moves from tool selection to perception and scoping — which is what the next sections are about.

16.3 Encodings of the Screen

A screen can be presented to the model in several ways, and the choice is a harness decision with direct consequences — exactly the multimodal point the companion volume makes about images becoming tokens (*LLM Foundations*, Ch 2). The main encodings:

- **Screenshot pixels** preserve visual layout and styling but are token-heavy and can blur small text. The model must locate elements visually.
- **DOM or HTML** (for web) preserves exact text and structure but loses what is actually visible — hidden, off-screen, and visually-overlapped elements all look the same.
- **Accessibility tree** exposes a structured, semantic view built for assistive tech: roles, labels, and states, often far more compact than raw DOM.
- **OCR** recovers text from pixels but discards structure and spatial relationships.

None is complete. A screenshot shows what a human sees but not the underlying structure; a DOM dump shows structure but not salience. Production systems often *combine* encodings — a screenshot for layout plus an accessibility tree or DOM for exact targets — which is more robust but spends more context. This is the same precision-versus-budget tension as retrieval (Ch 2), now in the visual domain.

16.4 Grounding: From Seeing to Clicking

The hardest problem unique to computer-use agents is *visual grounding*: translating an intention ("click the Submit button") into a concrete action (a click at specific coordinates). A model can correctly identify that a button should be pressed and still emit the wrong pixel location. Grounding error is a failure mode with no analog in API tools, where naming a function is exact.

A common harness technique is *Set-of-Mark prompting*: overlay the screenshot with numbered marks on candidate interactive elements, so the model selects a discrete label ("click element 7") instead of producing raw coordinates ([Yang et al. — Set-of-Mark Prompting](#)). This converts an error-prone continuous grounding problem into a more reliable discrete choice, at the cost of an element-detection step that the harness must provide. The related SeeAct line of work showed that even strong vision models need this kind of grounding scaffold to act reliably on real web

pages — perception and action are separable, and the gap between them is where the harness earns its keep (Zheng et al. — GPT-4V is a Generalist Web Agent).

16.5 The Action Space and Its Failure Modes

Computer-use actions are low-level and stateful in a way function calls are not. A click depends on the screen being in the state the model thinks it is in; a page that loaded slowly, a modal that popped up, or a layout that shifted can invalidate an action the model already decided on. The loop must therefore re-observe after every action — the screen is the new ground truth — and tolerate that observations and actions can desynchronize.

This makes self-verification, the headline lever of [Chapter 7](#), even more central. After acting, the agent should check that the screen changed as expected before proceeding. It also makes recovery harder: undoing a GUI action is rarely as clean as reversing an API call, so the stake-proportionate interaction of [Chapter 15](#) matters more — a computer-use agent about to confirm an irreversible dialog is exactly where a human checkpoint belongs.

16.6 Latency, Cost, and the Token Weight of Pixels

Every turn of a computer-use loop sends an image into context, and images are expensive in tokens — a single high-resolution screenshot can cost as much as a page of text (*LLM Foundations*, Ch 2). A task that takes a human thirty clicks is thirty screenshots through the context window, which interacts badly with the finite-context budget of [Chapter 2](#) and the prefill-skew of agentic workloads.

The harness levers are the familiar ones, sharpened. Downscale screenshots as far as the task tolerates — but no further, since shrinking can erase the very text the agent needs to read. Avoid retaining old screenshots in context after their information is extracted; keep a compact note of what a prior screen showed rather than the pixels (the *keep the wrong stuff out* discipline of [Chapter 3](#) applies to images most of all). And prefer a structured encoding (accessibility tree) over pixels when it suffices, reserving the expensive screenshot for when layout truly matters.

16.7 Safety: The Widest Attack Surface

A computer-use agent is the *lethal trifecta* (Ch 5) in its most concentrated form. It reads untrusted content (any web page or document on screen), it often has access to private data (whatever is open in the browser or filesystem), and it can communicate externally (it can navigate, submit forms, send messages). Prompt injection here is not hypothetical text in a tool result; it is a malicious instruction rendered on a web page the agent is looking at, which the model may read as a command.

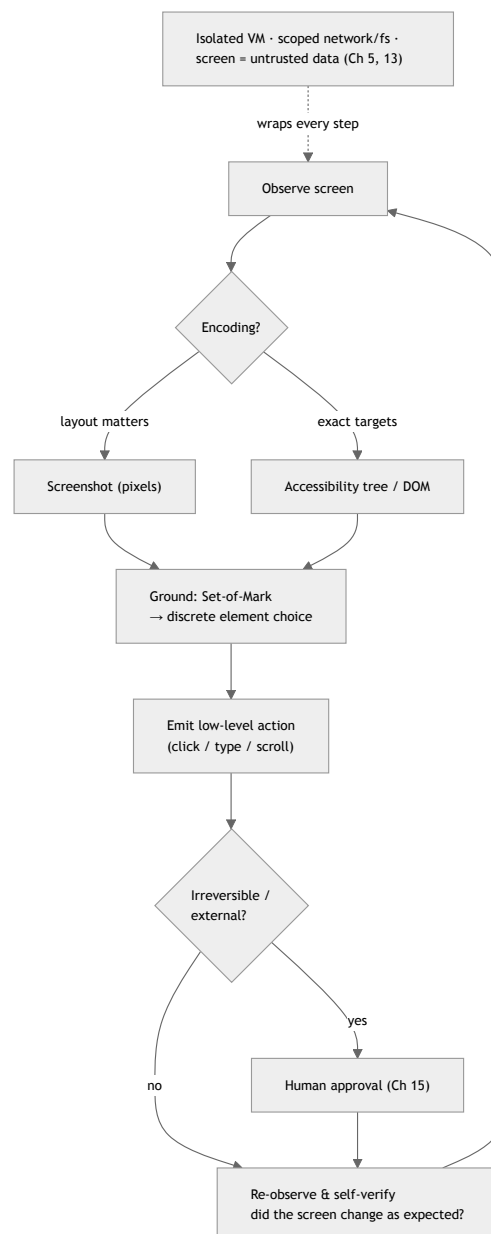
The defenses are the sandboxing and governance of [Chapter 5](#), made stricter because the action surface is so broad. Run the agent in an isolated environment — a dedicated VM or container, not the user's primary session. Scope network and filesystem access. Require human approval before irreversible or external actions (Ch 15). And treat everything on the screen as untrusted data, never as instruction — the instruction-hierarchy discipline of [Chapter 13](#) is load-bearing when the "data" is a full rendered web page.

16.8 Evaluating Computer-Use Agents

Because the environment is an entire OS or browser, evaluation must be environmental, not text-based — the *outcome* distinction of [Chapter 10](#) is unavoidable here, since the only thing that matters is whether the task was actually accomplished in the environment. Realistic benchmarks built for this provide the model: WebArena offers a self-hostable, realistic web environment with execution-based success checks ([WebArena](#)), and OSWorld extends the idea to open-ended tasks across a real desktop operating system ([OSWorld](#)).

The lesson from these benchmarks reinforces the whole book: computer-use agents are far from saturated on realistic tasks, and the gap between "the model can perceive the screen" and "the agent reliably completes the task" is exactly the harness gap — grounding scaffolds, re-observation, recovery, scoping, and verification. As always, the right eval is the workload eval (Ch 10): a public computer-use benchmark is background signal; whether the agent can drive *your* application reliably is the release signal.

Diagram: The Computer-Use Loop



Perceive → ground → act → re-observe, wrapped in a sandbox. Grounding turns "see" into "click"; re-observation handles a stateful screen; the sandbox contains the widest attack surface in the book.

Key Takeaways

- **Computer-use agents trade clean APIs for reach:** they operate GUIs the way humans do, so they can use software that has no agent API — at the cost of a harder loop.
- **The environment is the action surface:** tool curation gives way to perception and scoping; the harness helps the agent see the action space accurately and constrains where it may act.
- **Screen encoding is a harness choice:** pixels, DOM, accessibility tree, and OCR each preserve different things; combining them is more robust but costs context.
- **Grounding is the unique failure mode:** translating intent into a correct click is error-prone; discrete techniques like Set-of-Mark make it more reliable than raw coordinates.
- **Widest attack surface, strictest sandbox:** a computer-use agent is the lethal trifecta concentrated — isolate it, scope it, gate irreversible actions, and treat the whole screen as untrusted data.

Further Reading

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024.
<https://www.anthropic.com/news/3-5-models-and-computer-use>

- OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
- Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
- Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
- Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
- Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>

Chapter 17: Cost, Privacy, and Production Operations

The chapters so far have built an agent that works. Running it in production for real users adds three concerns that the agent loop itself hides: what each run costs, what it does with *sensitive data*, and how the harness is *changed safely* once people depend on it. None is glamorous, and each is where a working demo quietly fails to become a working product. [Chapter 5](#) covered the security threat model and [Chapter 11](#) covered measurement noise; this chapter covers the operational economics, data governance, and release discipline around them.

17.1 Cost: Budgeting the Agent

An agent is the most expensive way to run a model, because a single user request expands into many model calls, tool round-trips, and — with reasoning models — long internal token streams ([Chapter 14](#)). Worse, the cost of a task is not known in advance: an open-ended loop can take three turns or thirty. This is the operational reason the book keeps returning to "do the simplest thing that works" ([Chapter 6](#)) — every agentic turn is a recurring charge, not a one-time one.

A production agent therefore needs an explicit *budget*, the way [Chapter 7](#) needed a checkpoint. Concretely:

- **A per-task ceiling** on tokens, tool calls, or wall-clock cost, after which the agent stops and asks rather than looping indefinitely. An unbounded loop is a runaway bill.
- **The cost levers already established:** route easy steps to cheap models and reserve frontier and reasoning models for steps that need them (Ch 14); cache stable prefixes so repeated context is not repaid every turn (Ch 2); return token-efficient tool responses and use code execution to avoid dumping large results into context (Ch 4).
- **A build-or-not decision:** the cheapest agent is the one not run. Many tasks are better served by a workflow or a single call, and the cost analysis is part of choosing (Ch 6).

The cascade and routing patterns of [Chapter 14](#) are cost controls as much as quality controls; FrugalGPT's central result is that most of the spend on a naive "always call the best model" strategy is avoidable without losing accuracy ([FrugalGPT](#)).

Caching deserves a second look here because there are two distinct kinds. The KV/prefix cache of [Chapter 2](#) serves *identical* prefixes cheaply; a *semantic cache* serves *similar* requests by embedding the incoming query and returning a stored response when a prior query is close enough, as popularized by GPTCache ([Fu Bang — GPTCache](#)). For high-volume, repetitive queries this can eliminate whole model calls rather than just cheapening them — but it introduces a correctness risk the prefix cache does not: a false hit returns a subtly wrong answer for a question that only *looked* like a cached one. A semantic cache is therefore a tuned component — the similarity threshold trades savings against that risk — and belongs behind the same eval discipline as any other harness change (§17.5). Both cache types, along with the per-key budgets above, are naturally enforced at the *AI gateway* (§14.8), which sits on the path of every call and is the single place cost controls can be applied uniformly across agents.

17.2 Cost Attribution and the Trace

A budget is only enforceable if cost is *measured*, and the place it is measured is the trace. The span telemetry of [Chapter 12](#) — a tree of spans for model calls, tool calls, and retrieval — is also the cost ledger: attach token counts and dollar cost to each span and the trace tells you not just what the agent did but what each step cost. This turns "the agent is expensive" into "this tool returns 8,000-token results on every call," which is an engineering task rather than a complaint.

Cost attribution also feeds the same iteration loop as failures. A step that is both expensive and low-value is a candidate for a cheaper model, a tighter tool response, or removal — the trace-driven iteration of [Chapter 12](#) applied to the cost axis instead of the correctness axis. As [Chapter 11](#) warned, cost and resource configuration also interact with measured behavior, so cost should be reported alongside capability, not separately.

17.3 Privacy and Data Governance

An agent touches data on every run — reading files, querying databases, retrieving documents, sending requests to model providers and tools. Each of those is a place where sensitive data can leak, and the harness owns the controls. The threat is sharpest as the *lethal trifecta* of [Chapter 5](#): an agent with access to private data, exposure to untrusted content, and a path to communicate externally can be induced to exfiltrate that data ([Simon Willison — The lethal trifecta](#)). Privacy is not a separate concern from security; it is the data-handling face of it.

The harness-level controls:

- **Minimize what enters context.** Data the agent never sees cannot leak. Retrieve the slice needed for the task, not the whole record (Ch 2), and prefer handles over raw sensitive content.
- **Redact at the boundary.** Strip secrets and PII before content crosses into the model context or into a trace — traces are durable and often shipped to third-party observability tools, so an un-redacted trace is its own leak (Ch 12).
- **Respect permission at retrieval time.** Filter by what the *user* is allowed to see before retrieval, not after generation; an agent must not become a way to read data its user could not (Ch 5, Ch 12).
- **Govern egress.** The external-communication leg of the trifecta is the one the harness can cut: constrain where the agent can send data, and require approval for external sends (Ch 15).

17.4 Multi-Tenancy and Isolation

When one agent platform serves many users or organizations, isolation becomes a correctness property, not just a nicety. Two failure modes matter. *State bleed* — one tenant's context, memory, or cached prefix leaking into another's session — is catastrophic and easy to introduce when memory (Ch 3) and prefix caching (Ch 2) are added without tenant scoping. *Authority bleed* — an agent acting with one tenant's credentials on another's behalf — is the multi-tenant version of the delegated-auth concern from Chapter 5.

The discipline follows the managed-agent architecture of Chapter 7: durable per-tenant workspaces, sandboxes scoped to a single tenant's resources, and credentials that are scoped and never shared across the brain/hands boundary. The event log (Ch 9) should carry tenant identity so that every action is attributable, which is also what makes audit (Ch 5) meaningful in a shared system.

17.5 Releasing Harness Changes

A harness is software, and changing software that people depend on needs release discipline — yet harness changes are unusually risky because their effect is statistical, not deterministic. A prompt edit, a new tool, a model swap, or a retrieval tweak can shift behavior on inputs no one tested, and the regression may be a few percent of cases rather than a crash (Chapter 10, Chapter 12).

The release practices that follow:

- **Gate every change on evals.** Run the suite before and after, comparing pass rate, failure categories, cost, and latency — the regression discipline of Chapter 10 is the release gate.
- **Roll out gradually.** Because eval suites never cover the full input distribution, treat a harness change like any risky deploy: release to a fraction of traffic, watch production traces and outcome metrics, and be ready to roll back. Canarying is how the cases the eval missed are caught cheaply.
- **Version the whole configuration together.** The model-harness coupling of Chapter 12 means the unit of release is the *configuration* — model, prompts, tools, retrieval policy — not any one part. Readiness validation (Ch 10) is per configuration, and rollback must restore the configuration, not just the prompt.

17.6 Monitoring, SLOs, and Incident Response

Evals run before release; monitoring runs after. The two are complementary: evals measure a curated distribution, production tells you about the real one. The span telemetry of Chapter 12 is the monitoring substrate, and the operational additions are the ones any production system needs, specialized for agents:

- **Outcome metrics, not just uptime.** A liveness check that the API responds says nothing about whether tasks are succeeding. Track task success rate, escalation/approval rate, cost per task, and latency percentiles (p50/p95/p99, as in Ch 6) as first-class signals.
- **Alert on drift.** A rising failure rate, a climbing cost-per-task, or a falling self-verification pass rate is an incident even when nothing has crashed — these are the agent-specific failure shapes.
- **Production failures become regression cases.** The trace-to-eval loop of Chapter 12 is also the incident-response loop: a production failure is redacted, turned into a reproducible eval case, and engineered out permanently so the same incident cannot recur silently.

17.7 Governance Frameworks

These concerns are not only internal engineering; for many deployments they are external requirements. Four reference frameworks help structure the work without reinventing it, spanning voluntary guidance to binding law.

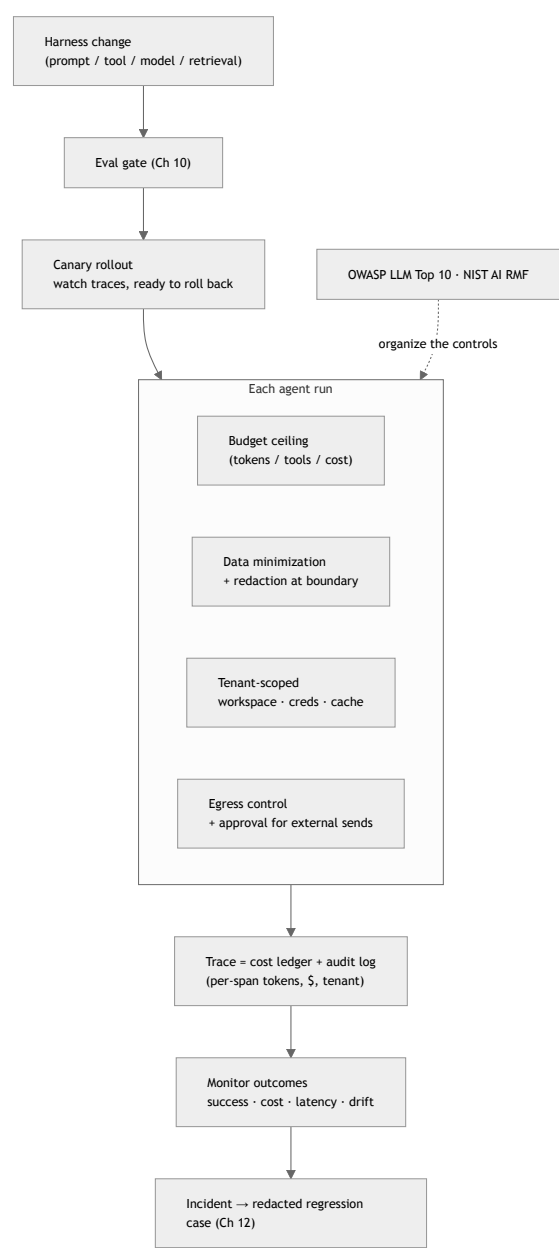
- **OWASP Top 10 for LLM Applications** enumerates the common risk classes — prompt injection, sensitive-information disclosure, excessive agency, supply-chain risks — and maps cleanly onto the controls of Chapter 5

and this chapter ([OWASP — Top 10 for LLM Applications](#)).

- **NIST AI Risk Management Framework** offers a higher-level structure — govern, map, measure, manage — for organizations that need an auditable risk process around an agent deployment ([NIST — AI Risk Management Framework](#)).
- **ISO/IEC 42001:2023**, the first AI management-system standard, is the AI analogue of ISO 27001: it specifies how an organization establishes, operates, and continually improves a management system for its AI, and it is certifiable ([ISO/IEC 42001:2023](#)).
- **The EU AI Act** (Regulation (EU) 2024/1689) is the first comprehensive AI law. It classifies systems by risk tier and imposes binding obligations — risk management, data governance, transparency, human oversight, logging — on high-risk uses, with provisions phasing in through 2026–2027 ([EU AI Act — Regulation \(EU\) 2024/1689](#)). Note that the two are complementary but distinct: ISO 42001 certification is evidence of good process, not automatic proof of AI Act compliance.

None of these replaces the engineering in this book; they organize it and make it legible to auditors, customers, and regulators. The [Outlook](#) flags cross-layer governance coherence as an open problem precisely because policy, audit, and runtime enforcement still live in separate layers — these frameworks are the current best scaffolding for keeping them aligned.

Diagram: The Production Wrapper



Cost, privacy, and tenancy are enforced per run; the trace is both the cost ledger and the audit log; every harness change passes an eval gate and a canary before reaching production.

Key Takeaways

- **Agents need explicit budgets:** an open-ended loop is a runaway bill; cap tokens/tools/cost per task, and use routing, cascades, caching, and token-efficient tools to control spend ([FrugalGPT](#)).
- **The trace is the cost ledger:** attach per-span tokens and dollars so "the agent is expensive" becomes a specific, fixable engineering task.
- **Privacy is the data face of the lethal trifecta:** minimize what enters context, redact at the boundary (traces leak too), filter retrieval by user permission, and control egress.
- **Multi-tenancy demands isolation:** scope workspaces, credentials, memory, and prefix caches per tenant, and carry tenant identity in the event log — state bleed and authority bleed are catastrophic.
- **Two caches, two risk profiles:** the prefix cache serves identical context cheaply; a semantic cache serves *similar* queries and can skip whole calls, but a false hit returns a wrong answer — tune its threshold and eval it like any harness change.
- **Releasing a harness change is a statistical deploy:** gate on evals, canary gradually, version the whole configuration together, and turn production failures into regression cases.
- **Governance frameworks span guidance to law:** OWASP LLM Top 10 and NIST AI RMF organize the controls; ISO/IEC 42001 certifies the process; the EU AI Act makes obligations binding for high-risk uses.

Further Reading

- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Simon Willison, *The lethal trifecta for AI agents*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
- OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
- NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
- *ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
- *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>

Chapter 18: Outlook

18.1 The Field Is Young

Much of the vocabulary used in this book — initializer agents, context firewalls, sprint contracts, reasoning sandwiches, ambient affordances, computational vs. inferential controls — entered the mainstream agent-engineering conversation within the last twelve to eighteen months. Some underlying ideas are older, but the shared language is recent. Most of the source articles for this textbook were published in 2025 and 2026. The field is moving faster than any single book can document — and there is a number attached to that pace: METR finds the task-completion *time horizon* of frontier models (the human-task-length they clear 50% of the time) has doubled roughly every seven months (§7.11) ([Kwa et al. — Measuring AI Ability to Complete Long Tasks](#)). A book about what a harness must supply is, in part, a book about a moving target.

LangChain frames the trajectory honestly: as models improve, some of what lives in the harness today will be absorbed into the model. Models will get better at planning, self-verification, and long-horizon coherence natively, requiring less context injection. But the space of interesting harness combinations does not shrink as models improve. It moves ([LangChain — The Anatomy of an Agent Harness](#); [Anthropic — Harness Design for Long-Running Application Development](#)).

18.2 Open Problems

Several recur across the literature:

- **Behavioral harnesses for functional correctness.** Maintainability and architecture-fitness harnesses have decades of pre-existing tooling. Behavior harnesses — does the application functionally do what the user wants? — do not. Today most teams rely on AI-generated tests, and the consensus is that this is not yet good enough ([Thoughtworks — Harness Engineering](#)).
- **Harness coherence at scale.** As guides and sensors multiply, how do they stay consistent? How do we know when sensors that never fire indicate quality versus inadequate detection? There is no equivalent of code coverage or mutation testing for harness coverage yet.
- **Cross-layer governance coherence.** The OpenReview survey highlights that policy, permission prompts, audit logs, constitutional instructions, and runtime hooks often live in separate layers and may interfere rather than compose. Portable policy and audit languages are still missing ([OpenReview — Agent Harness Engineering: A Survey](#)).
- **Standardized readiness reporting.** If model scores depend on execution environment, tool surface, context policy, retry rules, and governance gates, benchmark reports need a harness bill of materials. Today there is no widely adopted format for publishing that configuration ([OpenReview — Agent Harness Engineering: A Survey](#)).
- **Cost-quality-speed trilemma.** Stronger sandboxes, richer observability, deeper verification, and stricter governance usually increase cost and latency. Mature harnesses need explicit policy for which checks run synchronously, which run offline, and which risks justify expensive controls ([OpenReview — Agent Harness Engineering: A Survey](#)).
- **Capability-control tradeoff.** More tools, memory, autonomy, and network reach increase task coverage, but also increase selection errors, prompt-injection surface, provenance risk, and audit burden. Capability and control are one design axis, not separate concerns.
- **Multi-agent coordination beyond synchronous orchestration.** Anthropic's research system runs sub-agents synchronously; asynchronous coordination would unlock more parallelism but adds challenges in result coordination, state consistency, and error propagation ([Anthropic — How We Built Our Multi-Agent Research System](#)).
- **Continual learning at the harness level.** Memory primitives that let agents accumulate knowledge of a codebase or domain over many sessions, rather than starting fresh each time, are an active research area ([LangChain — Improving Deep Agents with Harness Engineering](#)).
- **Human-in-the-loop approval.** Where the harness should pause for a human decision — and how to surface enough context for that decision without flooding the operator — has no settled interface. Approval gates that are too coarse get rubber-stamped; too fine, and they defeat the point of autonomy.
- **Just-in-time tool assembly.** Harnesses that dynamically assemble the right tools and context for a given task, rather than pre-configuring everything, are explored by LangChain among others ([LangChain — Improving Deep Agents with Harness Engineering](#)).

- **Tracing as documentation.** LangChain's observation that "in software, the code documents the app; in AI, the traces do" hints at a different model of system documentation that the field has not fully worked out ([LangChain — The Anatomy of an Agent Harness](#)).
- **End-to-end supply-chain governance.** Tool integrity is only one piece. Agents also depend on MCP servers, external packages, datasets, retrieval sources, and generated dependency names. Provenance across that full chain is still underdeveloped ([OpenReview — Agent Harness Engineering: A Survey](#)).

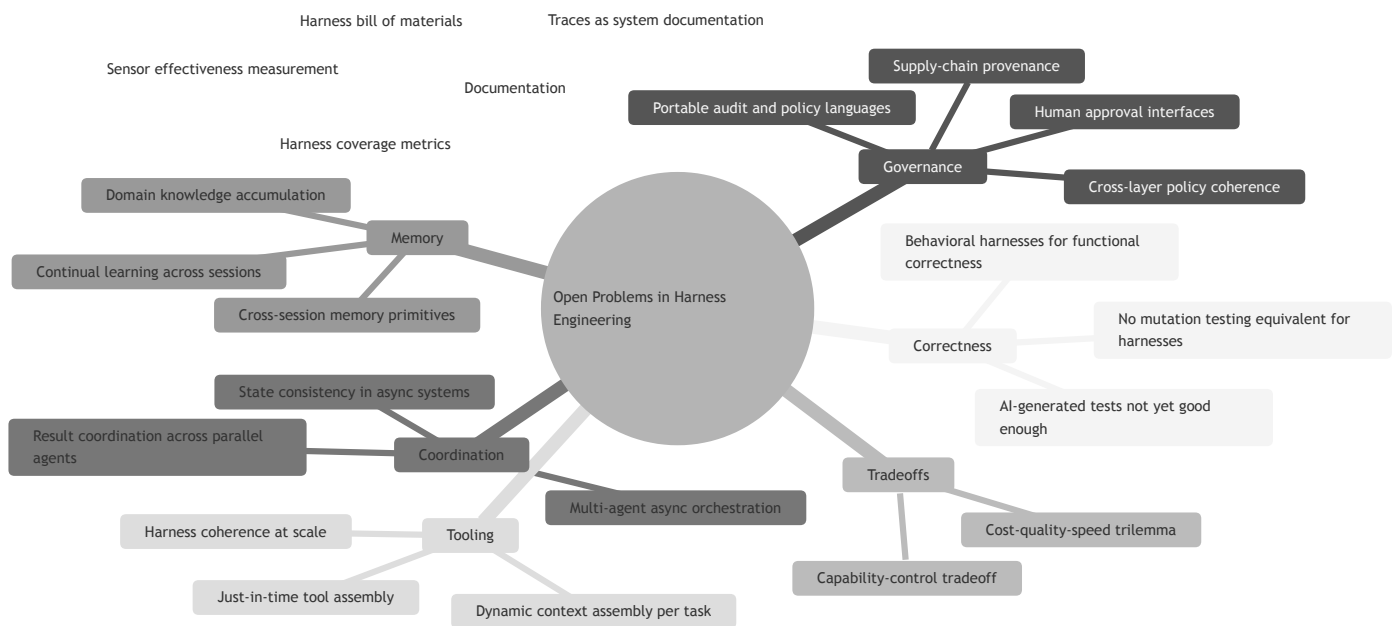
18.3 The Standing Advice

A few principles repeat across nearly every article in the corpus:

- **Treat context as a finite resource.** Find the smallest set of high-signal tokens that produces the desired outcome.
- **Do the simplest thing that works.** Agents are expensive; workflows often suffice; many tasks need neither.
- **Read the transcripts.** Everything else flows from this.
- **Iterate on what is load-bearing.** Stress-test components when models change. Strip what is no longer pulling weight; tune what is.
- **Tailor harnesses to models, but tailor principles to the field.** Specific prompts and tools change between models. The shape of the work — context engineering, tool design, evaluation, sandboxing, self-verification — does not.

The field is not yet old enough to have textbooks. This document is an attempt to write one anyway, knowing it will go out of date faster than most. The hope is that the citations make it possible to read the originals as the conversation continues.

Diagram: Open Problems Mindmap



Key Takeaways

- **The shared vocabulary is young:** many terms became common in 2025–2026, even when the underlying ideas are older.
- **Models absorb harness, but harness moves:** as models improve and take on more native capabilities, the interesting harness work moves to harder problems, not away.
- **Open problems now span the full ETCLOVG stack:** behavioral correctness, harness coherence, governance coherence, readiness reporting, cost-quality-speed, capability-control, async coordination, continual learning, just-in-time tool assembly, and traces-as-documentation.
- **Five standing principles cut across all contexts:** finite context, simplest-that-works, read-the-transcripts, iterate-load-bearing, tailor-to-model.

- **Harness engineering is permanent work:** not scaffolding to discard once models improve, but the ongoing craft of building effective systems around increasingly capable cores.

Further Reading

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>
- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

References

Full bibliography for *Agent Harness: A Practitioner's Textbook*.

Foundations (prerequisite)

- *LLM Foundations for Harness Engineering* — the prerequisite stage-1 volume this book builds on. [../llm-foundations/](https://llm-foundations/)
-

Foundations

- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
 - *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
 - Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
 - Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
 - Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
 - Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
 - Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022. <https://arxiv.org/abs/2210.03629>
 - Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
-

Context, Memory, and Working State

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 - Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
 - Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
 - Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
 - Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
-

Tools and Constraints

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>

- Simon Willison, *The lethal trifecta for AI agents: private data, untrusted content, and external communication*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
 - *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
 - Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
-

Specs and Workflow Design

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
-

Reasoning and Planning Patterns

(See also *ReAct* under Foundations, the base reasoning-and-acting pattern.)

- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
 - Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
 - Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
 - Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
 - Binfeng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>
 - Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
-

Multi-Agent Systems

- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (introducing the MAST failure taxonomy), UC Berkeley / arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
-

Reliability and Operational Safety

- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
 - LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
 - Martin Fowler, *CircuitBreaker*, martinowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
 - Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
-

Agent Capability Measurement

- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
-

Loop Engineering

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026. <https://addyosmani.com/blog/loop-engineering/>
- *Loop Engineering*, O'Reilly Radar, 2026. <https://www.oreilly.com/radar/loop-engineering/>
- Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026. <https://www.deeplearning.ai/the-batch/>

- *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026. <https://thenewstack.io/loop-engineering/>
 - *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026. <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
 - *Loop Engineering Guide* (2026), AI Builder Club. <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
 - *Loop Engineering Crash Course*, The AI Agent Factory (Panaversity). <https://agentfactory.panaversity.org/docs/loop-engineering-crash-course>
-

Evaluation and Observability

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
 - Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
 - Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
 - OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
-

Runtimes, Harnesses, and Reference Implementations

- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses, Oh My!*, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
 - Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
-

Instructions and Prompting

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
-

Model Selection, Routing, and Reasoning

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
 - Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
 - Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
 - LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
 - Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>
-

Human-Agent Interaction

- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
 - Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
-

Computer-Use and Multimodal Agents

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024. <https://www.anthropic.com/news/3-5-models-and-computer-use>
 - OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
 - Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
 - Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
 - Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
 - Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>
-

Cost, Privacy, and Governance Frameworks

- OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
 - NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
 - ISO/IEC 42001:2023 — *Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
 - *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
 - Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
-

Companion Volume

- *LLM Foundations for Harness Engineering*. The model-internals companion to this book. [../llm-foundations/](https://llm-foundations.com/)
-

Reading List

- *Awesome Harness Engineering*, walkinglabs. <https://github.com/walkinglabs/awesome-harness-engineering>
- *Learn Harness Engineering*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/>
- *Learn Harness Engineering — Skills*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/skills/>

Glossary

Concise definitions for terms used throughout this textbook. The chapter reference points to where each term is discussed in depth; for the source behind a definition, follow that chapter's inline citations and the [References](#).

Core Concepts

Agent — A language model plus the system that lets it do real work: browse a codebase, run code, call tools, recover from errors, and sustain multi-step tasks. Captured by the equation $Agent = Model + Harness$ (Ch 1).

Agent harness — Everything engineered around the model: system prompts, tools and their descriptions, bundled infrastructure, sub-agent orchestration, control flow, hooks/middleware, and evaluation infrastructure (Ch 1).

Agent loop — The core execution cycle: assemble context → model emits a tool call or a final answer → harness executes the call → append the result to context → repeat until done (Ch 1).

Augmented LLM — A model equipped with retrieval, tools, and memory, able to generate its own queries, select tools, and decide what to retain. The basic building block of an agent (Ch 1, 6).

ACI (agent-computer interface) — The design surface between an agent and its tools, by analogy with HCI: as much engineering should go into how an agent uses tools as into how a human uses a screen (Ch 4).

Builder harness — The system prompt and tools shipped by an AI lab as part of a coding-agent product; the middle of the three concentric harness rings (Ch 1).

User harness — The AGENTS.md files, hooks, skills, and review agents a team adds on top of a coding agent to fit its own codebase; the outer ring (Ch 1).

Harness engineering — Iterating on the whole system around the model — not just one prompt — so that each observed failure is permanently engineered out (Ch 1).

Binding-constraint thesis — The claim that long-horizon agent reliability is often limited by harness layers — execution, tools, context, lifecycle, observability, verification, and governance — rather than by model capability alone (Ch 1, 12).

ETCLOVG — A seven-layer taxonomy for agent harness engineering: Execution environment, Tool interface, Context, Lifecycle, Observability, Verification, and Governance (Ch 1).

Context engineering — Curating the smallest set of high-signal tokens in the context window during inference; the practice one level below harness engineering (Ch 1, 2).

MCP (Model Context Protocol) — An open client-server standard for exposing tools, resources, and prompts to agents, so any compatible client can discover and call them without bespoke integration (Ch 4).

A2A (Agent-to-Agent protocol) — A protocol boundary for delegation among opaque agentic applications; complementary to MCP, which primarily exposes tools and context to one agent runtime (Ch 4).

Protocol boundary — The integration line a tool or agent standard crosses: model-to-function, agent-to-external-capability, agent-to-agent, or agent-to-repo/environment (Ch 4).

Tool call — Structured output (typically JSON) in which the model names a tool and its arguments. Deterministic harness code decides what to do with it (see Foundations ch 12) (Ch 4, 9).

Structured output — Model output constrained into a machine-readable shape, usually JSON or XML, so software can parse it reliably. Tool calls are the agent-specific case (see Foundations ch 12) (Ch 1, 4, 9).

Context and Memory

Context window — The finite span of tokens a model can attend to in a single inference call (see Foundations ch 9). In a harness it is a budget every system prompt, tool result, and history turn competes for (Ch 2).

Context rot — The degradation of a model's ability to recall and use information accurately as the context grows longer (see Foundations ch 9). The harness angle: it is the binding operating constraint, framed here as an attention budget (Ch 2).

Attention budget — The framing of context as a finite resource that every added token spends (Ch 2).

KV-cache — A cache of the key/value tensors for already-processed tokens (see Foundations ch 9). Identical context prefixes can be served from it, cutting time-to-first-token and cost roughly tenfold; in a harness, prefix stability becomes a production cost lever (Ch 2).

Prefill / decode — Prefill is processing the input prompt; decode is generating output tokens (see Foundations ch 9). Agentic workloads are heavily prefill-skewed (~100:1 input-to-output) (Ch 2).

Lost-in-the-middle — The tendency of models to attend less reliably to information in the middle of a long context than to its start or end (see Foundations ch 9) (Ch 2, 3).

Compaction — Summarizing a conversation near its context limit and reinitiating a fresh window with the summary. Lossy (see Foundations ch 9) (Ch 3).

Context reset — Clearing the context entirely and starting a fresh agent with a structured handoff — distinct from in-place compaction (Ch 7).

Just-in-time retrieval — Loading data into context on demand via lightweight references (file paths, queries, links) rather than pre-embedding everything up front (Ch 2).

Recitation — Repeatedly rewriting a goal or todo list into the end of context so it stays in the model's recent attention span (Ch 3).

Structured note-taking (agentic memory) — Having the agent write progress notes to disk so they can be reloaded after a context reset (Ch 3).

MemGPT — A memory architecture that treats the context window as OS-style "main memory" and external stores as "disk," letting the model page information in and out via function calls (virtual context management) (Ch 3).

Mem0 — A memory layer that dynamically extracts, consolidates, and retrieves salient facts across sessions, with an optional graph variant for entity relations (Ch 3).

Sleep-time compute — Processing context offline between requests — anticipating likely queries and precomputing inferences — to cut the compute a later query needs (Ch 3).

Sub-Agents and Workflows

Sub-agent — A specialized agent that handles a focused task in its own context window and returns only a condensed summary to its parent (Ch 3).

Context firewall — The property of the sub-agent pattern by which the parent never sees a sub-agent's intermediate noise, only its condensed result (Ch 3).

Orchestrator-workers — A workflow where a central LLM dynamically decomposes a task, delegates to worker LLMs, and synthesizes their results (Ch 6).

Evaluator-optimizer — A workflow where one LLM generates and another critiques, in a loop, until evaluation criteria are met (Ch 6).

Micro-agent — A small, focused agent (≈3–20 steps) embedded in an otherwise deterministic workflow, rather than an open-ended "loop until done" agent (Ch 6, 9).

Multi-agent topology — The coordination shape of a multi-agent system: orchestrator-worker, hierarchical, blackboard/shared-memory, or debate/voting (Ch 3, 6).

MAST (Multi-Agent System failure taxonomy) — An empirical taxonomy of 14 multi-agent failure modes in three categories: specification issues, inter-agent misalignment, and task verification (Ch 3).

Reasoning / self-correction patterns — Single-agent deliberation patterns that trade tokens for reliability: Reflexion (self-critique memory), Self-Refine (critique-and-revise), CRITIC (tool-grounded critique), Tree of Thoughts and LATS (branching search), and ReWOO (plan-then-execute) (Ch 6).

Tools and Sandboxing

Namespacing — Grouping related tools under common prefixes (`asana_*` , `browser_*`) to prevent name collisions and enable group-level masking (Ch 4).

Action masking — Keeping the full tool set stable in context while constraining which actions can be selected in a given state (Ch 2).

Progressive disclosure — Loading tool definitions, files, or instructions only when needed, rather than all up front (Ch 4).

Skill — A reusable, file-backed capability (often a `SKILL.md` plus supporting code) that an agent can load on demand (Ch 4).

Code execution (as a meta-tool) — Presenting tools as a code API the agent invokes by writing code, rather than as direct calls — sharply reducing token cost (Ch 4).

Shell — A command-line interface such as Bash or zsh. In agent systems, shell access is powerful because it lets the agent run tests, inspect files, install packages, and compose ad hoc tools (Ch 1, 4, 5).

Filesystem — The directories and files the agent can read or write. It acts as workspace, durable memory, and a collaboration surface between agents and humans (Ch 1, 2, 5).

Sandbox — An isolated environment, with filesystem and network boundaries, within which an agent can act freely without per-action approval prompts (Ch 5).

Sandbox liveness — The sandbox's role as an authorization region: it lets an agent act without per-action approval prompts while staying inside configured boundaries (Ch 5).

Governance — Harness mechanisms for identity, permission policy, scoped credentials, human approval, audit logs, and cross-layer security accountability (Ch 5, 18).

Delegated auth — A pattern where the agent acts through scoped credentials or a proxy-authorized identity rather than inheriting the user's full ambient authority (Ch 5).

Supply-chain provenance — Evidence about the origin and integrity of tools, packages, datasets, MCP servers, and retrieval sources the agent depends on (Ch 5, 18).

Hook / middleware — Harness-executed scripts or checkpoints that run automatically on lifecycle events (start, post-tool-call, stop), enforcing rules deterministically (Ch 5).

Feedforward / feedback — Feedforward controls (guides) steer the agent before it acts; feedback controls (sensors) observe after it acts and help it self-correct (Ch 5).

Computational / inferential control — Computational controls (linters, type checkers) are deterministic and fast; inferential controls (AI review, LLM-as-judge) handle nuance but are slower and non-deterministic (Ch 5).

Ambient affordances — Properties of the environment itself (strong typing, clear module boundaries, opinionated frameworks) that make a codebase legible and tractable to agents (Ch 5).

CI (continuous integration) — Automated checks that run around code changes, usually tests, linters, builds, and deployment gates. In harness design, CI-like checks become feedback sensors (Ch 5, 10).

Linter / type checker — Deterministic tools that detect style, syntax, structural, or type errors before runtime. They are common computational sensors in an outer harness (Ch 5).

Prompt injection — An attack in which instructions hidden in content the agent reads (a web page, a file, a tool result) are interpreted by the model as commands (see Foundations ch 8, ch 12) (Ch 5).

Lethal trifecta — The dangerous combination, in one agent, of access to private data, exposure to untrusted content, and the ability to communicate externally (Ch 5).

Circuit breaker — A reliability wrapper that trips after a threshold of failures so further calls to a failing tool, service, or sub-agent fail fast instead of hanging or retry-storming (Ch 5).

Kill switch — A human- or policy-triggered stop that halts an agent or fleet immediately, independent of the agent's own control flow; it lives in the harness because a steered agent cannot be trusted to stop itself (Ch 5).

Canary token — A planted fake secret (unused key, decoy file, tripwire URL) whose access or exfiltration raises a high-signal alarm that an agent has been steered — detection for the lethal-trifecta exfiltration path (Ch 5).

Action budget — A hard ceiling on tool calls, tokens, wall-clock time, or spend, after which the loop stops and escalates rather than running away (Ch 5, 8, 17).

Evaluation

Eval harness — The infrastructure that runs an evaluation end-to-end; distinct from the agent harness being evaluated (Ch 10).

Readiness validation — Verification that a specific model-plus-harness configuration is ready for a specific task distribution, environment, budget, and governance regime (Ch 10).

Failure attribution — Labeling an agent failure by likely harness layer — for example execution, tool interface, context, lifecycle, observability, verification, or governance — before choosing a fix (Ch 10, 12).

Task / trial — A *task* has defined inputs and success criteria; a *trial* is a single attempt at it (Ch 10).

Grader — A component that scores some aspect of a trial: code-based, model-based, or human (Ch 10).

Transcript (trace, trajectory) — The full record of a trial: every message, tool call, and result (Ch 10, 12).

Outcome — The final environmental state at the end of a trial, distinct from the agent's text response (Ch 10).

Capability eval / regression eval — Capability evals measure what an agent can newly do (low pass rates, climbing); regression evals protect what it already does reliably (near 100%) (Ch 10).

pass@k / pass^k — pass@k is the probability of at least one success in k attempts (rises with k); pass^k is the probability that *all* k trials succeed (falls with k) (see Foundations ch 13) (Ch 10).

Infrastructure noise — Variation in benchmark scores caused by the runtime's resource configuration rather than by model capability (Ch 11).

Long-Running Agents and the Field

Shift-change problem — The challenge that successive agent sessions arrive with no memory of prior ones, because context windows are limited (Ch 7).

Initializer agent — An agent that runs once to set up a project (init script, progress log, feature list) for the later coding-agent sessions to build on (Ch 7).

Managed agent — A platform-managed agent architecture that separates the model-side brain, execution-side hands, and durable session/event log so each can fail, reset, or migrate independently (Ch 7).

Brain / hands split — The managed-agent separation between decision-making context (brain) and replaceable execution environment (hands) (Ch 7).

Sprint contract — A file-based agreement between a generator and an evaluator agent on what will be built and how success is verified, settled before each build sprint (Ch 7).

Event log — An append-only record of messages, tool calls, results, approvals, and errors. Execution state can be derived from it, making agents easier to replay and debug (Ch 9).

Agent platform — Infrastructure beyond a local framework: durable workspaces, managed sandboxes, identity, billing, observability, evaluation, governance, and human handoff across many runs and users (Ch 9, 18).

Checkpoint / resume — A reliability pattern where an agent periodically saves enough state to restart after failure or context reset without losing work (Ch 7, 9).

Durable execution — An infrastructure guarantee that each workflow step is persisted so a crashed or interrupted agent resumes from the last recorded step; non-deterministic model/tool calls are recorded and replayed, not recomputed (Ch 7).

Time horizon — METR's capability metric: the human-task length a model completes with 50% reliability; the frontier value has roughly doubled every seven months (Ch 7, 18).

Stateless reducer — Modeling an agent as a pure fold over an event log, making it serializable, replayable, and testable (Ch 9).

Model-harness co-evolution — The coupling created when frontier models are post-trained with their harnesses in the loop, so that changing either side can degrade performance (Ch 12).

Span telemetry — Structured trace data represented as a tree of spans for model calls, tool calls, retrieval, context assembly, permissions, costs, and outcomes (Ch 12).

OpenTelemetry GenAI semantic conventions — An emerging standard schema of span and attribute names for LLM and agent telemetry (`invoke_agent` , `chat` , `execute_tool` spans), letting agent traces join the ordinary observability stack (Ch 12).

Trace-to-eval loop — Converting real production failures into redacted, reproducible regression cases with outcome assertions (Ch 12).

Meta-harness — Treating harness design itself as an optimization object: prompts, tools, retries, context policies, evaluators, and control loops are ablated or searched using eval feedback (Ch 12).

Cost-quality-speed trilemma — The tradeoff that stronger execution environments, observability, verification, and governance improve reliability but increase cost and latency (Ch 18).

Capability-control tradeoff — The design axis where more authority, tools, memory, and autonomy improve capability while expanding the control, provenance, and audit problem (Ch 18).

Ralph Wiggum loop — A hook that intercepts an agent's exit attempt and reinjects the original prompt in a clean context window, forcing it to continue against its goal (Ch 7, 8).

Loop engineering — Treating the agent loop itself as the unit of design — specifying the trigger, topology, verifier, and stop rules that surround the model so it can run unattended. The operator-facing view of the outer control loop (Ch 8).

Trigger (heartbeat) — What starts a loop pass without a human prompt: a schedule, a webhook, or another agent (Ch 8).

Verifier (maker-checker) — The fixed standard that decides "good enough," applied by an agent separate from the one that produced the work, so the maker cannot grade its own homework; the bottleneck of loop design (Ch 8).

Stop rule — An explicit condition that ends a loop — success, no-op, or ask-for-approval — plus the three hard stops that bound a runaway: a maximum iteration count, no-progress detection, and a budget ceiling (Ch 8).

Closed vs. open loop — A closed loop pins hard, checkable acceptance criteria up front and is safe to leave running; an open loop explores a fuzzy goal and needs an even stronger verifier or it ships confident garbage (Ch 8).

Instructions and Model Selection

Instruction hierarchy — The principle that instructions carry different authority by origin — system over developer over user over tool/retrieved content — so a lower-priority instruction cannot override a higher one. Prompt injection is a failure of this hierarchy (Ch 13).

Right altitude — The target specificity for a system prompt: concrete enough to steer behavior reliably, general enough to transfer across cases, avoiding both brittle hardcoded rules and vague guidance (Ch 13).

Model routing — Classifying a request's difficulty and dispatching easy ones to a cheap, weak model and hard ones to an expensive, strong one. Pays off only when the routing decision is much cheaper than the saving (Ch 14).

LLM cascade — Trying a cheap model first and escalating to a stronger one only when a verifier rejects the cheap answer. Matches strong-model accuracy at lower cost when the escalation signal is reliable (Ch 14).

Fallback — Failing over to an alternate model when the primary errors, times out, or is rate-limited, so the agent degrades gracefully (Ch 14).

AI gateway — An infrastructure component between the harness and model providers that presents one interface over many models and hosts routing, fallback, budgets, caching, and logging (e.g., LiteLLM, Portkey) (Ch 14, 17).

Reasoning model — A model post-trained, often with RL on verifiable rewards, to generate long internal reasoning before answering, spending inference tokens for better performance on hard tasks (Ch 14; *LLM Foundations* Ch 7–8).

Test-time compute — Spending more inference tokens, time, and money at answer time to do better on hard problems — a scaling axis distinct from larger models or more hardware (Ch 14).

Human Interaction

Permission fatigue — The degradation of oversight when an agent prompts for approval too often, training the human to rubber-stamp without reading (Ch 5, 15).

Mixed-initiative — An interaction style in which the system decides per action whether to act autonomously or defer to a human, managing the cost of interruption (Ch 15).

Approval as a tool call — Modeling human approval as a tool the agent invokes, so the request becomes a durable, replayable, auditable event that composes with suspend/resume (Ch 15).

Steering — Injecting a new instruction into a running agent so it is incorporated on the next turn, redirecting without losing session state (Ch 15).

Calibrated trust — The goal of the human interface: the human trusts the agent exactly as much as it deserves on a given task, achieved through transparency and grounded uncertainty rather than fluency (Ch 15).

Computer-Use Agents

Computer-use agent — An agent that operates software through its GUI — viewing screenshots and emitting cursor, keyboard, and navigation actions — rather than calling defined APIs (Ch 16).

Visual grounding — Translating an intention ("click Submit") into a concrete action (a click at specific coordinates); an error mode with no analog in API tools (Ch 16).

Set-of-Mark prompting — Overlaying numbered marks on candidate interactive elements so the model selects a discrete label instead of producing raw coordinates, improving grounding reliability (Ch 16).

Accessibility tree — A structured, semantic representation of a UI (roles, labels, states) built for assistive technology; often a more compact and exact screen encoding than raw pixels or DOM (Ch 16).

Cost and Operations

Per-task budget — An explicit ceiling on tokens, tool calls, or cost for a single agent run, after which the agent stops and asks rather than looping indefinitely (Ch 17).

Cost attribution — Attaching token and dollar cost to each span of a trace, turning "the agent is expensive" into a specific, fixable engineering finding (Ch 17).

Multi-tenancy / tenant isolation — Serving many users or organizations from one platform while preventing state bleed (context/memory/cache leaking across tenants) and authority bleed (acting with the wrong tenant's credentials) (Ch 17).

Canary rollout — Releasing a harness change to a fraction of traffic and watching production traces and outcome metrics before full deployment, catching cases the eval suite missed (Ch 17).

Semantic cache — A cache that serves *similar* (not just identical) requests by embedding the query and returning a stored response above a similarity threshold; can skip whole model calls but risks false hits (Ch 17).

AI management system (ISO/IEC 42001) — The first certifiable standard for governing an organization's AI: how to establish, operate, and continually improve an AI management system — the AI analogue of ISO 27001 (Ch 17).

EU AI Act — Regulation (EU) 2024/1689, the first comprehensive AI law; it classifies systems by risk tier and imposes binding obligations on high-risk uses (Ch 17).